

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

计算机组成原理



讲师：老汤

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

第五章：指令系统



第六章：中央处理器 (CPU)



说明了计算机能做什么



说明了计算机是如何做

第七章：输入输出 (I/O) 系统



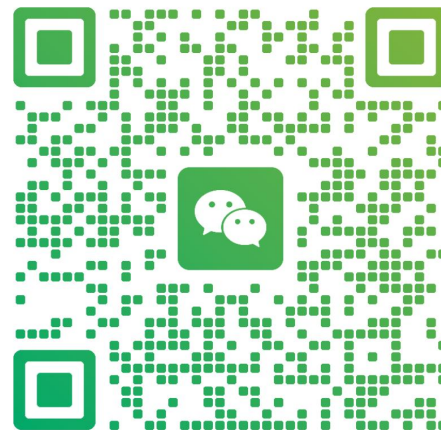
扫一扫上面的二维码图案，加我为朋友。

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

408 考研是如何考察这一章的？

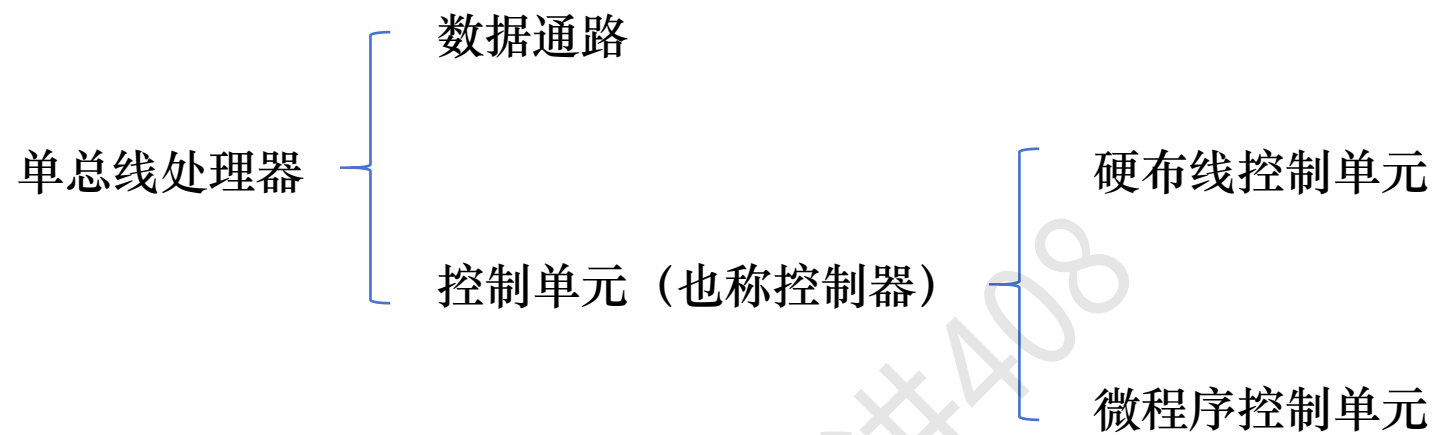


老汤
浙江 杭州

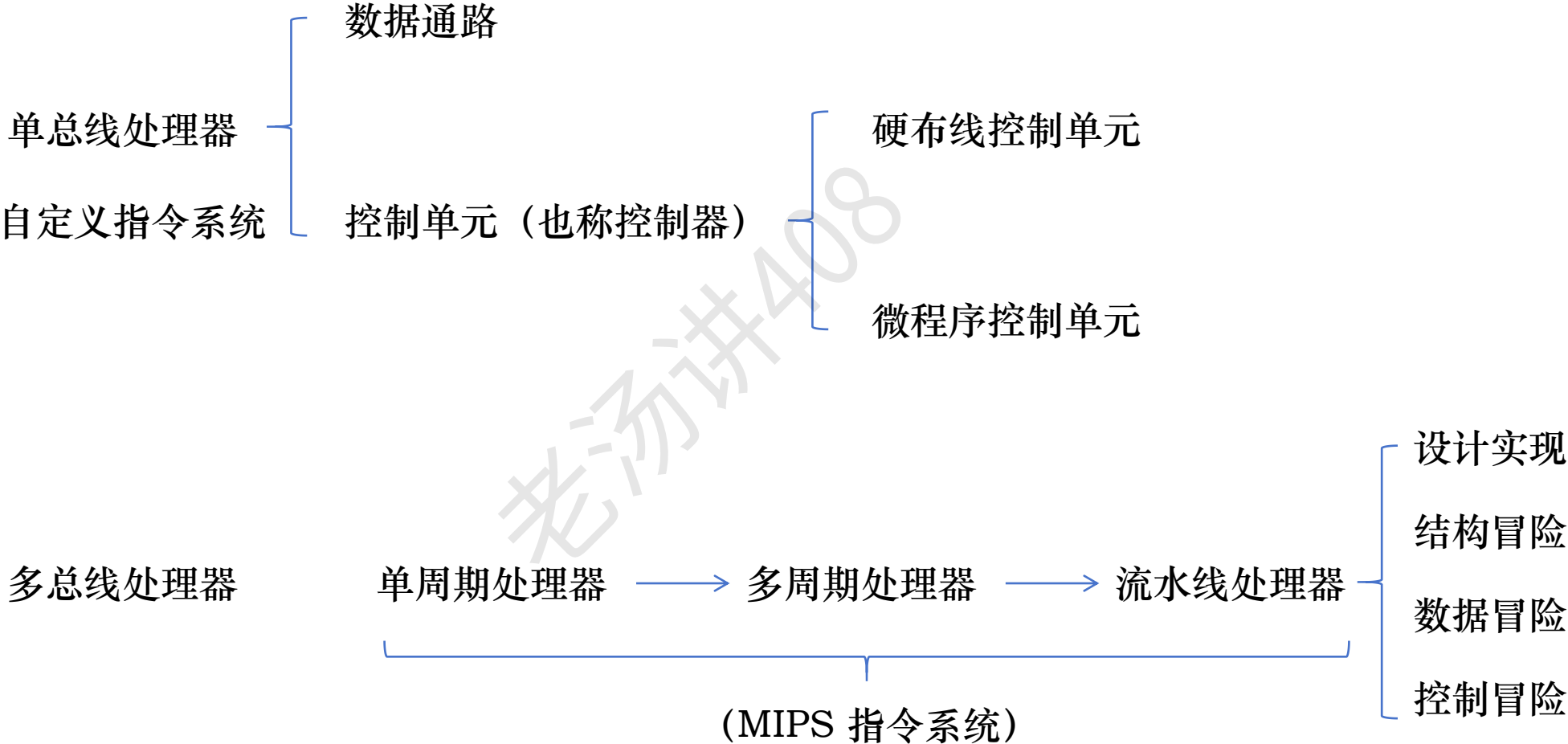


扫一扫上面的二维码图案，加我为朋友。

本章课程内容



本章课程内容



数字逻辑电路

组合逻辑电路（简称组合电路）

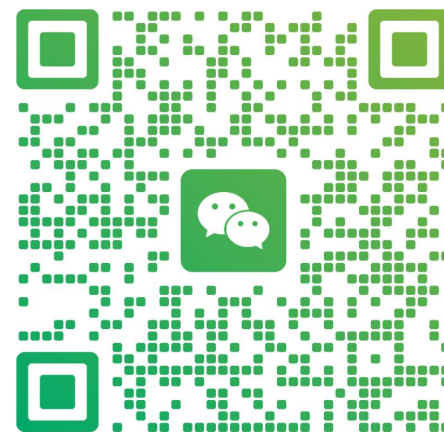
时序逻辑电路（简称时序电路）

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

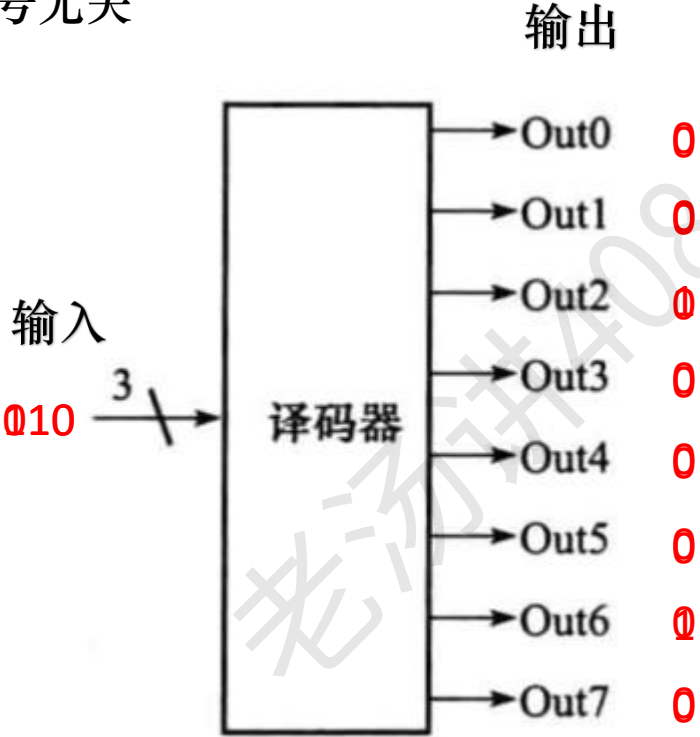
浙江 杭州



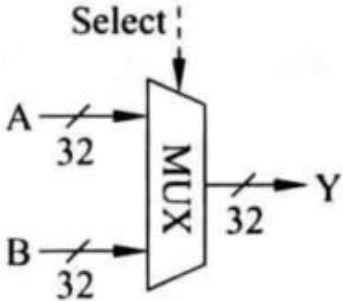
扫一扫上面的二维码图案，加我为朋友。

补一补电路基础：组合逻辑电路（操作元件）

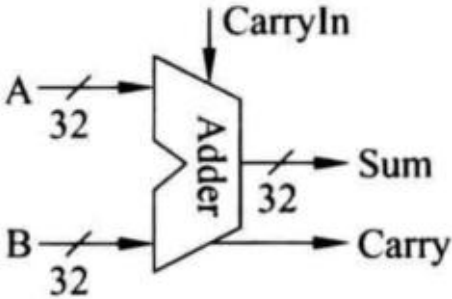
组合逻辑电路在任何时刻的输出信号的稳定值，仅仅与该时刻的输入信号有关，而与该时刻以前的输入信号无关



3-8 译码器



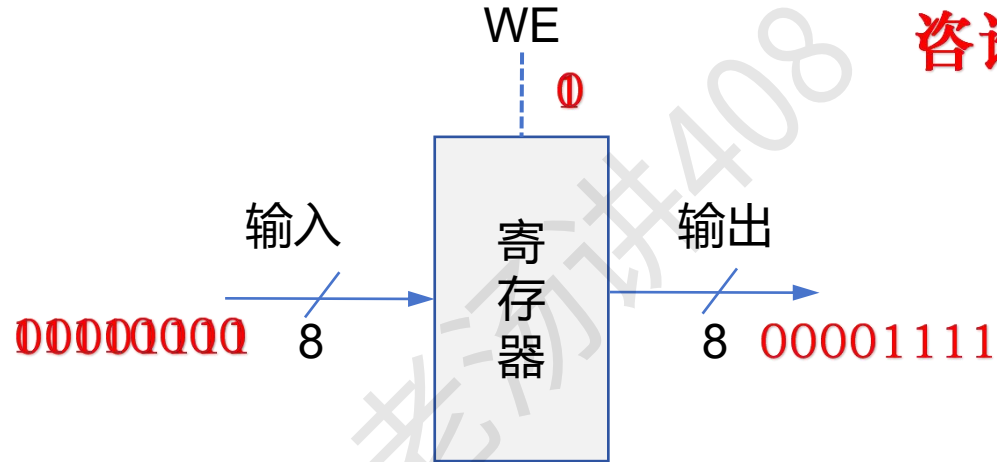
多路选择器



加法器

补一补电路基础：时序逻辑电路（状态元件）

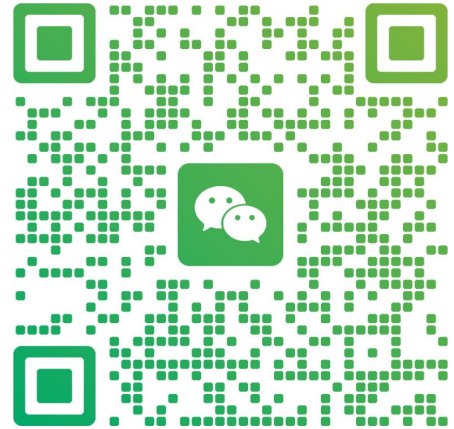
时序电路在任何时刻的输出稳态值，不仅与该时刻的输入信号有关，而且与该时刻以前的输入信号也有关



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



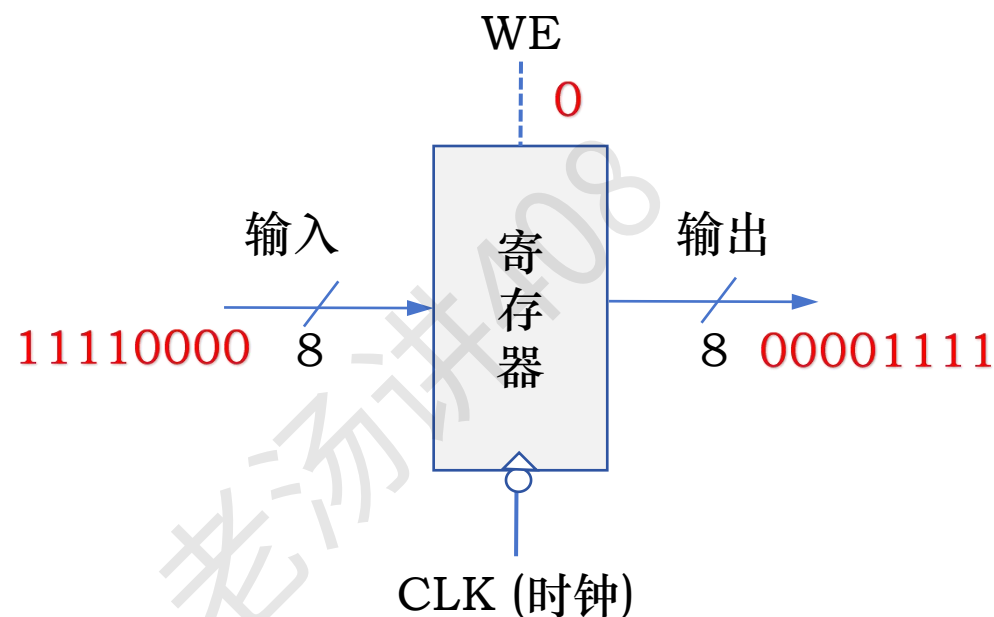
老汤
浙江 杭州



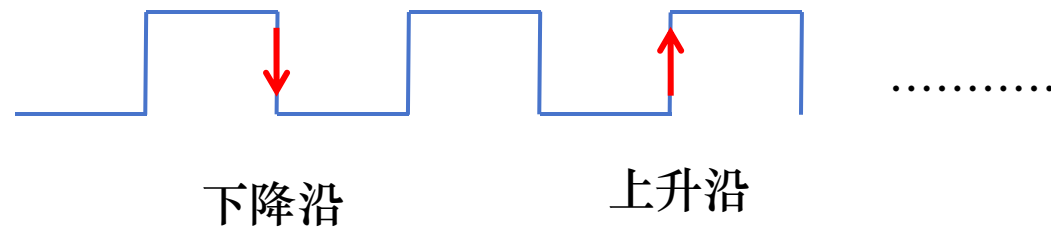
扫一扫上面的二维码图案，加我为朋友。

补一补电路基础：时序逻辑电路（状态元件）

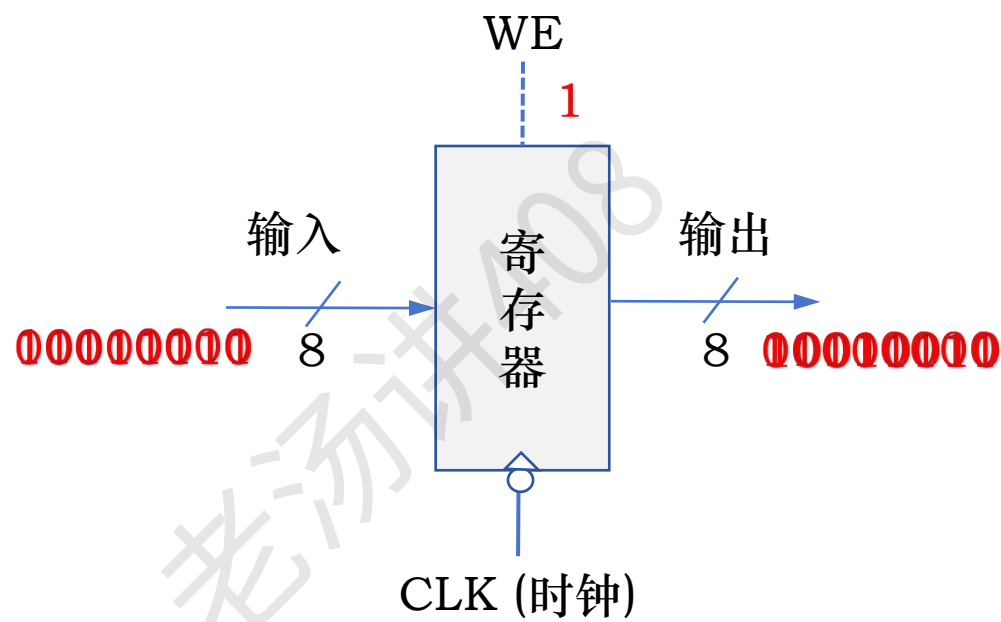
时序电路在任何时刻的输出稳态值，不仅与该时刻的输入信号有关，而且与该时刻以前的输入信号也有关



时钟信号



补一补电路基础：时序逻辑电路（状态元件）

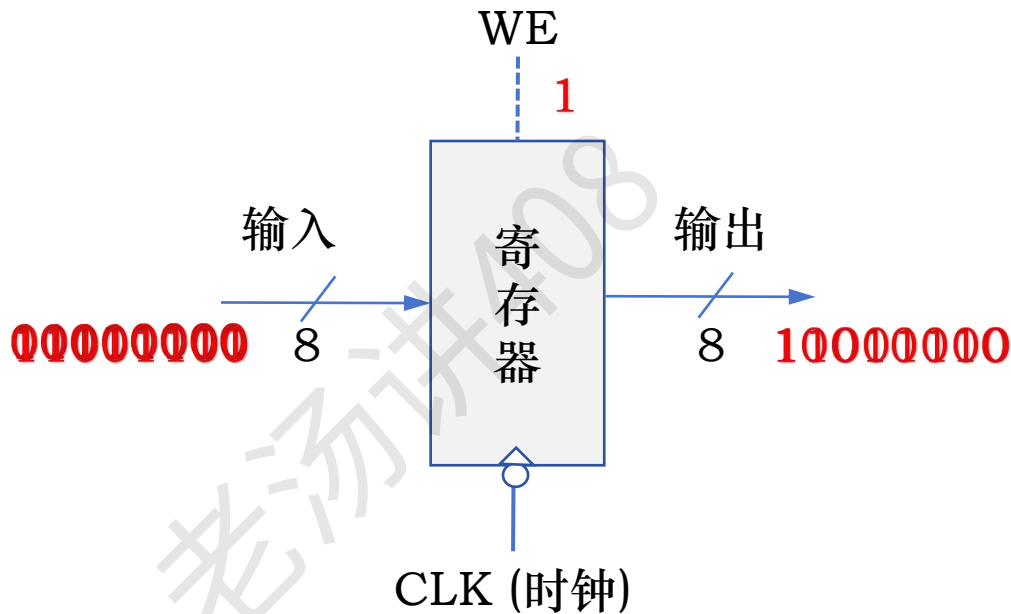


补一补电路基础：时序逻辑电路（状态元件）

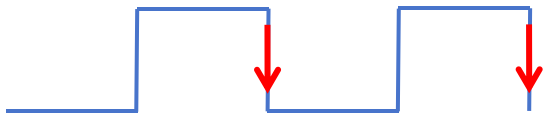
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。



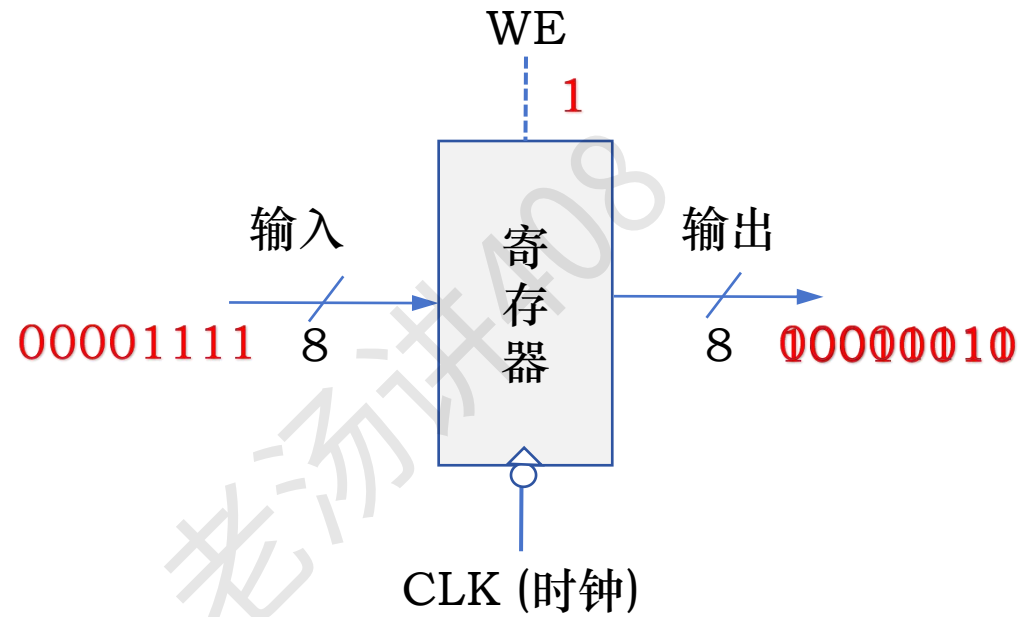
时钟信号



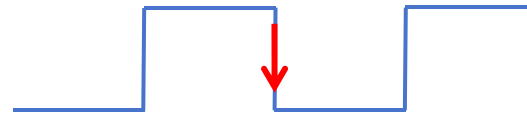
第一个好处是减少输出数据的震荡不稳定

补一补电路基础：时序逻辑电路（状态元件）

第二个好处是，当在同一个时钟周期内同时读写寄存器，不会出现因竞争而出现中间数据

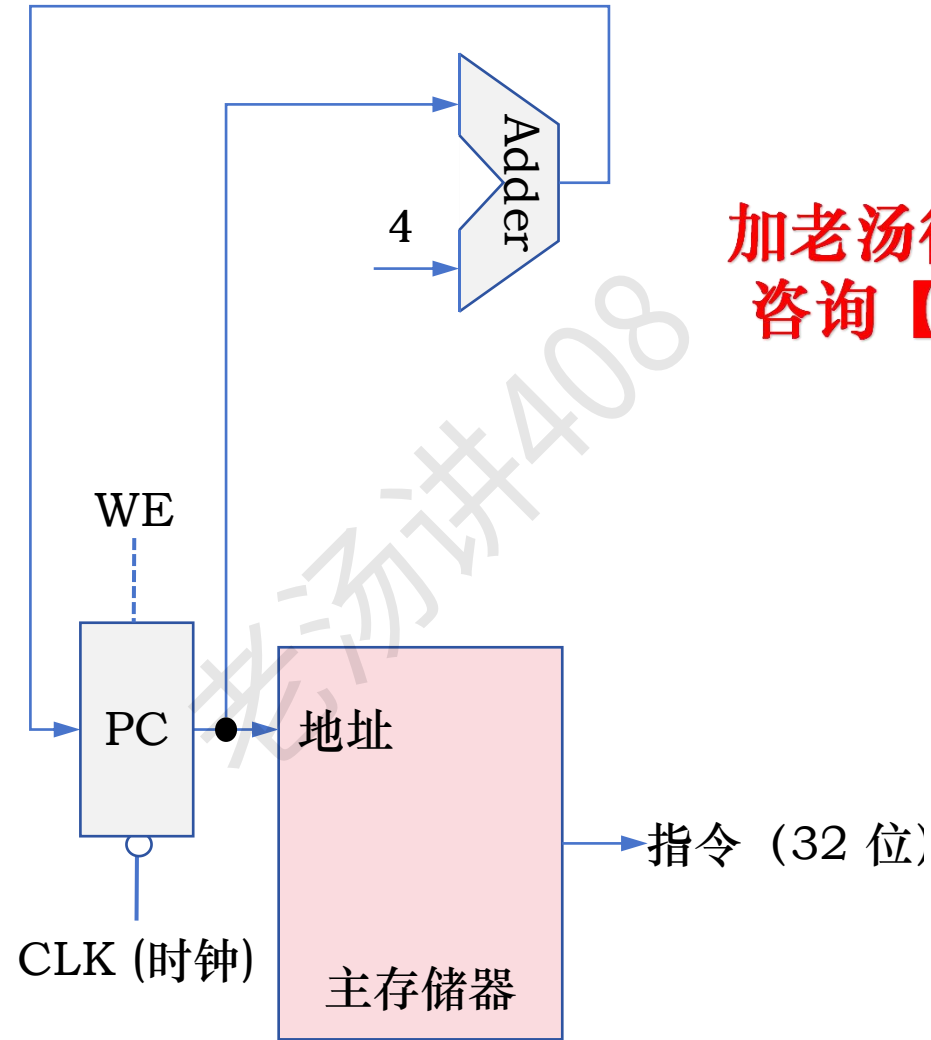


时钟信号



补一补电路基础：时序逻辑电路（状态元件）

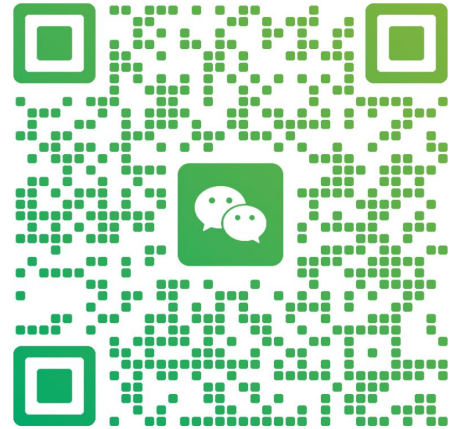
第二个好处是，当在同一个时钟周期内同时读写寄存器，不会出现因竞争而出现中间数据



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

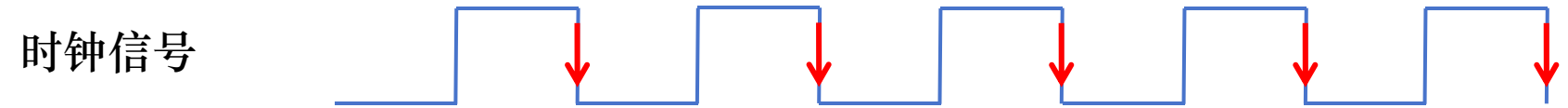
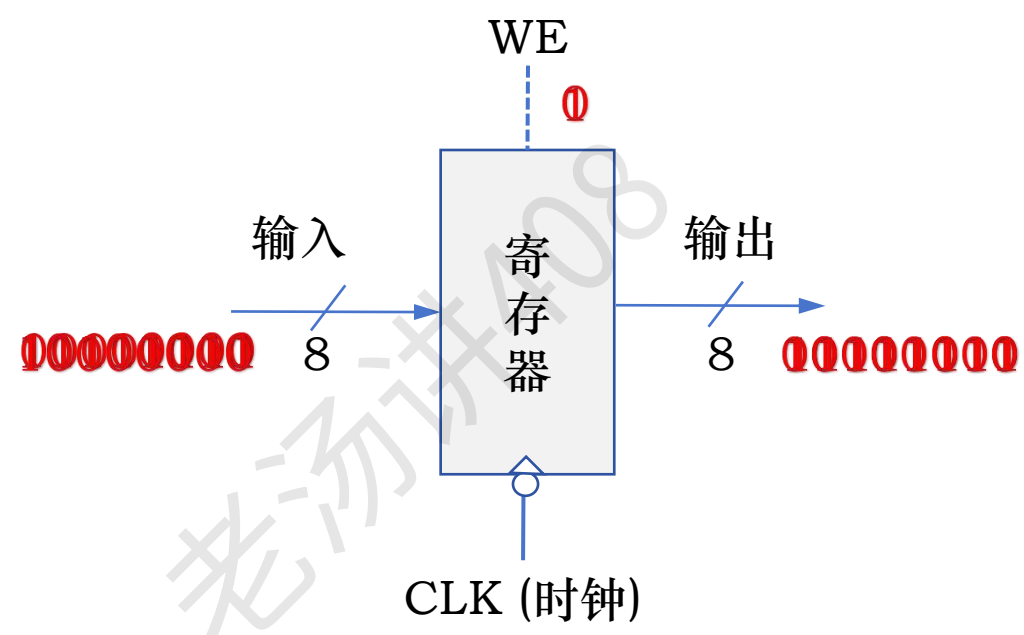


老汤
浙江 杭州

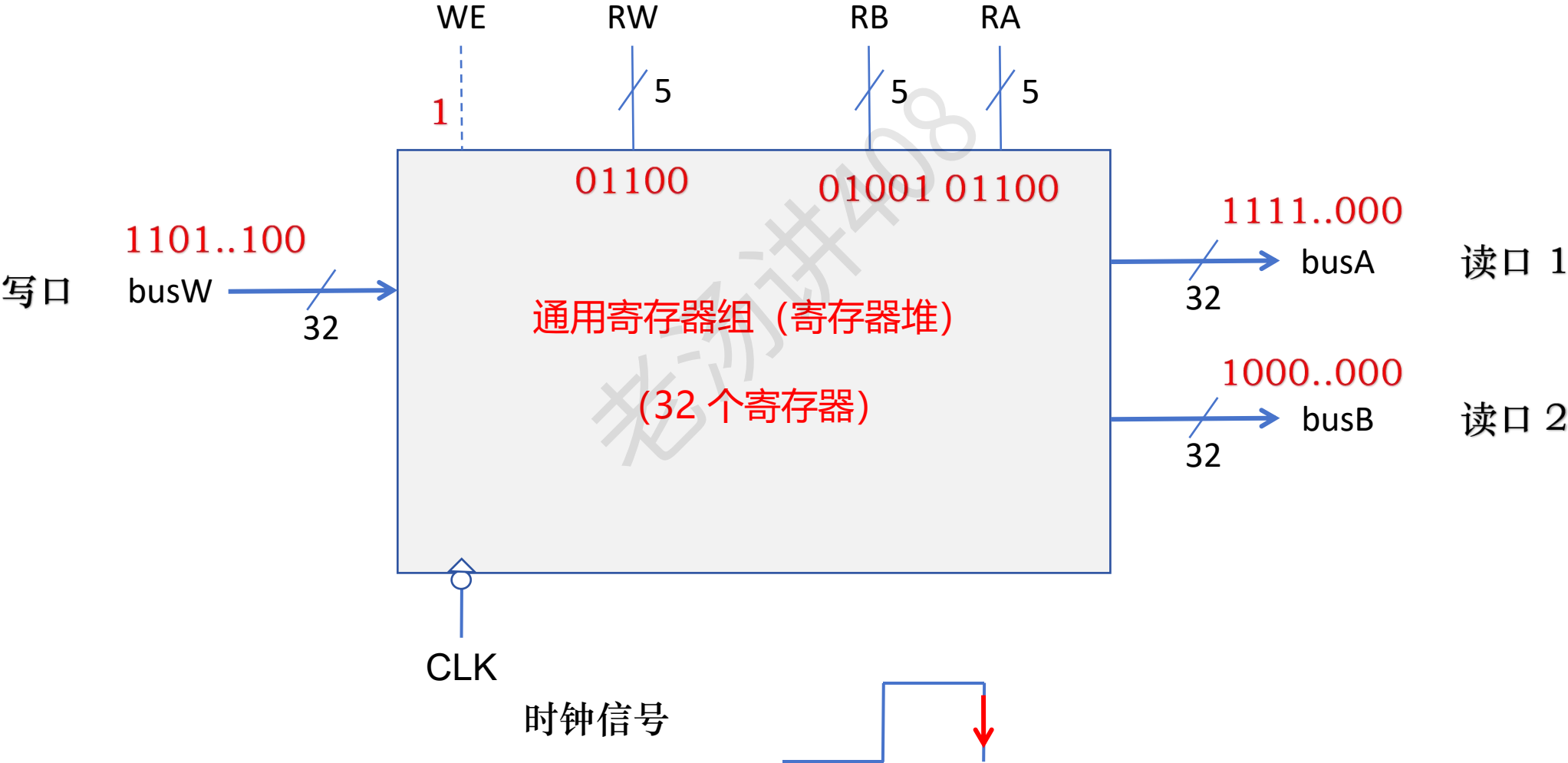


扫一扫上面的二维码图案，加我为朋友。

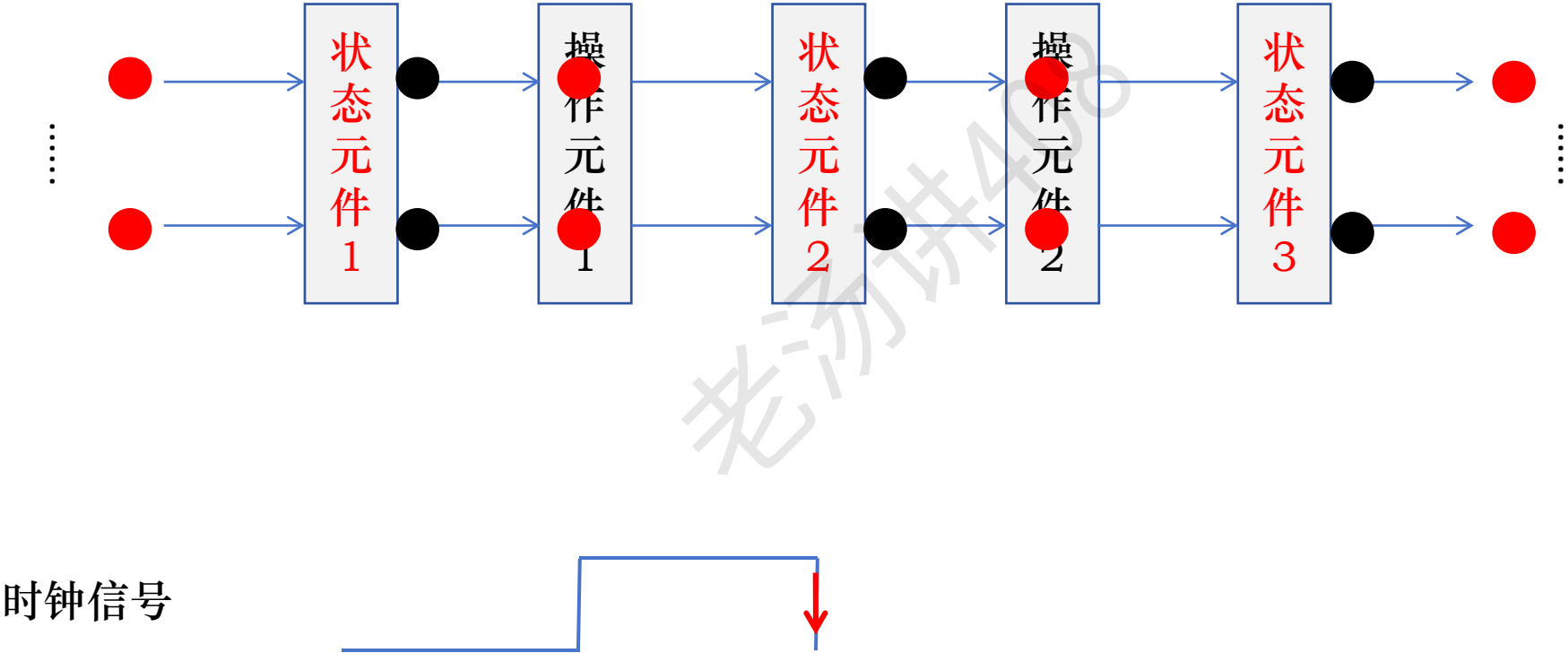
补一补电路基础：时序逻辑电路（状态元件）



补一补电路基础：通用寄存器组



数据通路和时序控制



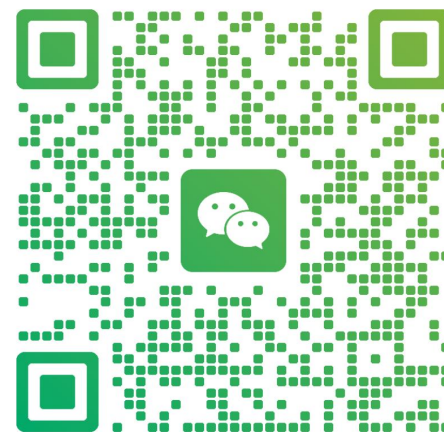
中央处理器 (Central Processing Unit, CPU)



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

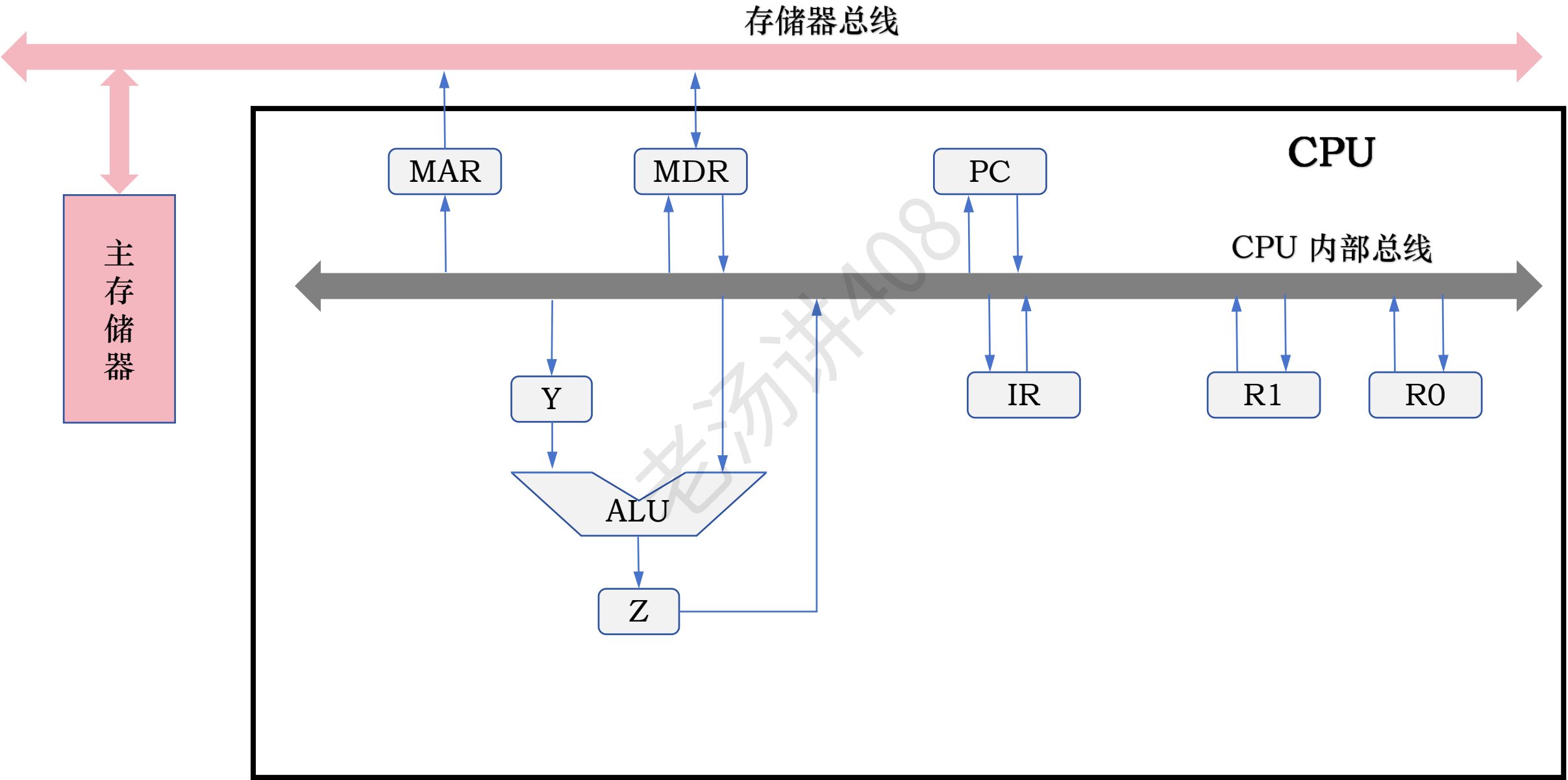


老汤
浙江 杭州

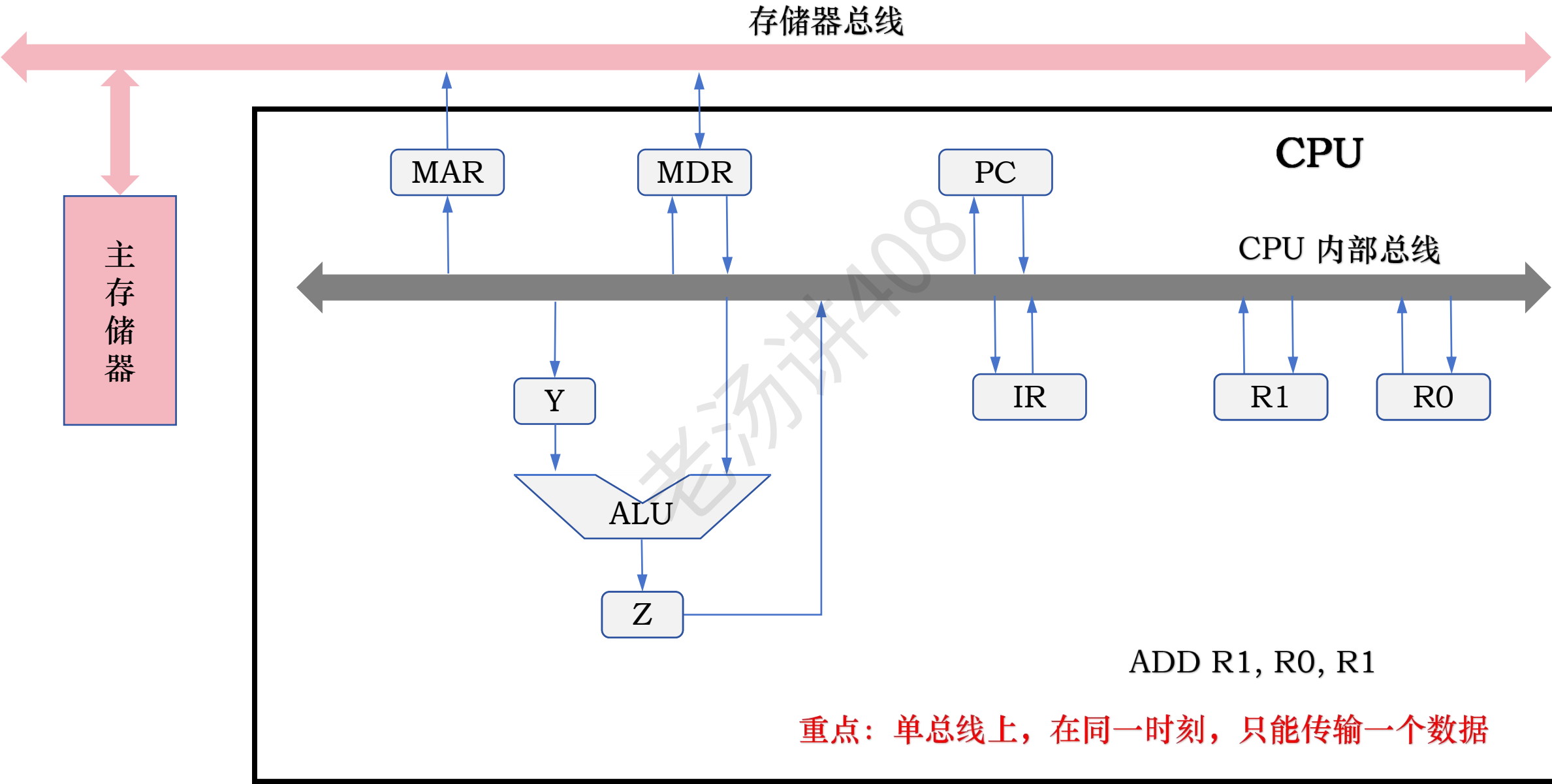


扫一扫上面的二维码图案，加我为朋友。

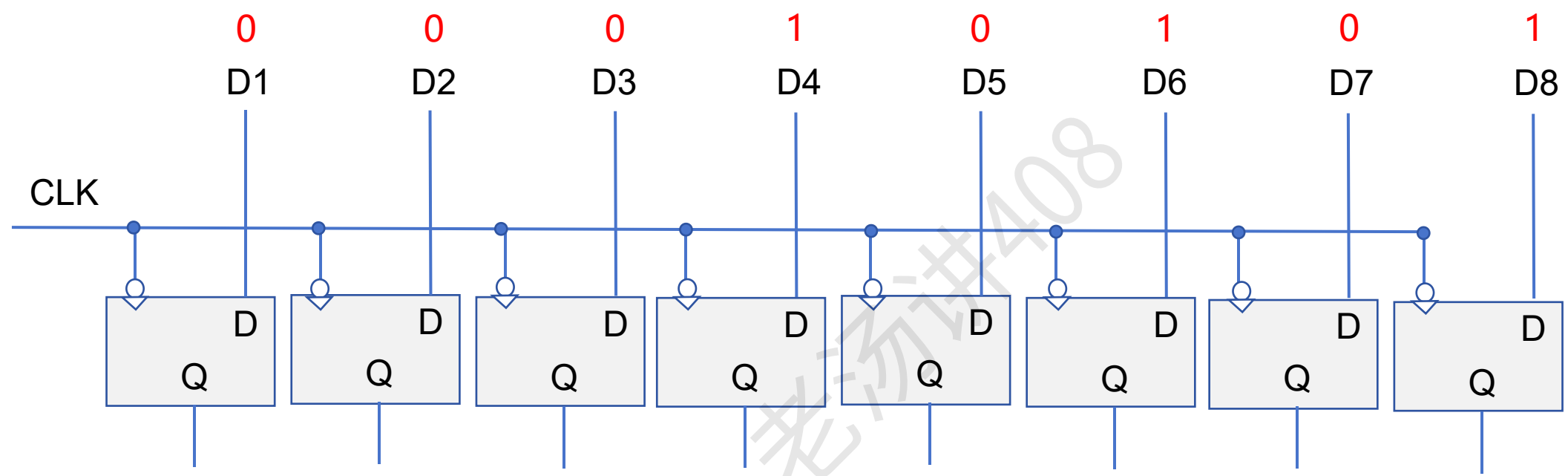
中央处理器（CPU）：什么是数据通路和控制单元呢？



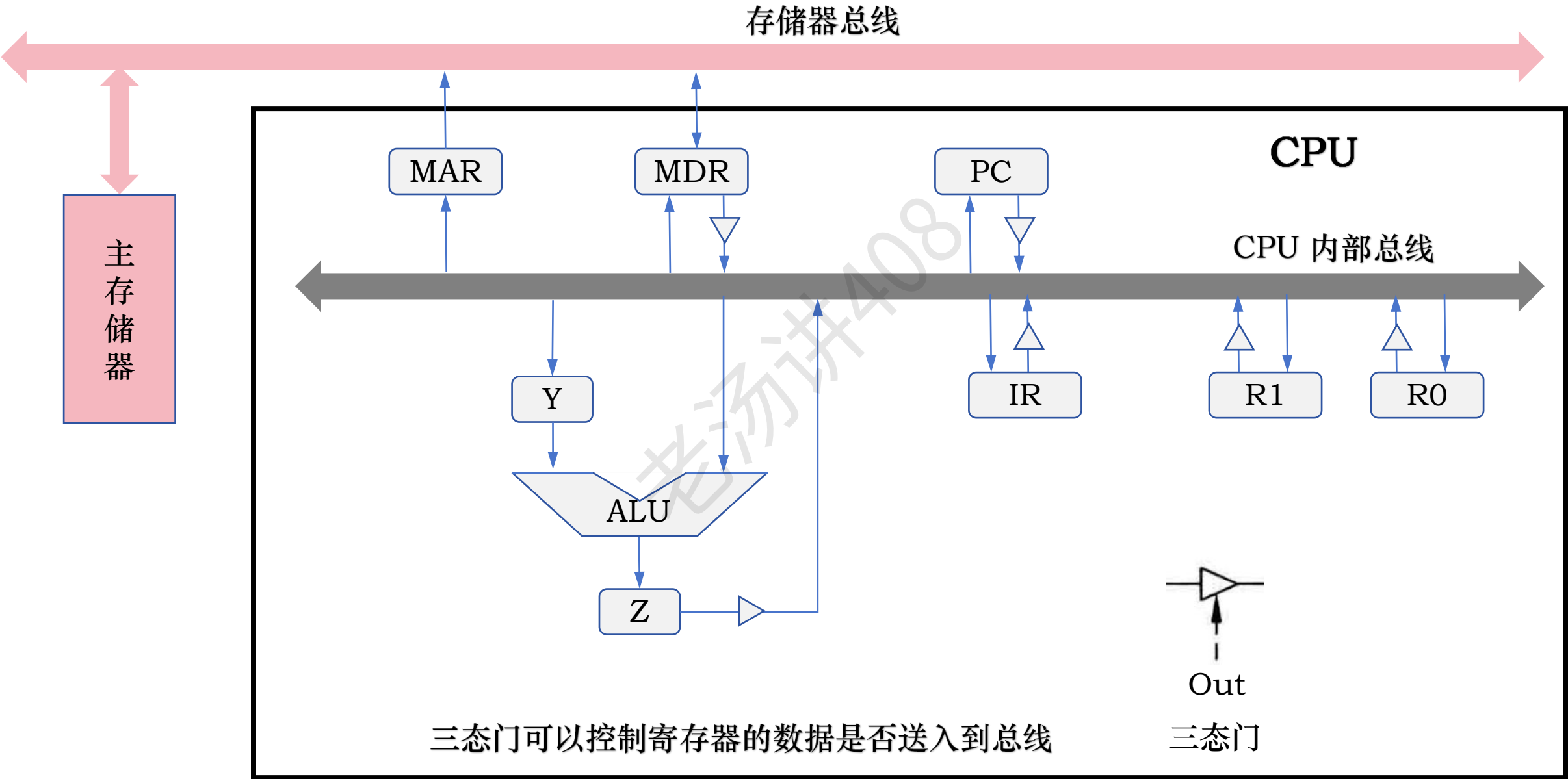
中央处理器（CPU）：什么是数据通路和控制单元呢？



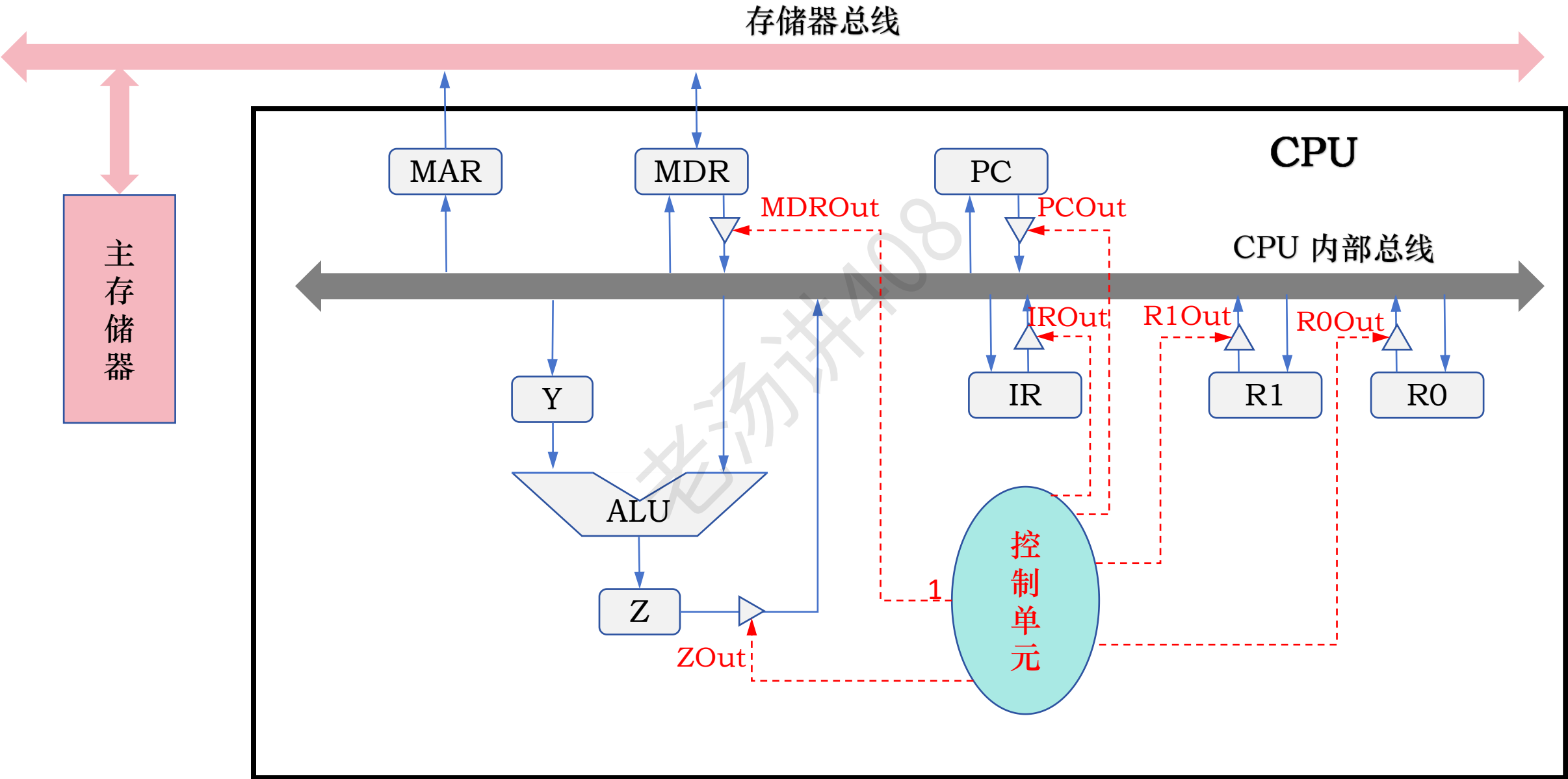
中央处理器（CPU）：什么是数据通路和控制单元呢？



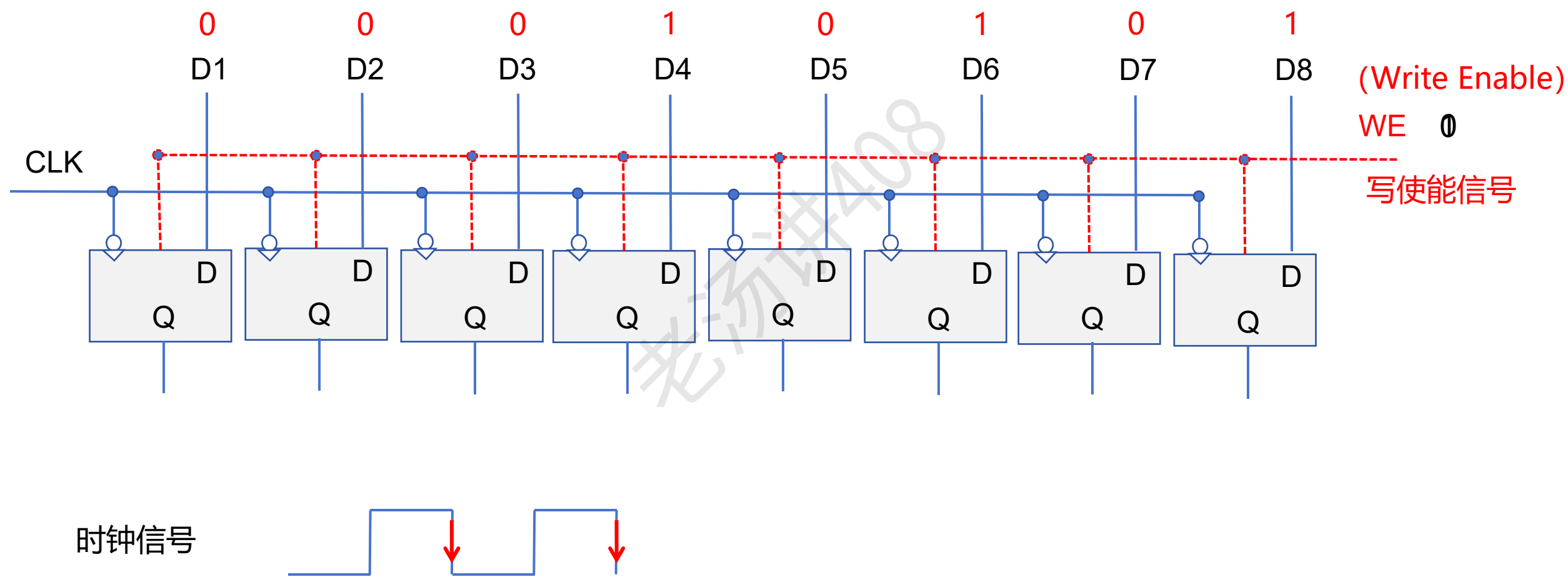
中央处理器（CPU）：什么是数据通路和控制单元呢？



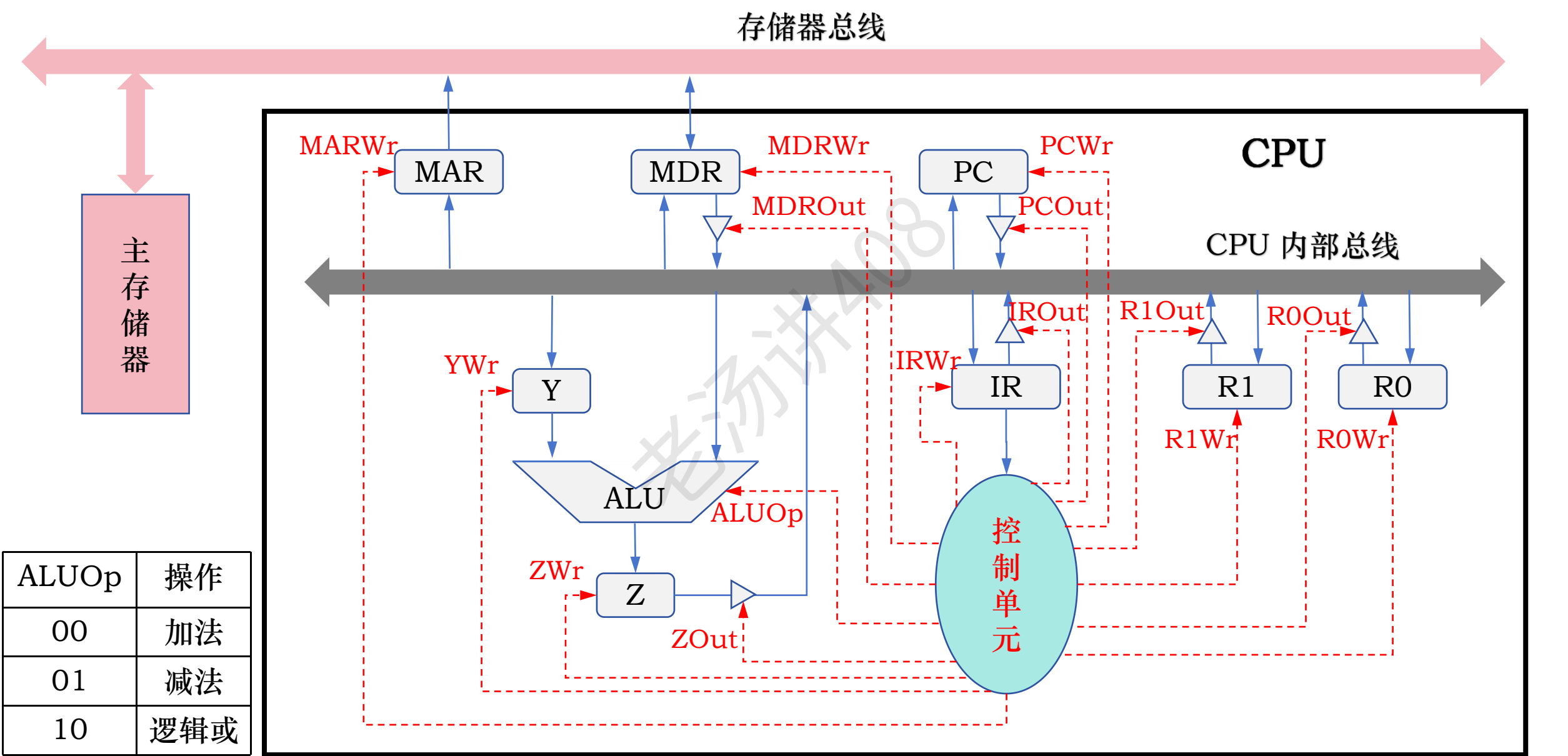
中央处理器（CPU）：什么是数据通路和控制单元呢？



中央处理器 (CPU)：什么是数据通路和控制单元呢？

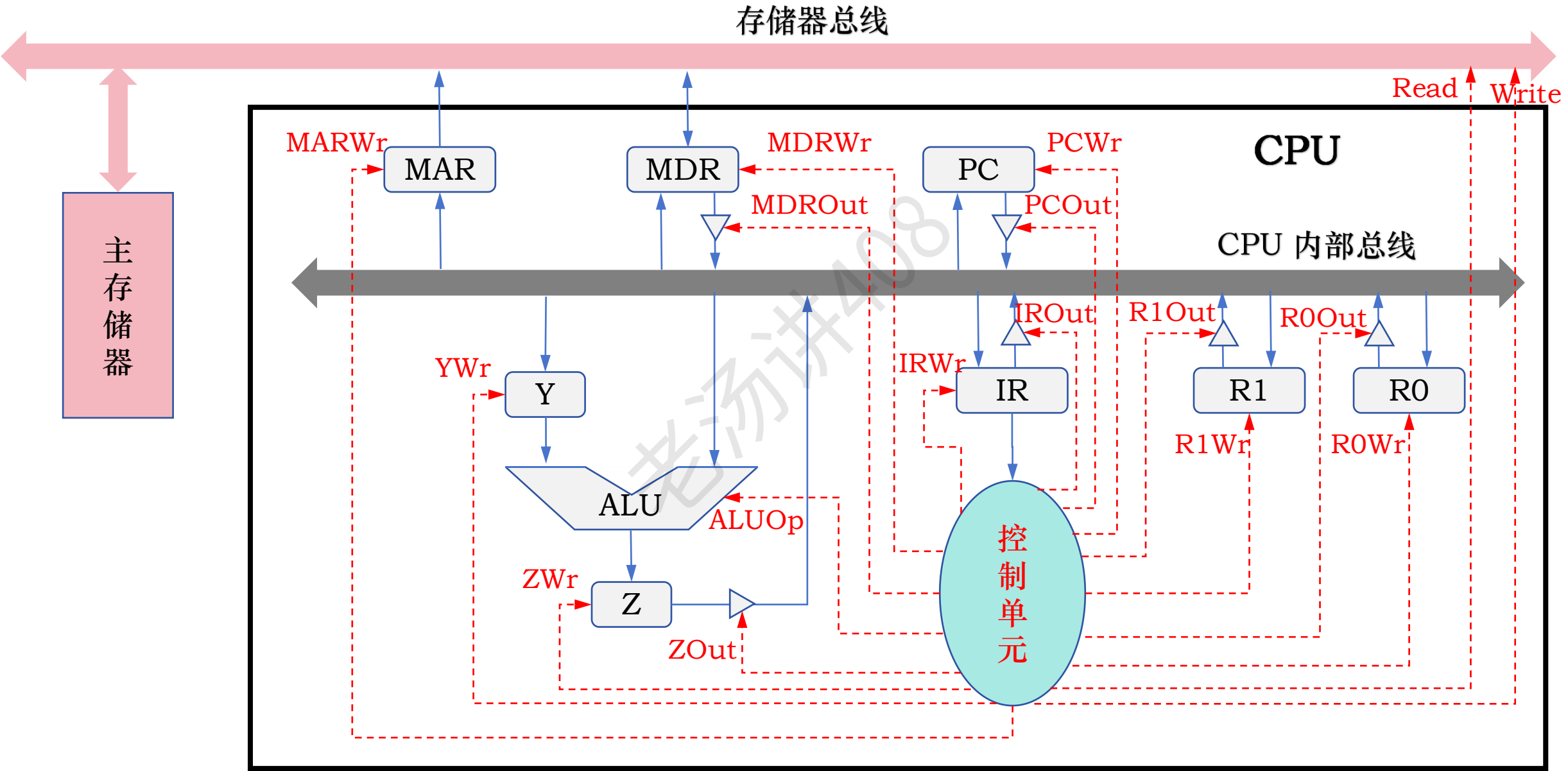


中央处理器（CPU）：什么是数据通路和控制单元呢？



ALUOp	操作
00	加法
01	减法
10	逻辑或

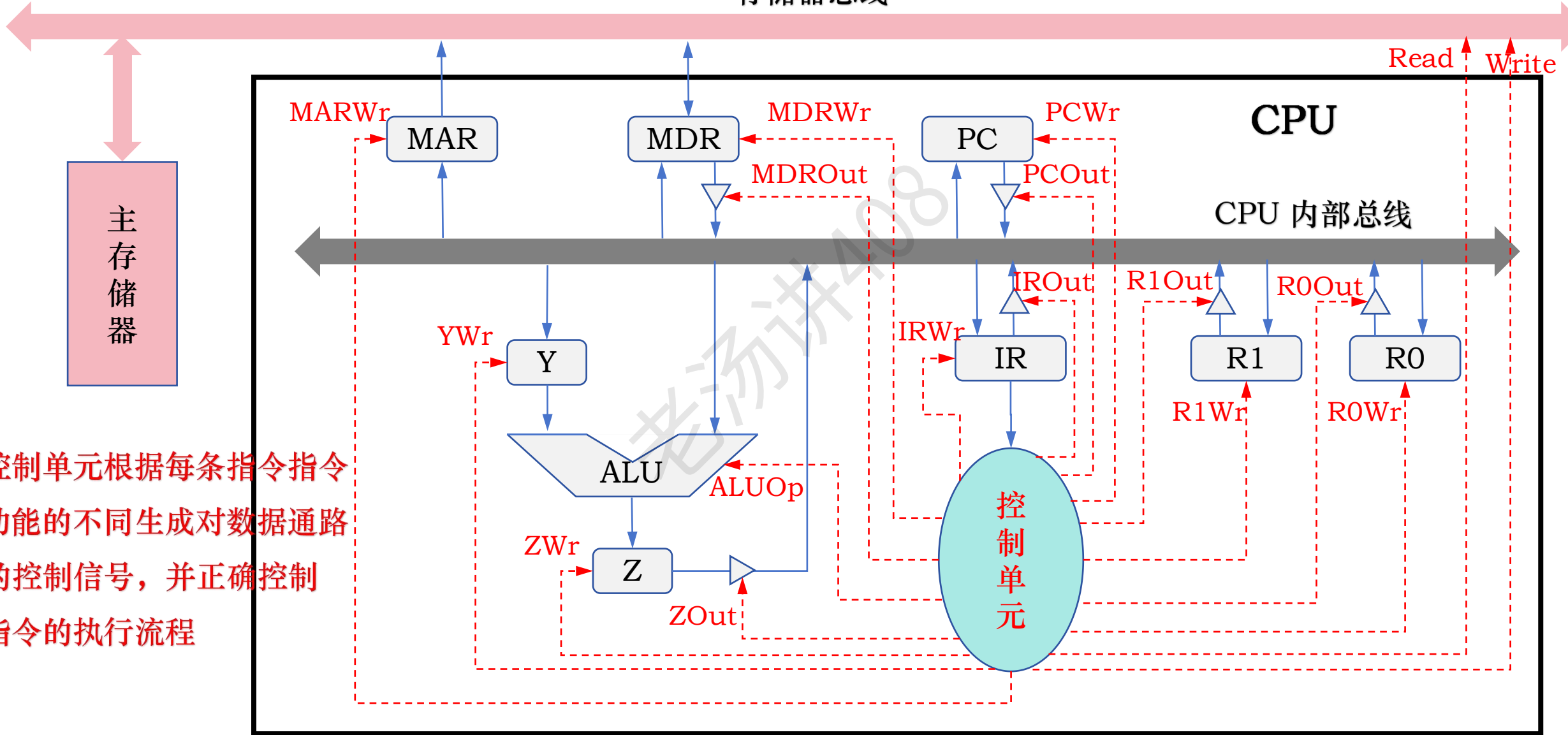
中央处理器（CPU）：什么是数据通路和控制单元呢？



中央处理器 (CPU)：什么是数据通路和控制单元呢？

寄存器，ALU Cache、MMU、
浮点运算逻辑部件、
异常和中断处理逻辑电路

通常将指令执行过程中数据所经过的路径，包括路径上的部件，称为数据通路
存储器总线



控制单元根据每条指令指令
功能的不同生成对数据通路
的控制信号，并正确控制
指令的执行流程

中央处理器 (CPU) : 单总线 CPU 如何执行指令?

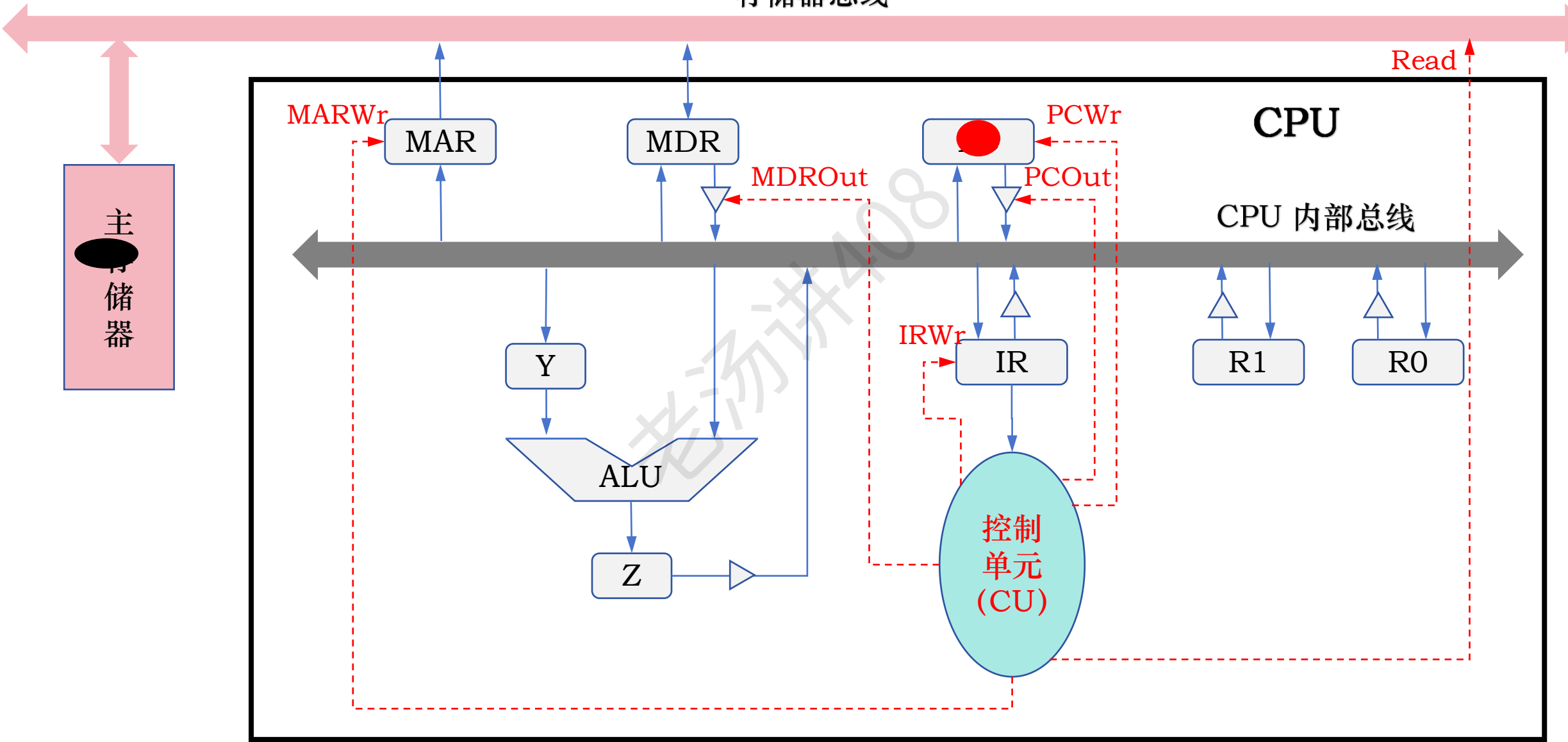
操作码	指令	功能
000	LOAD R0, Addr	$R[R0] \leftarrow \text{Mem}[\text{Addr}]$
001	LOAD R1, Addr	$R[R1] \leftarrow \text{Mem}[\text{Addr}]$
010	STROE Addr, R0	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
011	STROE Addr, R1	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
100	ADD R1, R0	$R[R1] \leftarrow R[R1] + R[R0]$
101	SUB R1, R0	$R[R1] \leftarrow R[R1] - R[R0]$
110	JUMP T	$[\text{PC}] \leftarrow T$

中央处理器 (CPU)：取指阶段

- ① $PC \rightarrow MAR$
- ② $1 \rightarrow \text{Read}$
- ③ $M[MAR] \rightarrow MDR$
- ④ $MDR \rightarrow IR$
- ⑤ $OP(IR) \rightarrow CU$
- ⑥ $PC + 1 \rightarrow PC$

微操作

存储器总线



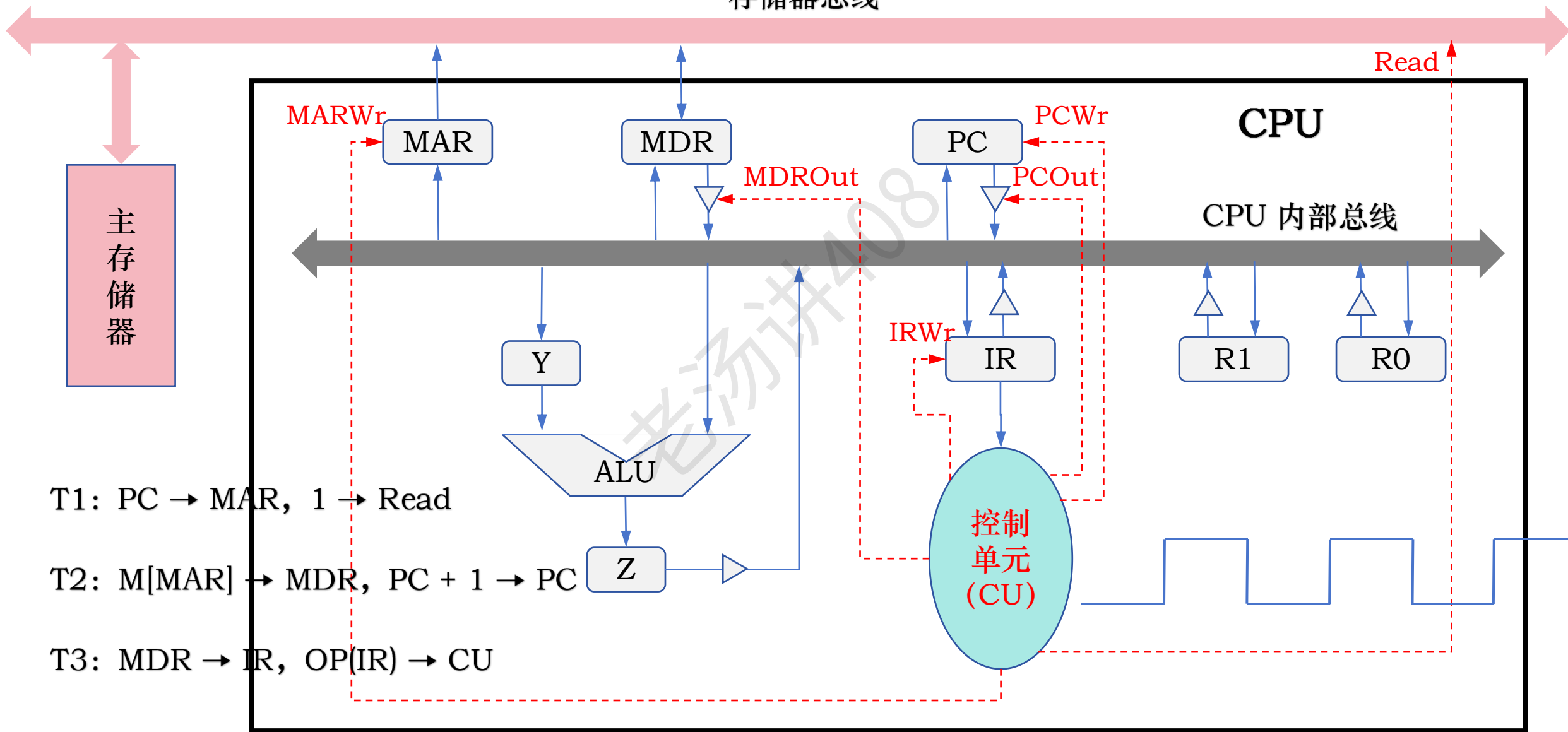
中央处理器 (CPU)：取指阶段

- ① 有些微操作的次序不能改变的，故安排微操作时钟周期时必须注意微操作的先后顺序
- ② 凡是被控制对象不同的微操作，若能在一个时钟周期内执行，应尽量可能安排在同一时钟周期内，以节省时间
- ③ 如果有些微操作所占时间不长，应该将它们安排在一个时钟周期内完成，并且允许这些微操作有先后顺序

中央处理器 (CPU)：取指阶段

- ① $PC \rightarrow MAR$
- ② $1 \rightarrow \text{Read}$
- ③ $M[MAR] \rightarrow MDR$
- ④ $MDR \rightarrow IR$
- ⑤ $OP(IR) \rightarrow CU$
- ⑥ $PC + 1 \rightarrow PC$

存储器总线



T1: $PC \rightarrow MAR, 1 \rightarrow \text{Read}$

T2: $M[MAR] \rightarrow MDR, PC + 1 \rightarrow PC$

T3: $MDR \rightarrow IR, OP(IR) \rightarrow CU$

中央处理器 (CPU)：取指阶段的三个时钟周期

状态: IFetch01

IFetch, 是 Instruction Fetch 的缩写, 意思是取指

PCOut=1
MARWr=1
Read=1
R1Out=0
...

T1: $PC \rightarrow MAR, 1 \rightarrow Read$

状态: IFetch02

PCWr=1
MARWr=0
Read=0
R1Out=0
...

T2: $M[MAR] \rightarrow MDR, PC + 1 \rightarrow PC$

状态: IFetch03

IRWr=1
MDROut=1
PCWr=0
R1Out=0
...

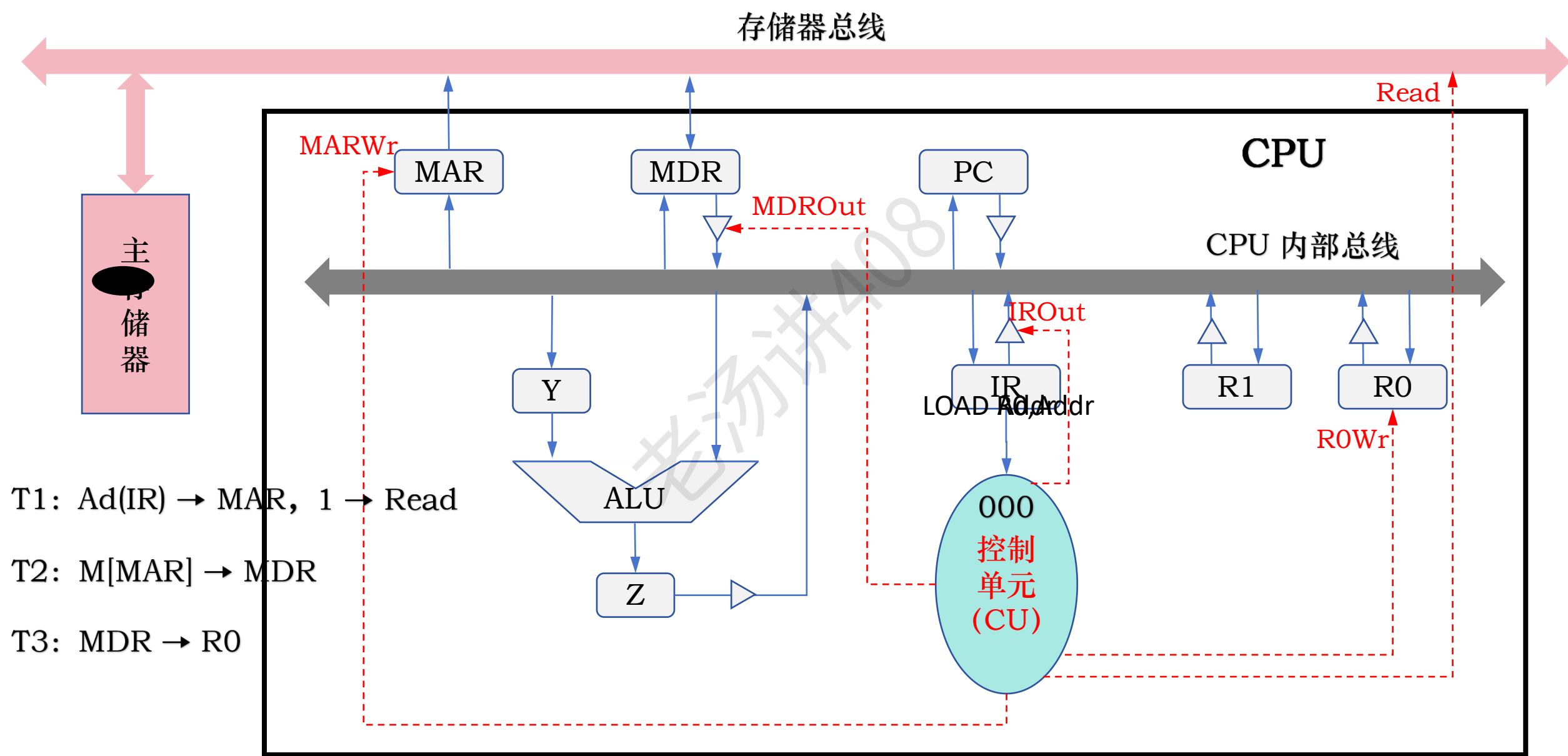
T3: $MDR \rightarrow IR, OP(IR) \rightarrow CU$

中央处理器 (CPU) : 单总线 CPU 如何执行指令?

操作码	指令	功能
000	LOAD R0, Addr	$R[R0] \leftarrow \text{Mem}[\text{Addr}]$
001	LOAD R1, Addr	$R[R1] \leftarrow \text{Mem}[\text{Addr}]$
010	STROE Addr, R0	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
011	STROE Addr, R1	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
100	ADD R1, R0	$R[R1] \leftarrow R[R1] + R[R0]$
101	SUB R1, R0	$R[R1] \leftarrow R[R1] - R[R0]$
110	JUMP T	$[\text{PC}] \leftarrow T$

中央处理器 (CPU) : LOAD Addr, R0

- ① Ad(IR) → MAR
- ② 1 → Read
- ③ M[MAR] → MDR
- ④ MDR → R0



T1: Ad(IR) → MAR, 1 → Read
T2: M[MAR] → MDR
T3: MDR → R0

中央处理器 (CPU) : LOAD 执行阶段的三个时钟周期

状态: LOAD01

IROut=1
MARWr=1
Read=1
R1Out=0
...

T1: Ad(IR) → MAR, 1 → Read

加老汤微信, 帮你 408 冲 120+
咨询【老汤 408 高分特训营】

状态: LOAD02

IROut=0
MARWr=0
Read=0
R1Out=0
...

T2: M[MAR] → MDR

状态: LOAD03

MDROut=1
R0Wr=1
Read=1
R1Out=0
...

T3: MDR → R0



老汤
浙江 杭州



扫一扫上面的二维码图案, 加我为朋友。

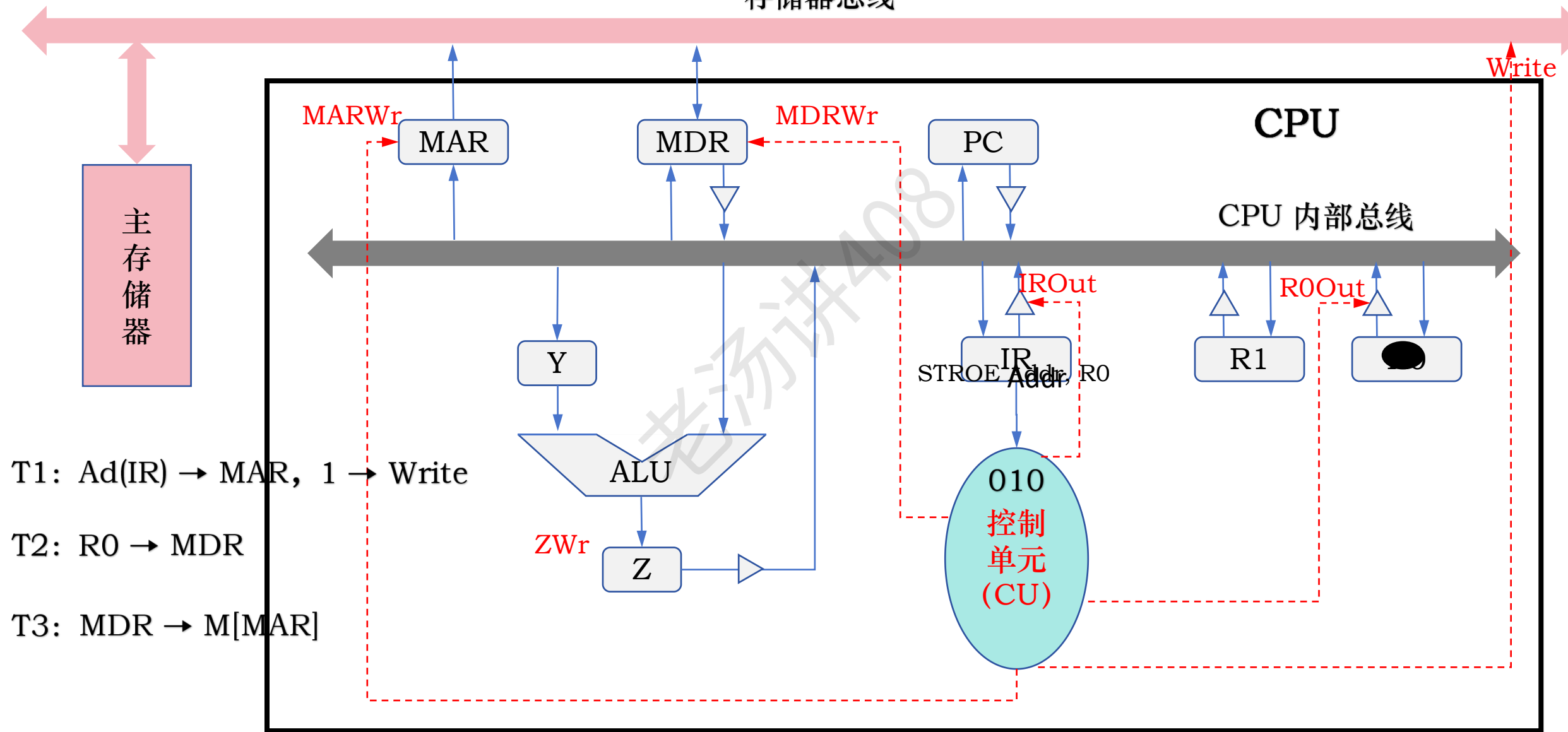
中央处理器 (CPU) : 单总线 CPU 如何执行指令?

操作码	指令	功能
000	LOAD R0, Addr	$R[R0] \leftarrow \text{Mem}[\text{Addr}]$
001	LOAD R1, Addr	$R[R1] \leftarrow \text{Mem}[\text{Addr}]$
010	STROE Addr, R0	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
011	STROE Addr, R1	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
100	ADD R1, R0	$R[R1] \leftarrow R[R1] + R[R0]$
101	SUB R1, R0	$R[R1] \leftarrow R[R1] - R[R0]$
110	JUMP T	$[\text{PC}] \leftarrow T$

中央处理器 (CPU) : STORE 指令

- ① $Ad(IR) \rightarrow MAR$ ② $1 \rightarrow Write$ ③ $R0 \rightarrow MDR$
④ $MDR \rightarrow M[MAR]$

存储器总线



- T1: $Ad(IR) \rightarrow MAR, 1 \rightarrow Write$
T2: $R0 \rightarrow MDR$
T3: $MDR \rightarrow M[MAR]$

中央处理器 (CPU) : STORE 执行阶段的三个时钟周期

状态: STORE01

IROut=1
MARWr=1
Write=1
R1Out=0
...

T1: Ad(IR) → MAR, 1 → Write

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

状态: STORE02

R0Out=1
MDRWr=1
Read=0
R1Out=0
...

T2: R0 → MDR

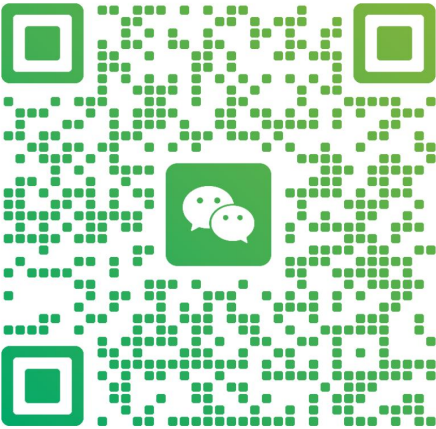
状态: STORE03

MDROut=0
R0Wr=0
Read=1
R1Out=0
...

T3: MDR → M[MAR]



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

中央处理器 (CPU) : 单总线 CPU 如何执行指令?

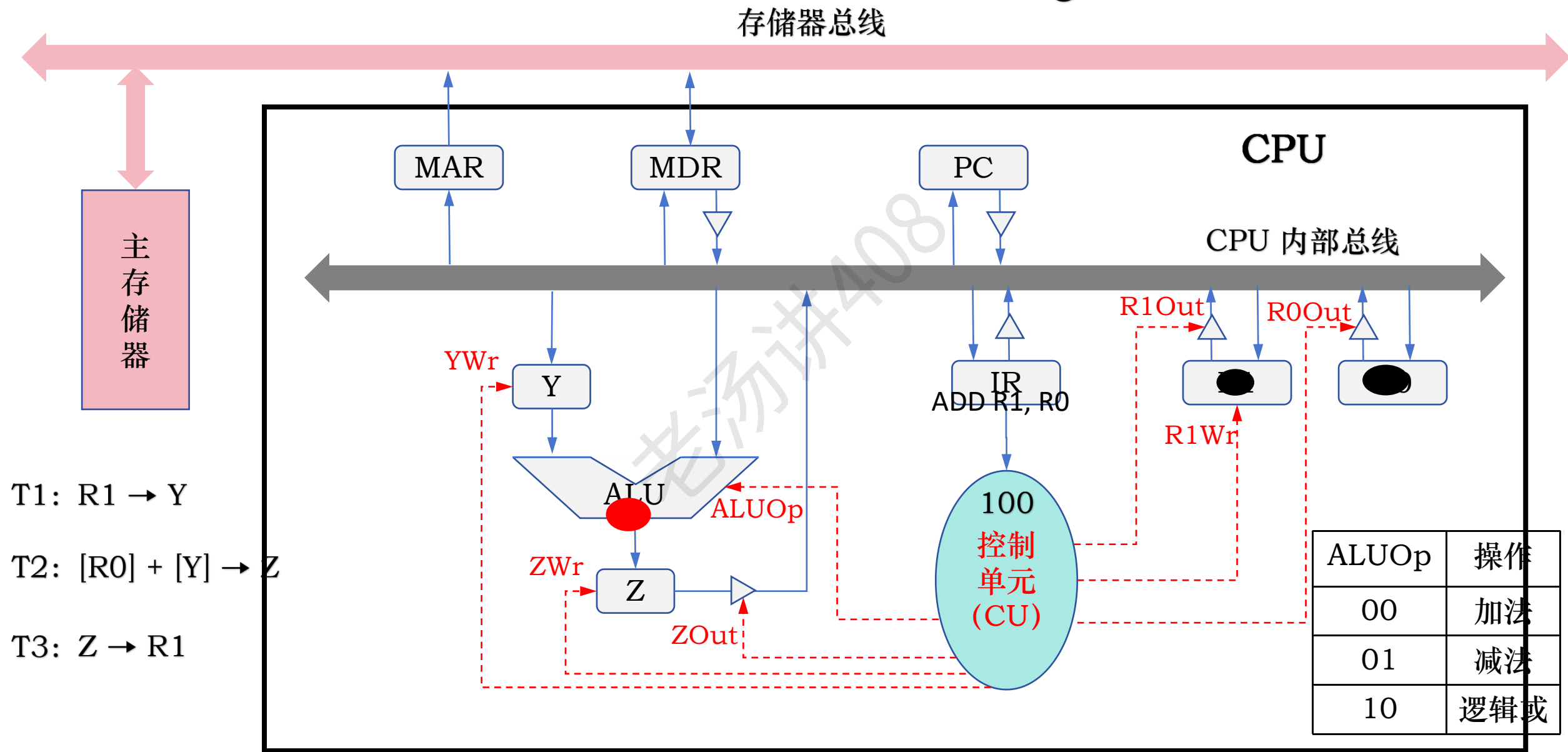
操作码	指令	功能
000	LOAD R0, Addr	$R[R0] \leftarrow \text{Mem}[\text{Addr}]$
001	LOAD R1, Addr	$R[R1] \leftarrow \text{Mem}[\text{Addr}]$
010	STROE Addr, R0	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
011	STROE Addr, R1	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
100	ADD R1, R0	$R[R1] \leftarrow R[R1] + R[R0]$
101	SUB R1, R0	$R[R1] \leftarrow R[R1] - R[R0]$
110	JUMP T	$[\text{PC}] \leftarrow T$

中央处理器 (CPU) : ADD 和 SUB 指令

① $R1 \rightarrow Y$

② $[R0] + [Y] \rightarrow Z$

③ $Z \rightarrow R1$



中央处理器 (CPU) : ADD 和 SUB 执行阶段的三个时钟周期

状态: ADD01

R1Out=1
YWr=1
Read=0
...

T1: R1 → Y

状态: ADD02

R0Out=1
ZWr=1
ALUOp=00
R1Out=0
...

T2: [R0] + [Y] → Z

状态: ADD03

ZOut=1
R1Wr=1
Read=0
R1Out=0
...

T3: Z → R1

状态: SUB01

R1Out=1
YWr=1
Read=0
...

状态: SUB02

R0Out=1
ZWr=1
ALUOp=01
R1Out=0
...

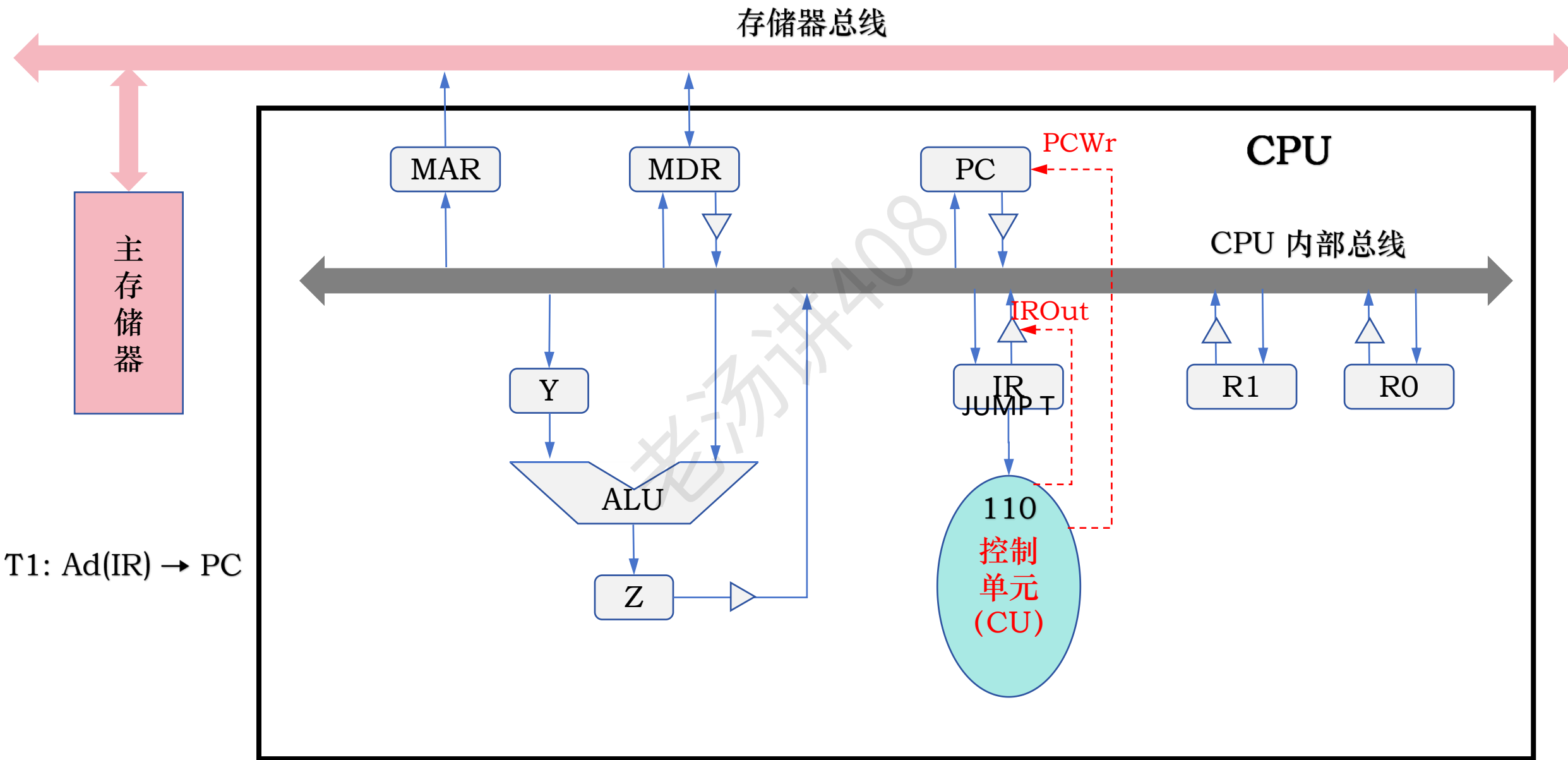
状态: SUB03

ZOut=1
R1Wr=1
Read=0
R1Out=0
...

中央处理器 (CPU) : 单总线 CPU 如何执行指令?

操作码	指令	功能
000	LOAD R0, Addr	$R[R0] \leftarrow \text{Mem}[\text{Addr}]$
001	LOAD R1, Addr	$R[R1] \leftarrow \text{Mem}[\text{Addr}]$
010	STROE Addr, R0	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
011	STROE Addr, R1	$\text{Mem}[\text{Addr}] \leftarrow R[R0]$
100	ADD R1, R0	$R[R1] \leftarrow R[R1] + R[R0]$
101	SUB R1, R0	$R[R1] \leftarrow R[R1] - R[R0]$
110	JUMP T	$[\text{PC}] \leftarrow T$

中央处理器 (CPU) : JUMP 指令执行阶段



中央处理器 (CPU) : JUMP 执行阶段的一个时钟周期

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

JUMP01

IROut=1
PCWr=1
Read=0
...

T1: Ad(IR) → PC



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

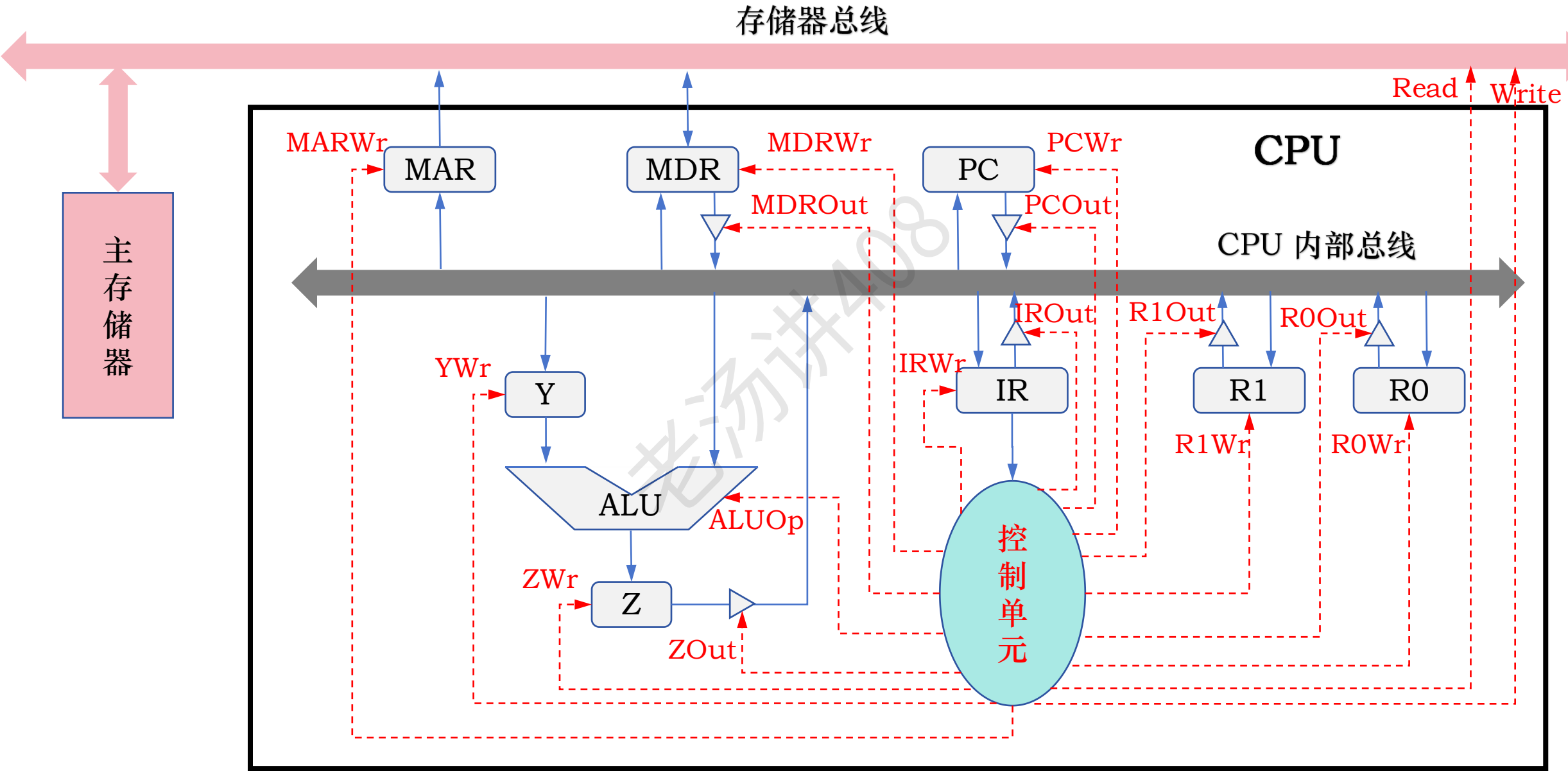
中央处理器 (CPU) : 指令周期

CPU 每取出并执行一条指令所需的全部时间称为指令周期

取指阶段完成取指令和分析指令的操作，又称取指周期；

执行阶段完成执行指令的操作，又称执行周期

中央处理器 (CPU)：控制单元的设计实现



中央处理器（CPU）：控制单元的设计实现

① 组合逻辑控制单元（组合逻辑控制器）

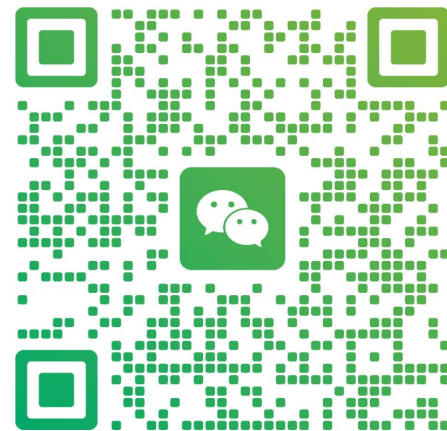
② 微程序控制单元（微程序控制器）

加老汤微信，帮你 408 冲 120+



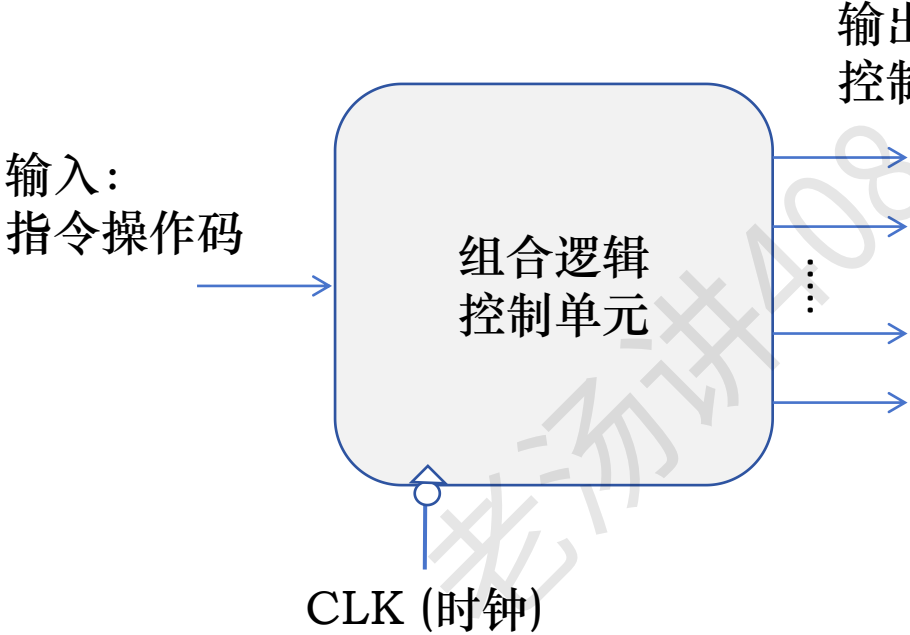
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

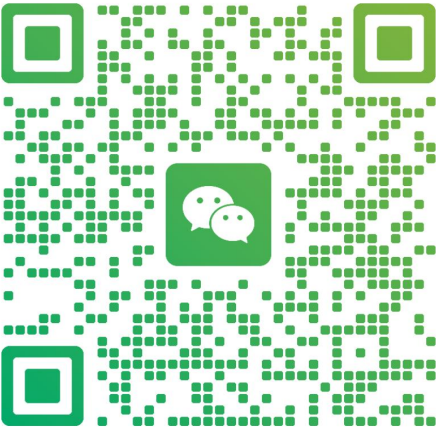
中央处理器（CPU）：控制单元的设计实现 - 组合逻辑控制单元



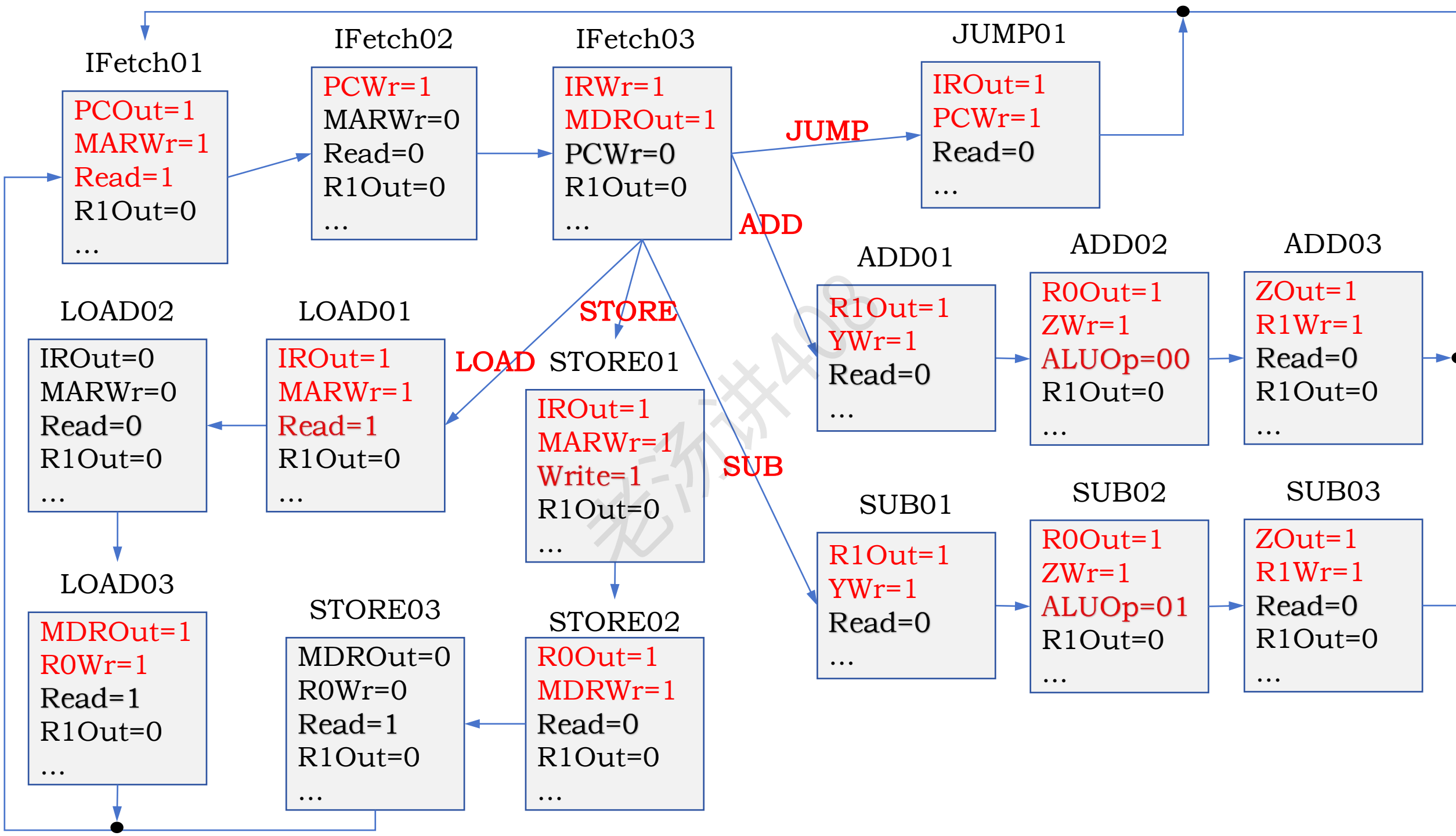
**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

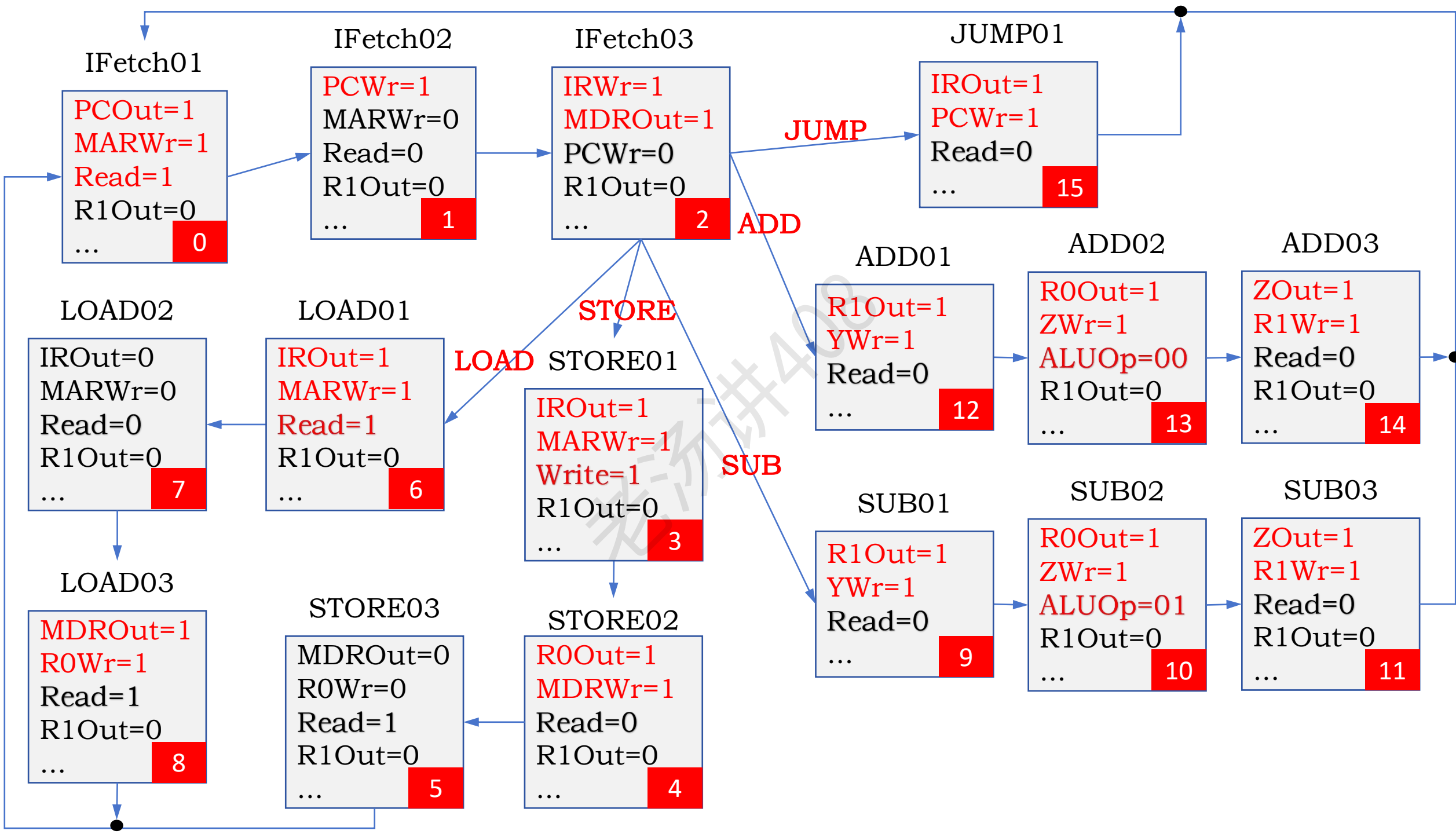


老汤
浙江 杭州



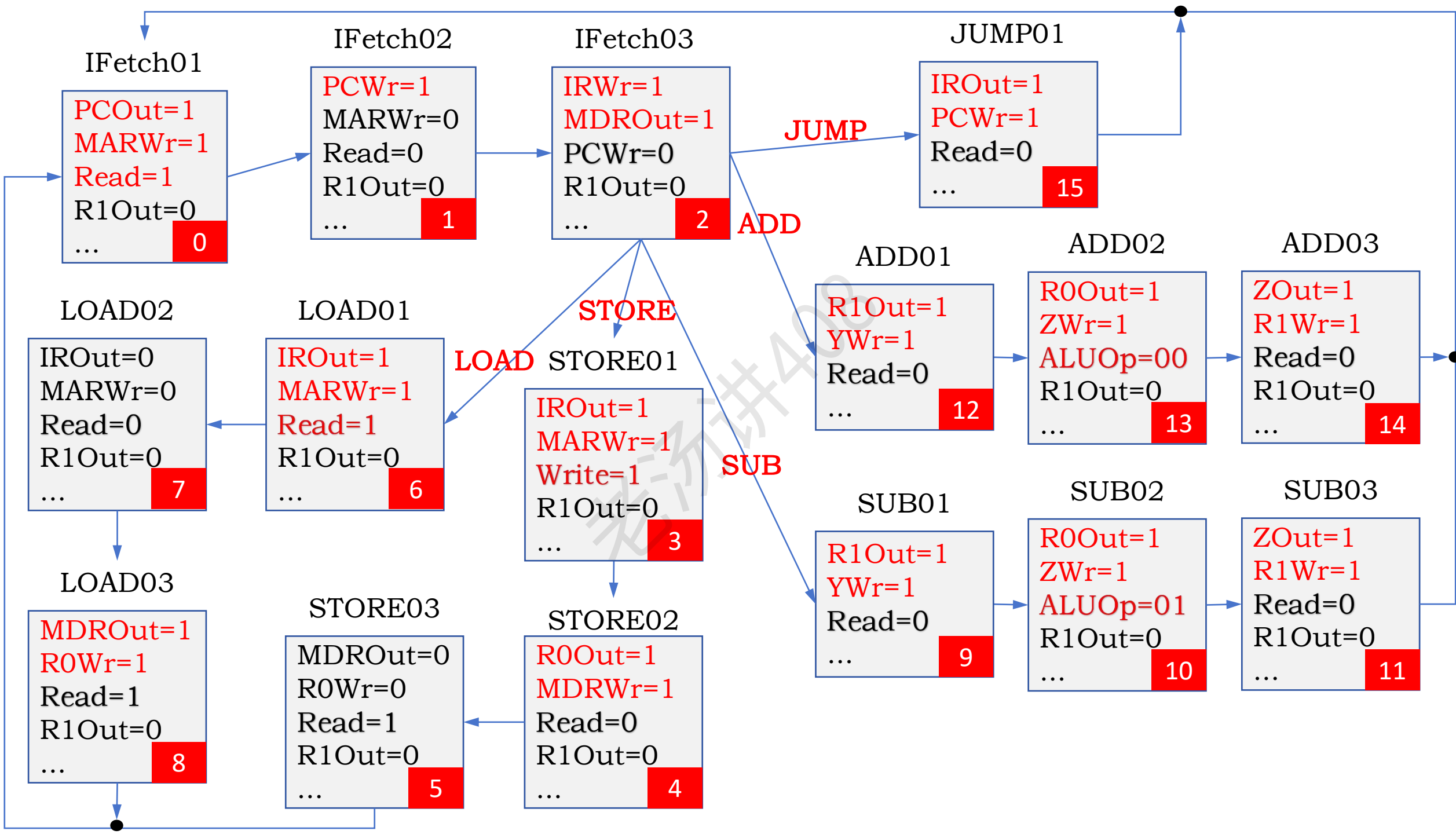
扫一扫上面的二维码图案，加我为朋友。





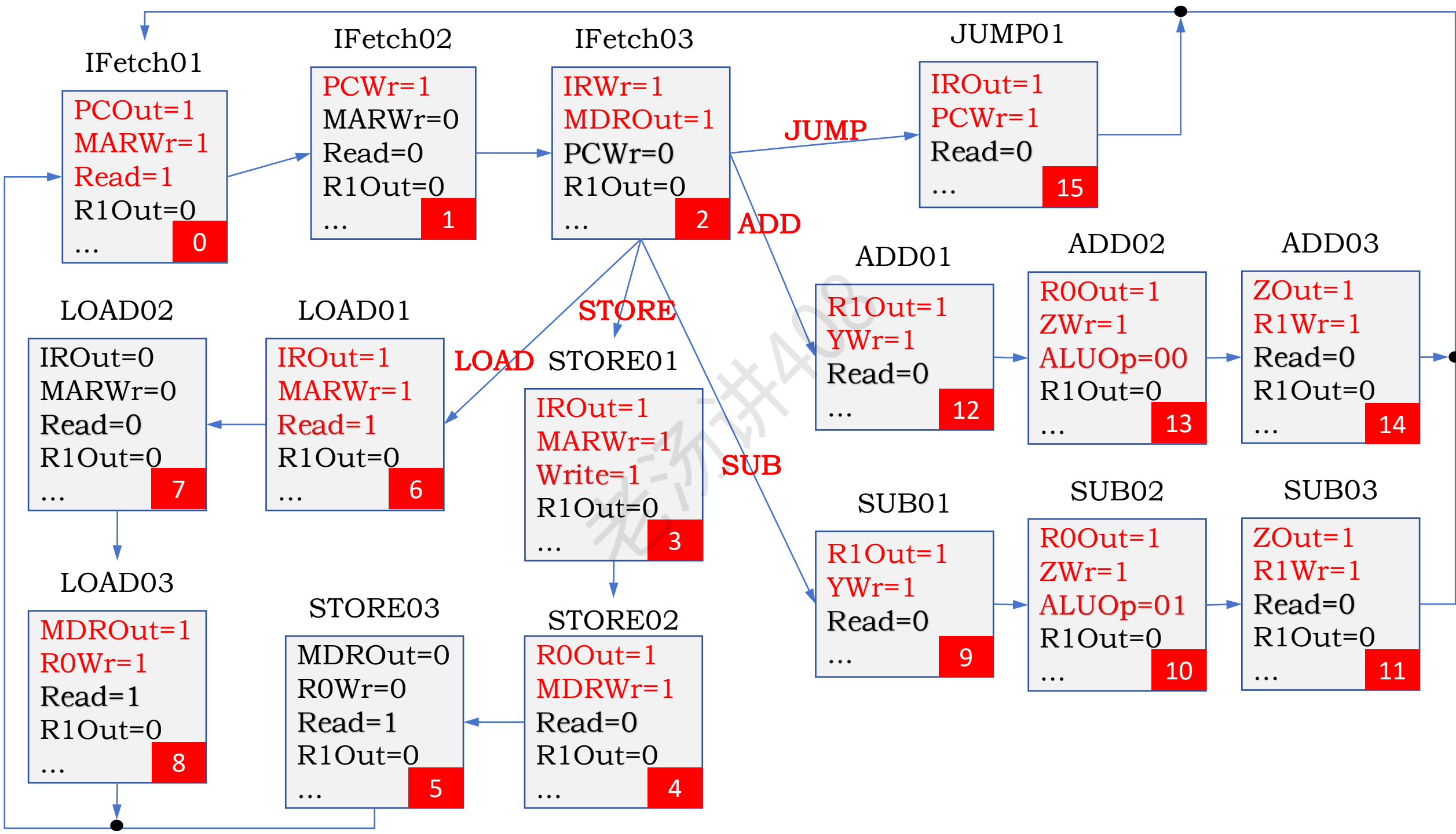
当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000



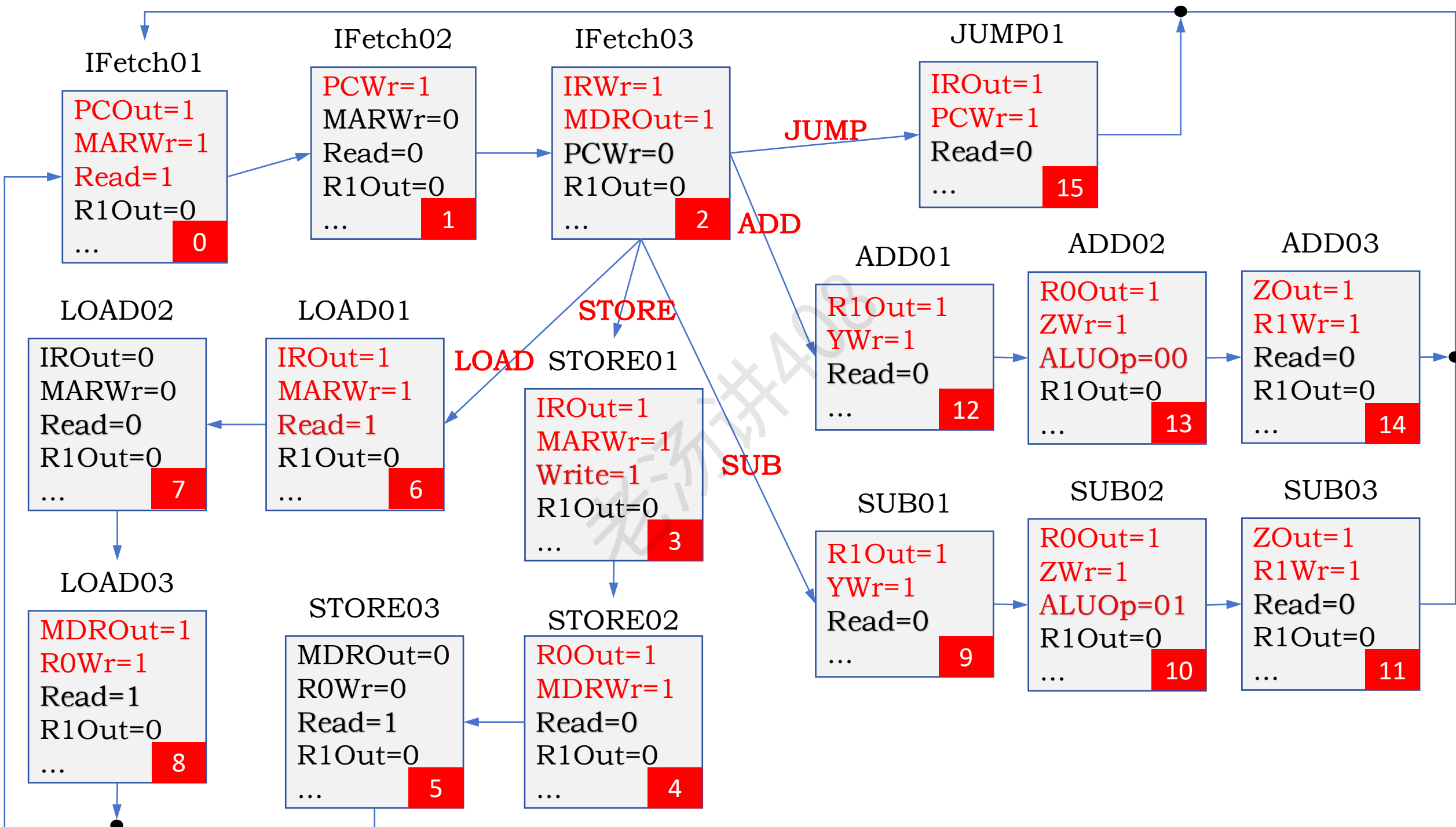
当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000



当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000

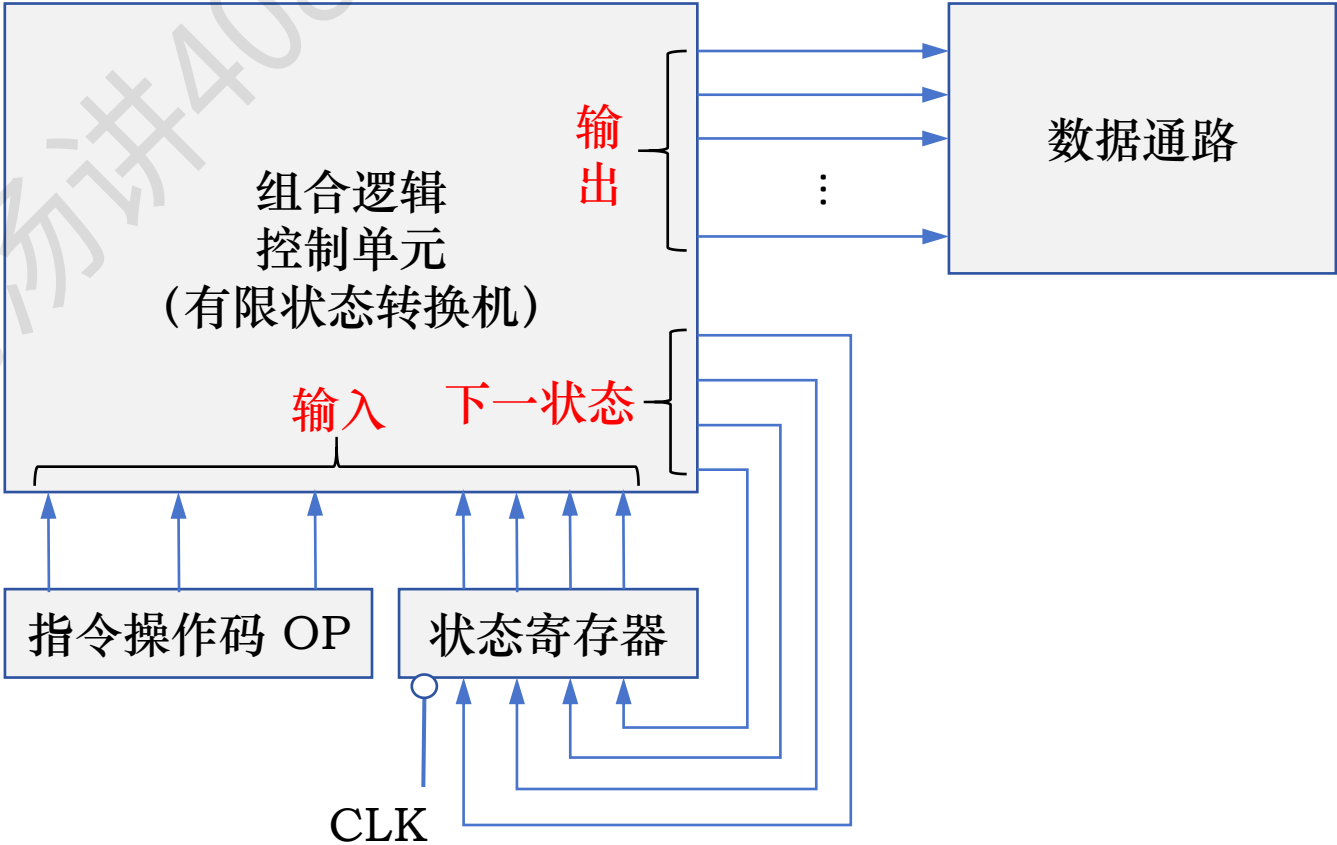


当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000

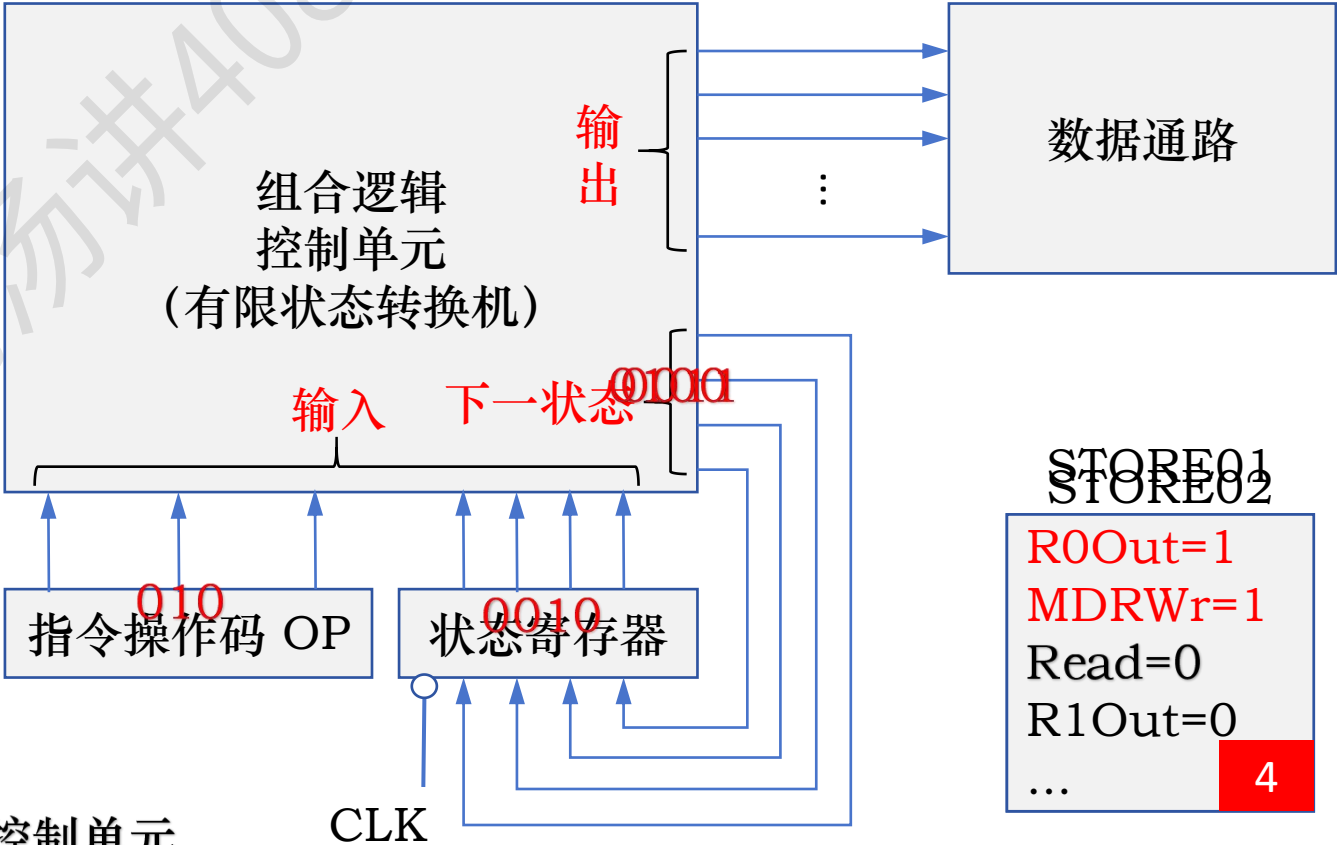
当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000



当前状态	指令操作码	下一状态
0000		0001
0001		0010
0010	010 (STORE)	0011
0011		0100
0100		0101
0101		0000
0010	000 (LOAD)	0110
0110		0111
0111		1000
1000		0000
0010	101 (SUB)	1001
1001		1010
1010		1011
1011		0000
0010	100 (ADD)	1100

当前状态	指令操作码	下一状态
1100		1101
1101		1110
1110		0000
0010	110 (JUMP)	1111
1111		0000



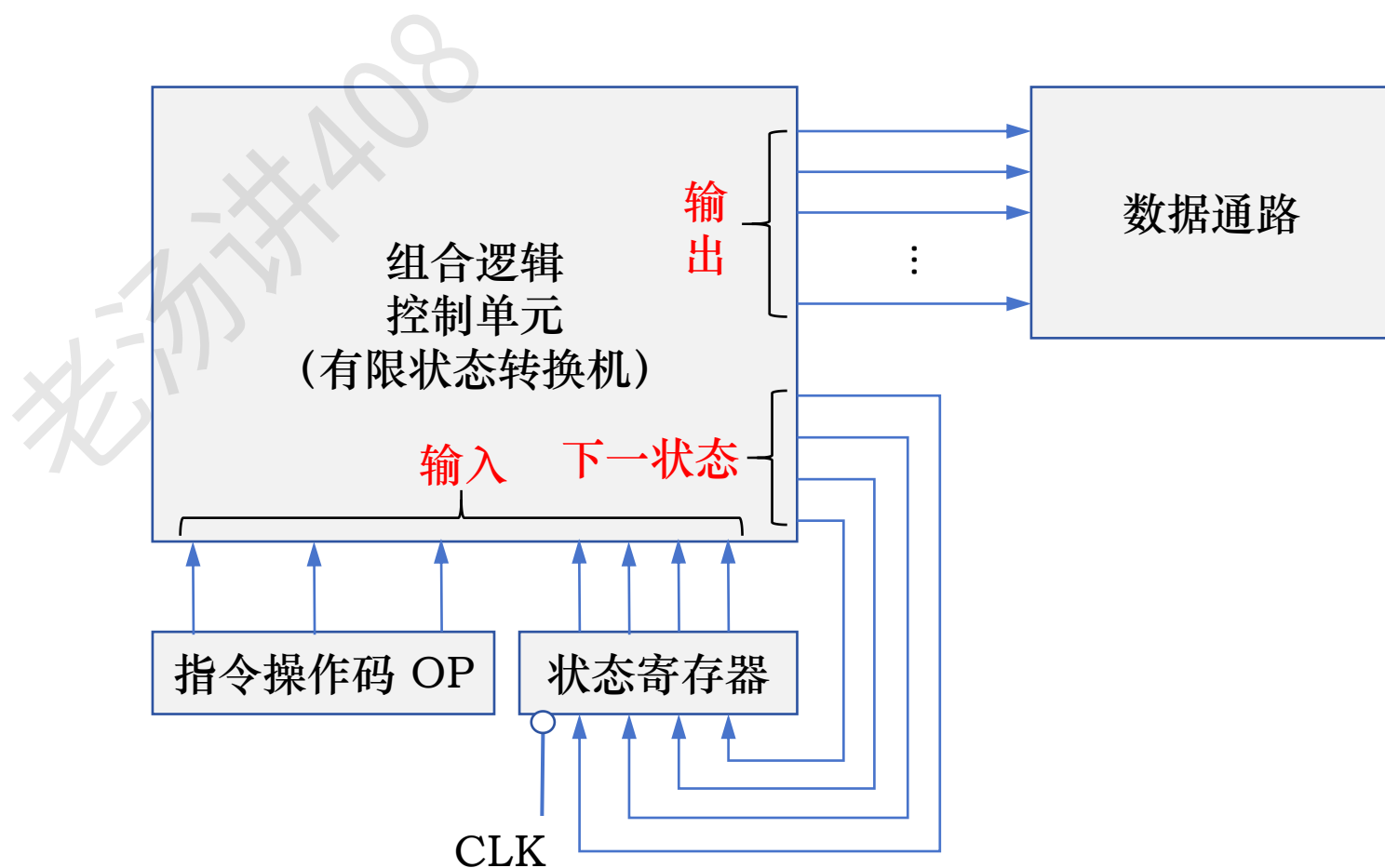
硬布线控制单元

中央处理器（CPU）：组合逻辑控制单元（硬布线控制器）

优点：速度快

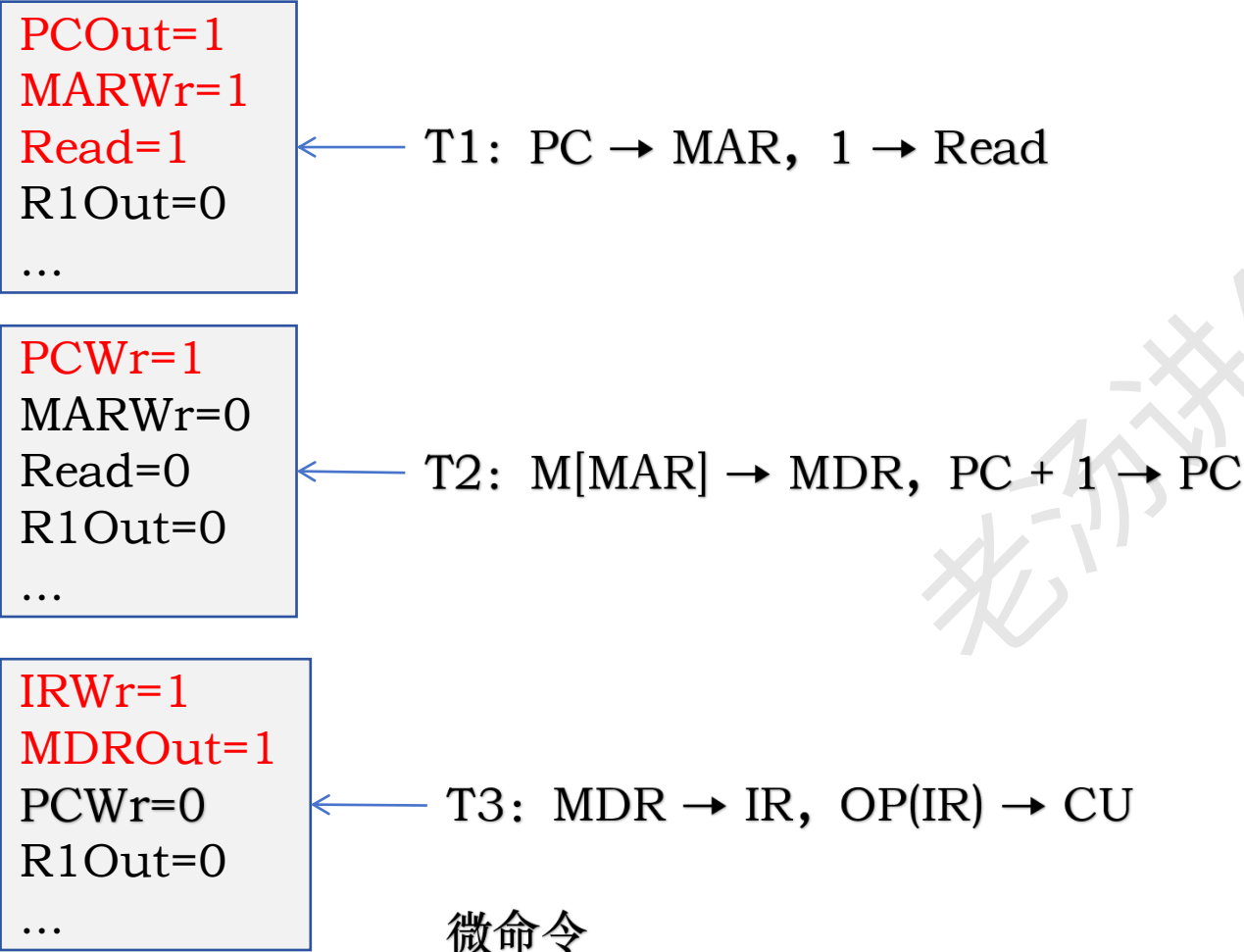
缺点：结构复杂，实现困难，不容易扩充和修改指令

只适合 RISC 指令系统

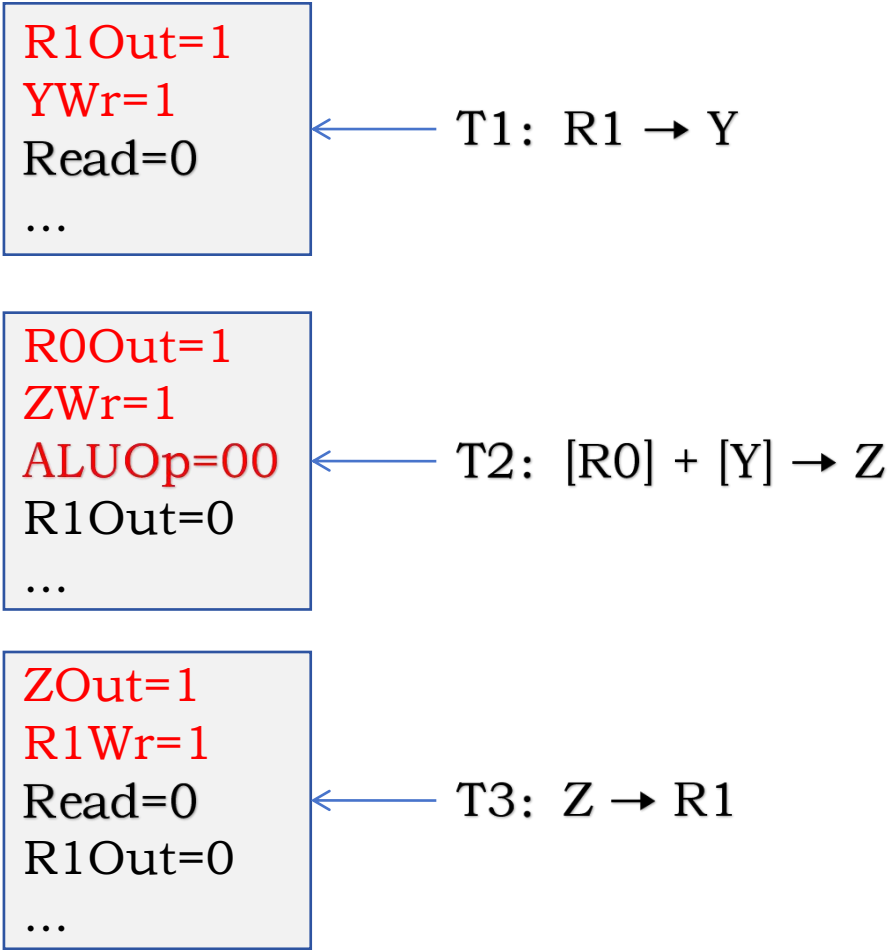


中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

ADD 指令取指阶段



ADD 指令执行阶段

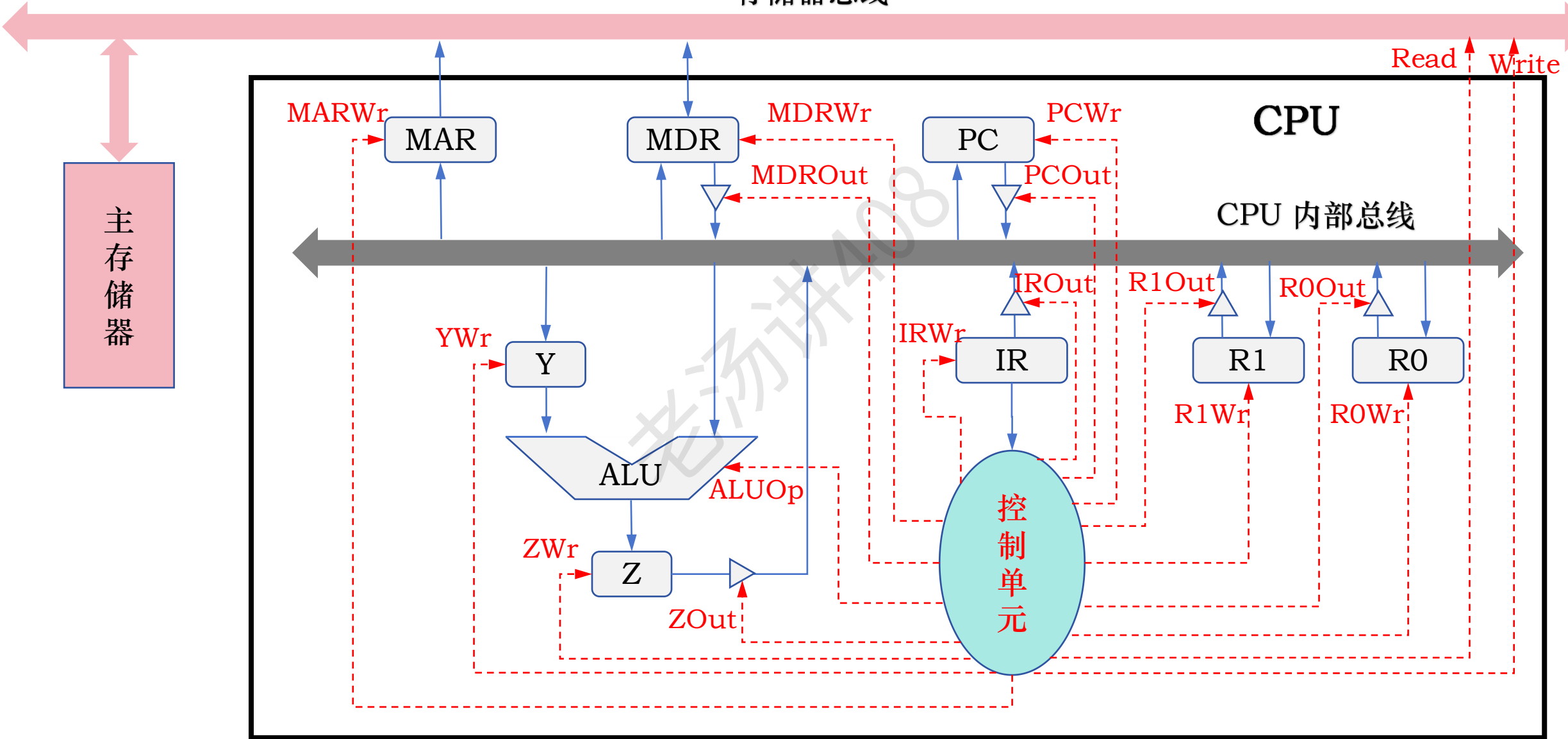


微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成

中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

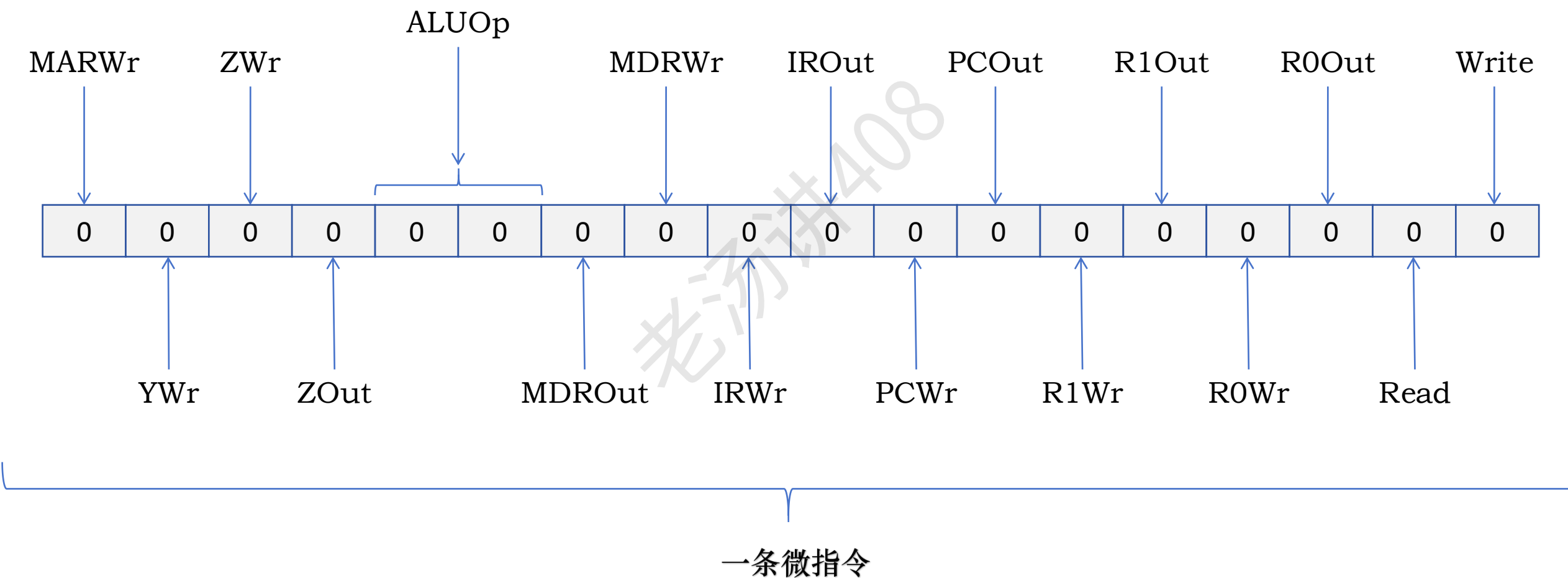
微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成

存储器总线



中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成



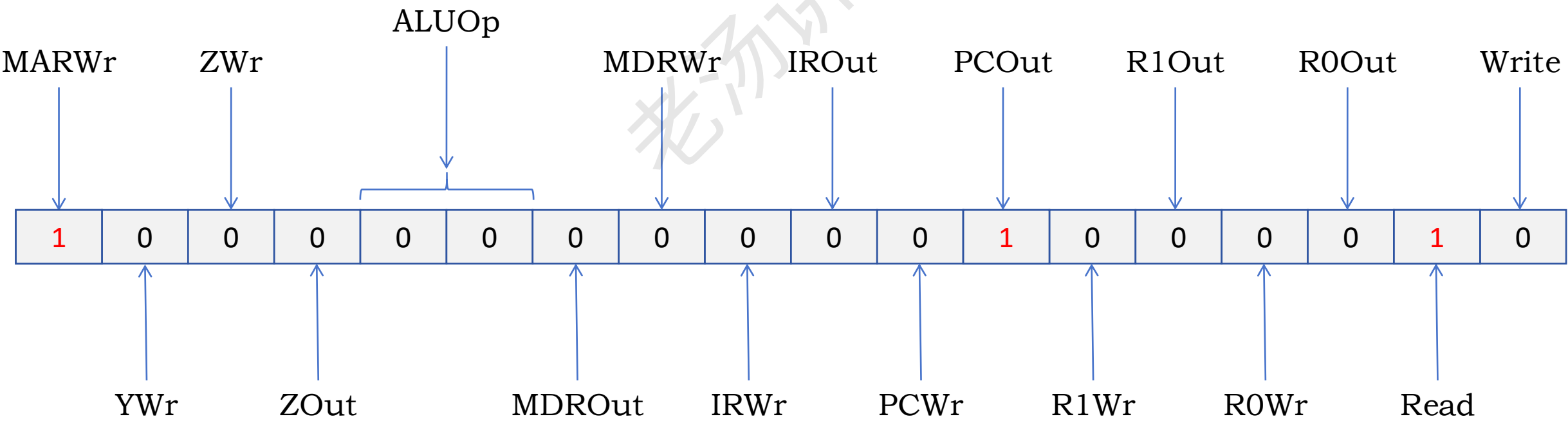
中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成

ADD 指令取指阶段

PCOut=1
MARWr=1
Read=1
R1Out=0
...

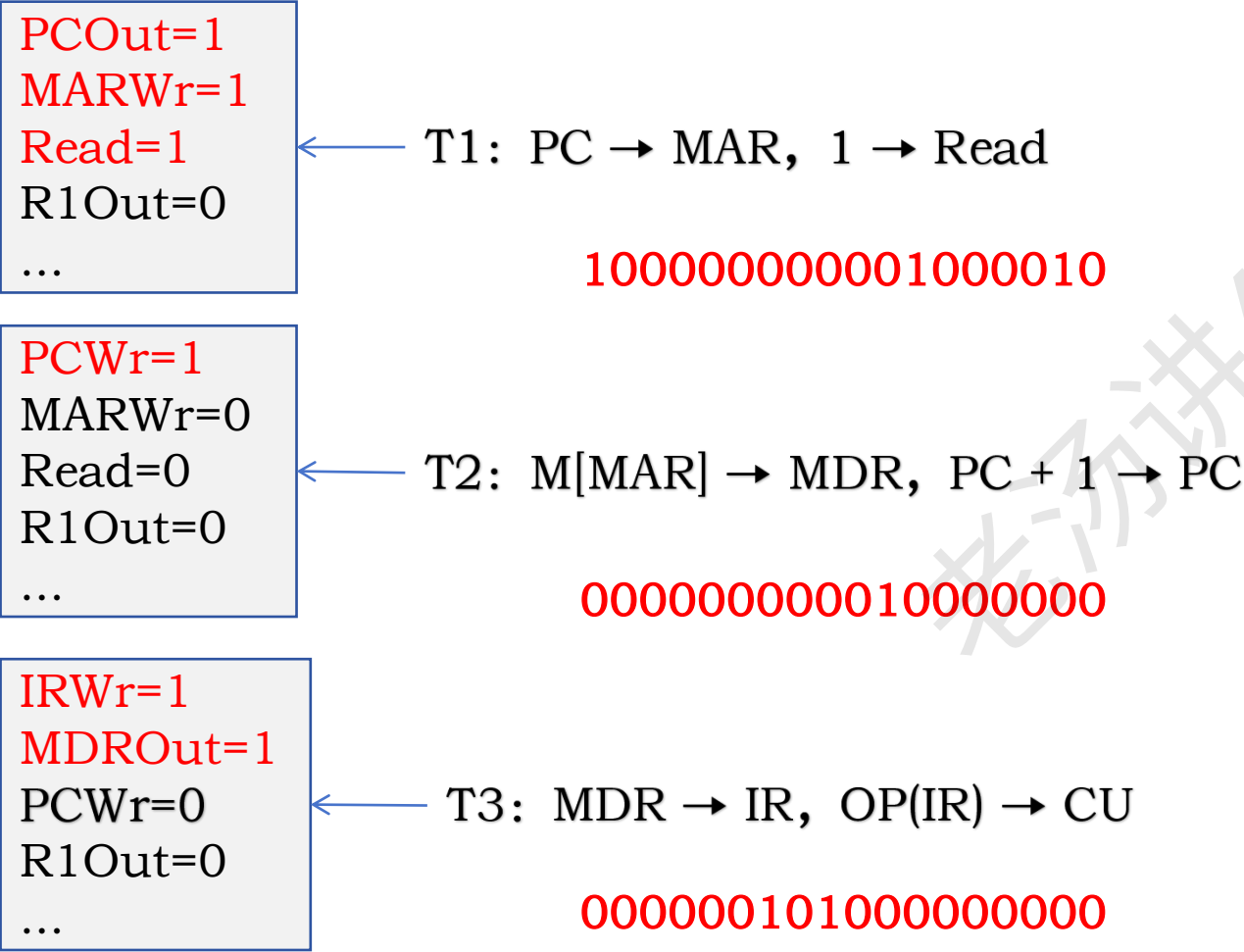
T1: PC → MAR, 1 → Read



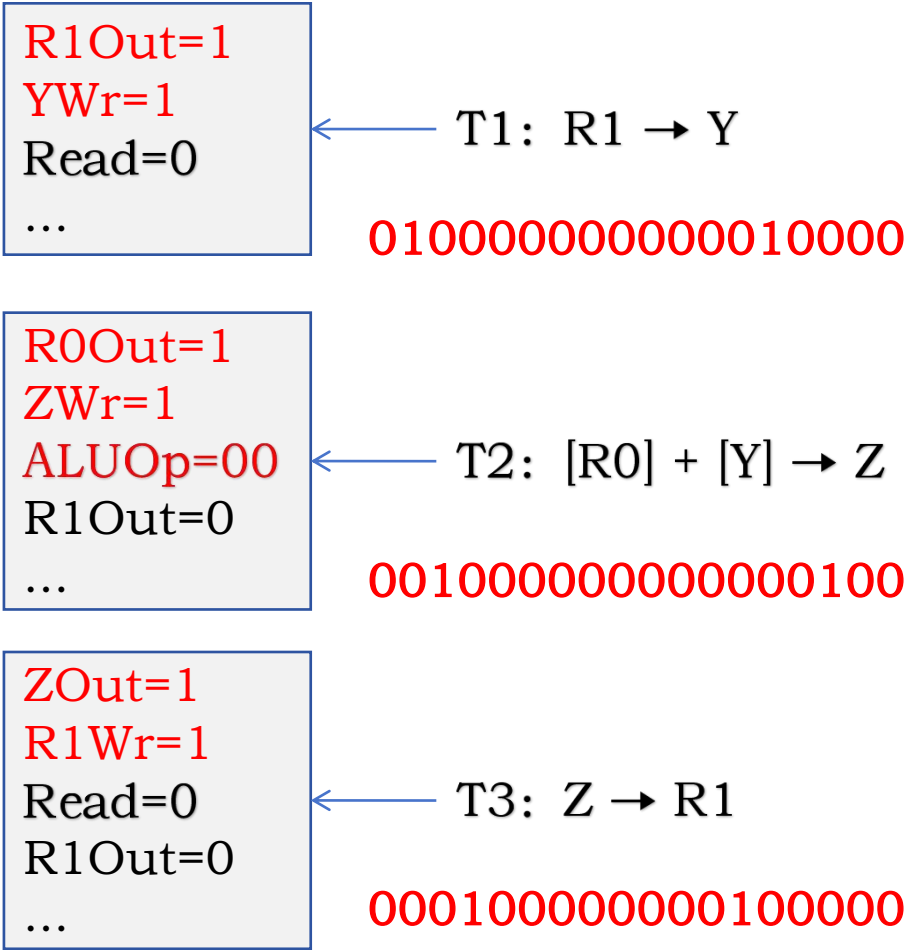
中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成

ADD 指令取指阶段

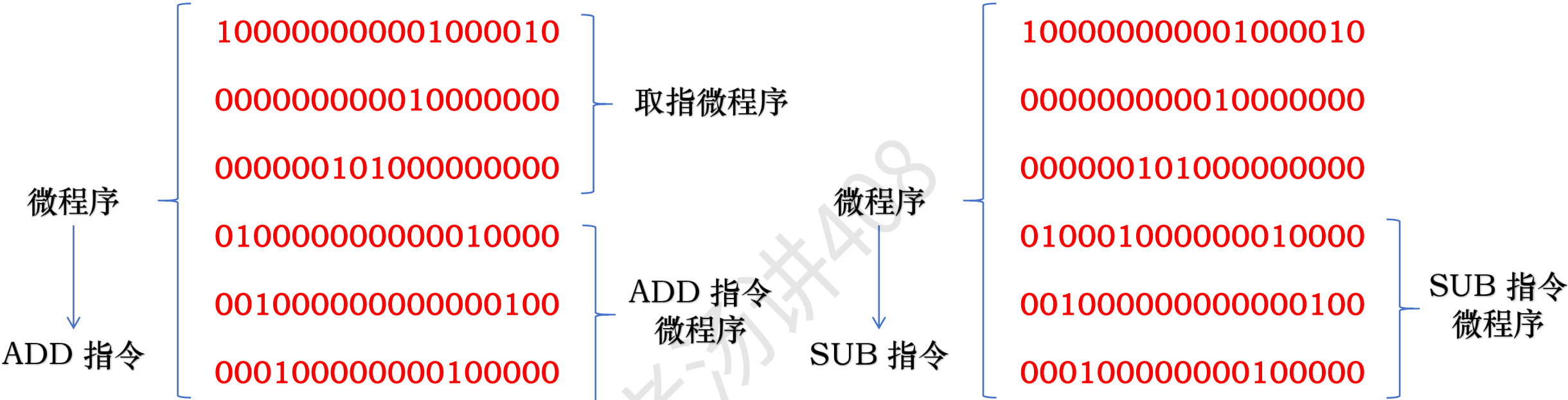


ADD 指令执行阶段

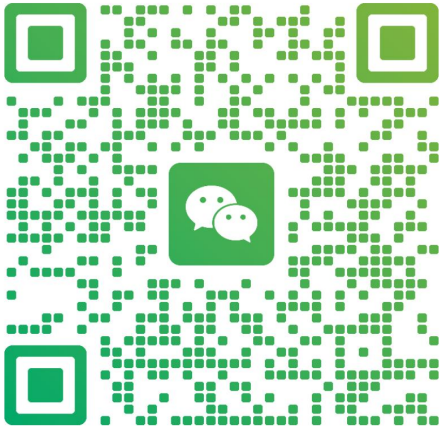


中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微程序是一个微指令序列 一条微指令包含若干个微命令 一条机器指令的功能由一个微程序完成



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



扫一扫上面的二维码图案，加我为朋友。

控制存储器（Control Storage，简称 CS）

微地址

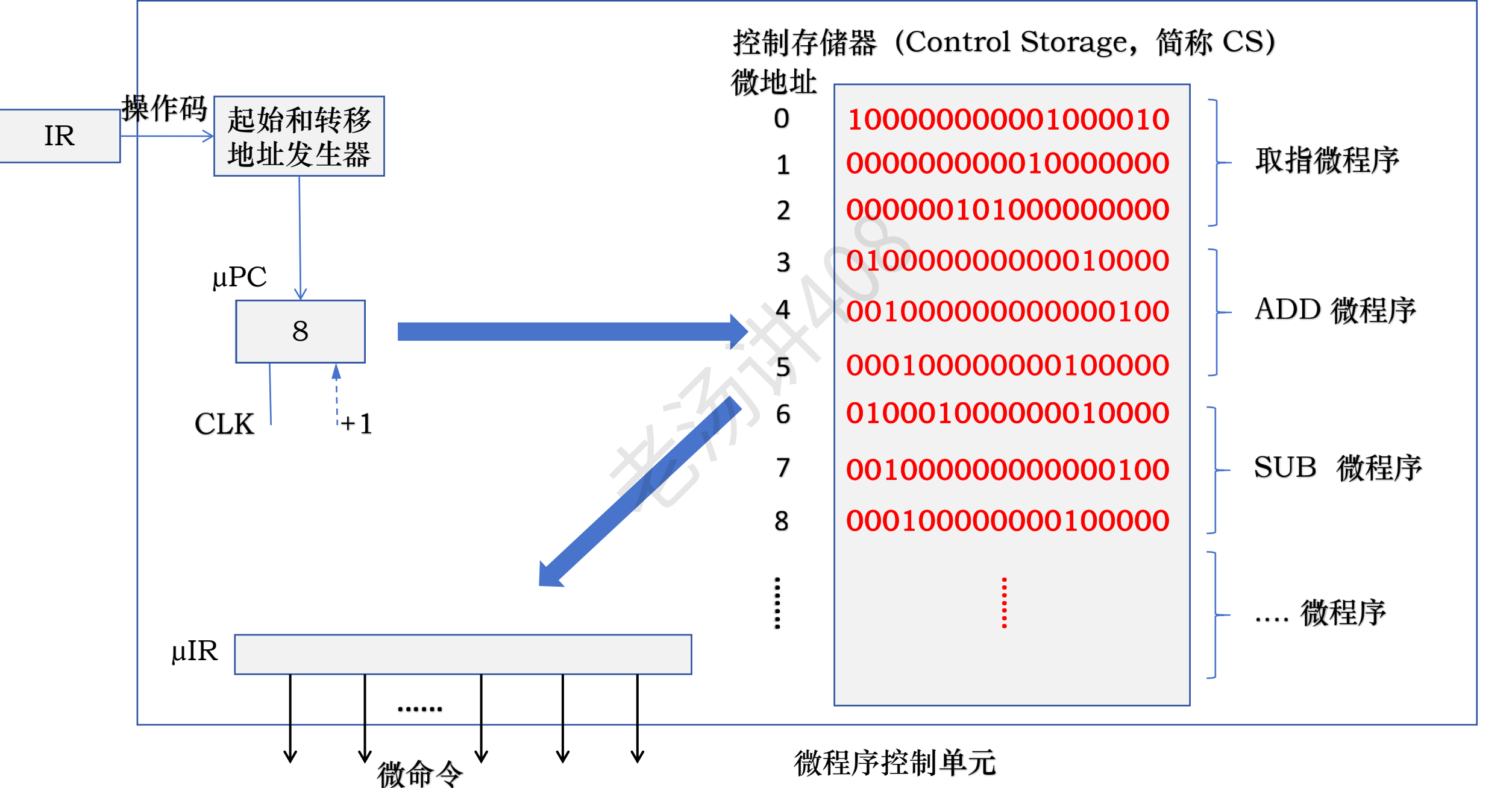
0	100000000001000010
1	000000000010000000
2	000000101000000000
3	010000000000010000
4	0010000000000000100
5	000100000000100000
6	010001000000010000
7	0010000000000000100
8	000100000000100000
⋮	⋮

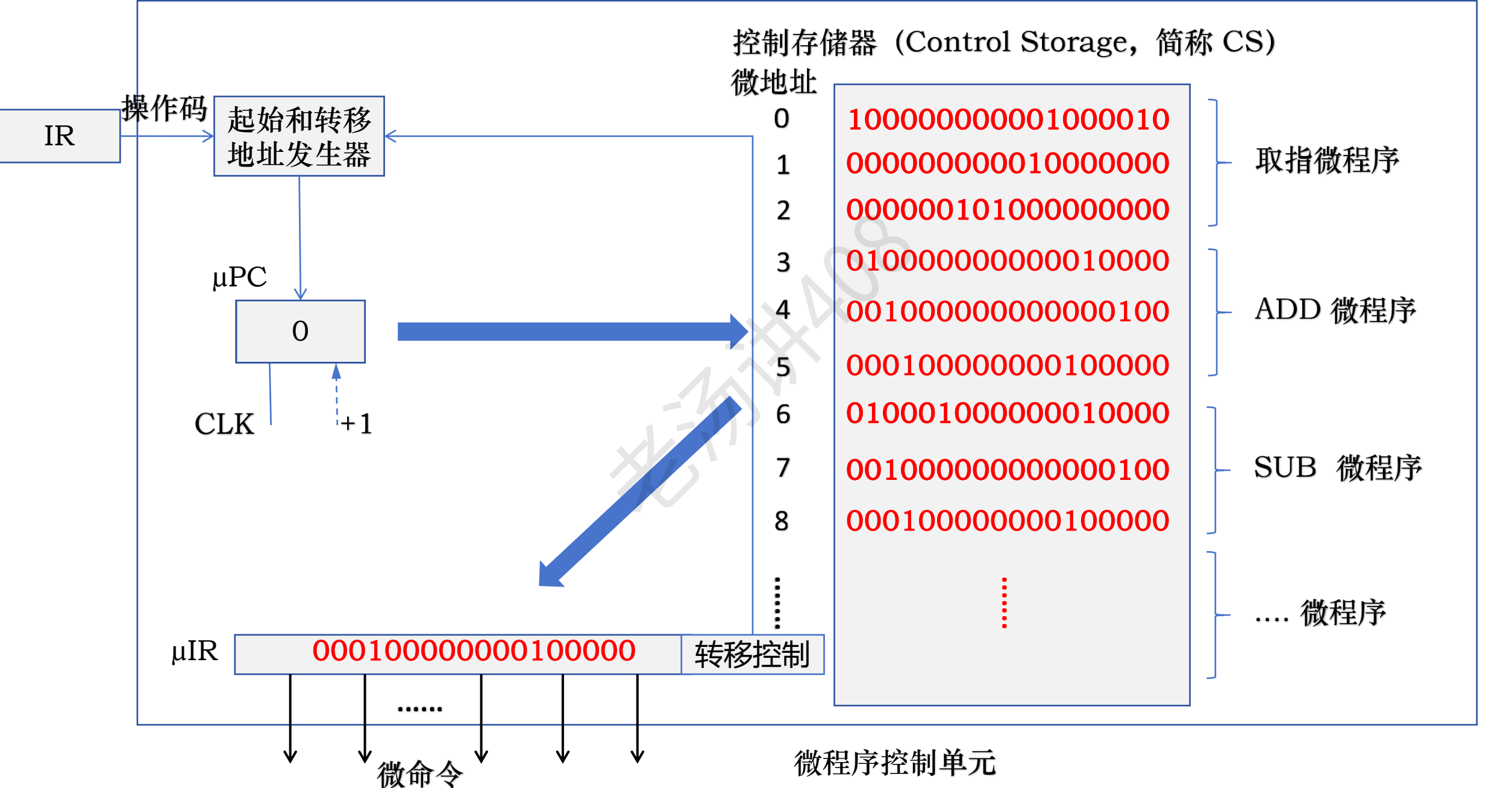
取指微程序

ADD 微程序

SUB 微程序

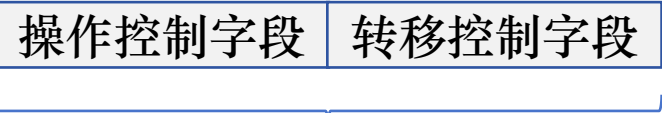
.... 微程序



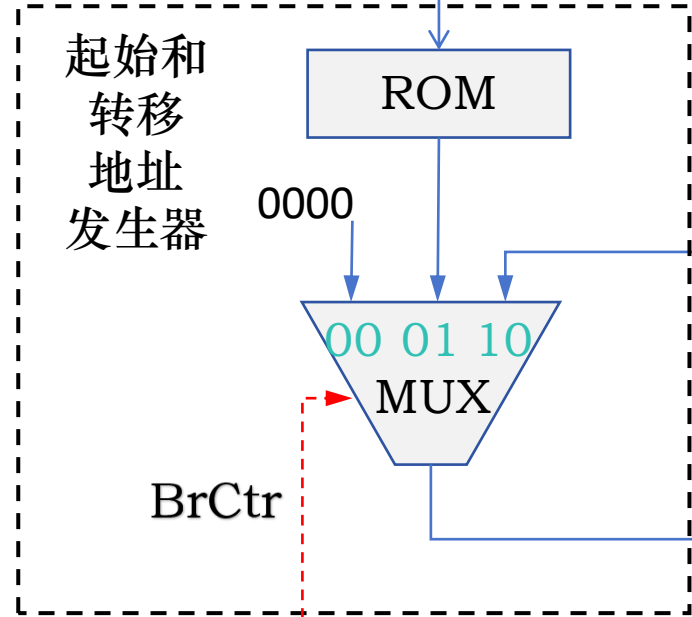


中央处理器（CPU）：控制单元的设计实现 - 微程序控制单元

增量计数法



微指令



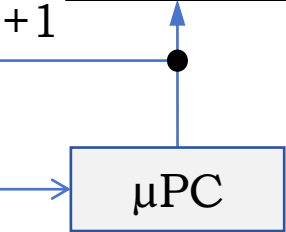
地址	控存 (CS)	
0000	100000000001000010	10
0001	000000000010000000	10
0010	000000101000000000	01
0011	010000000000010000	10
0100	001000000000000100	10
0101	000100000000100000	10
0110	000000010001000000	00
0111	001000000000000100	10
1000	000100000000100000	10
1001	000000000000000000	10
1010	001100010000000100	10
1011	001110000010000000	00

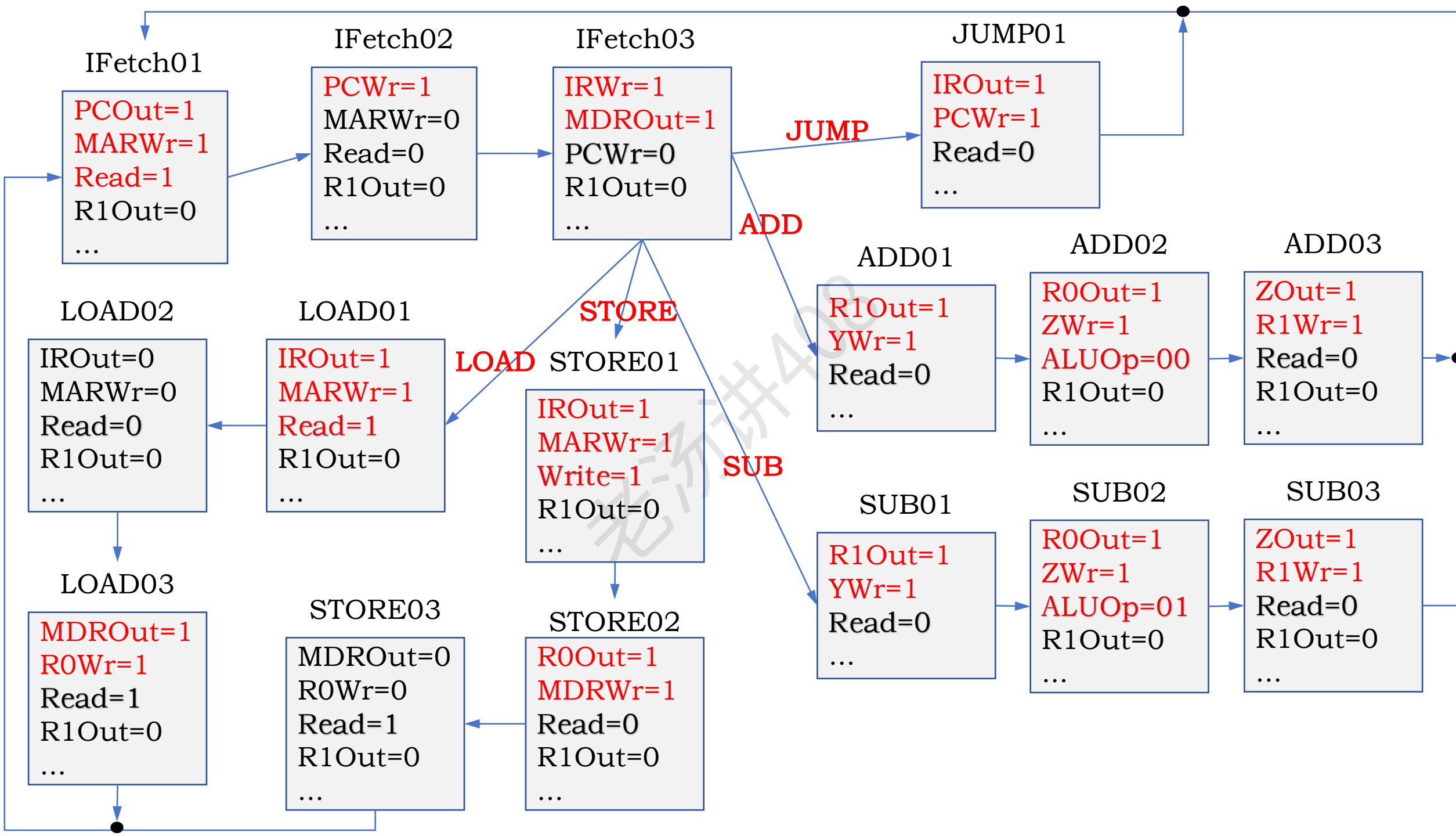
转移控制

BrCtr=00,
转到取指令微程序首条微指令执行

BrCtr=01,
转到操作码对应微程序首条微指令执行

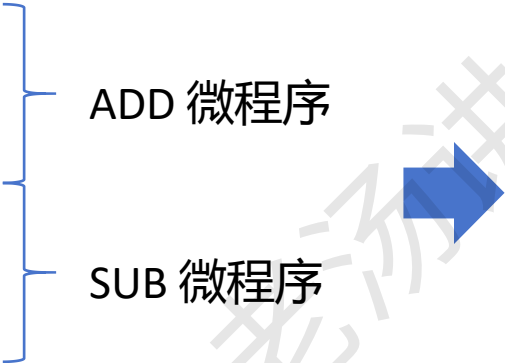
BrCtr=10,
按顺序执行下一条微指令





中央处理器（CPU）：控制单元的设计实现 - 微程序控制单元

地址	控存 (CS)
0000	100000000001000010
0001	000000000001000000
0010	000000101000000000
0011	010000000000001000
0100	0010000000000000100
0101	000100000000100000
0110	010000000000001000
0111	0010000000000000100
1000	000100000000100000
1001	000000000000000000
1010	0011000100000000100
1011	001110000010000000

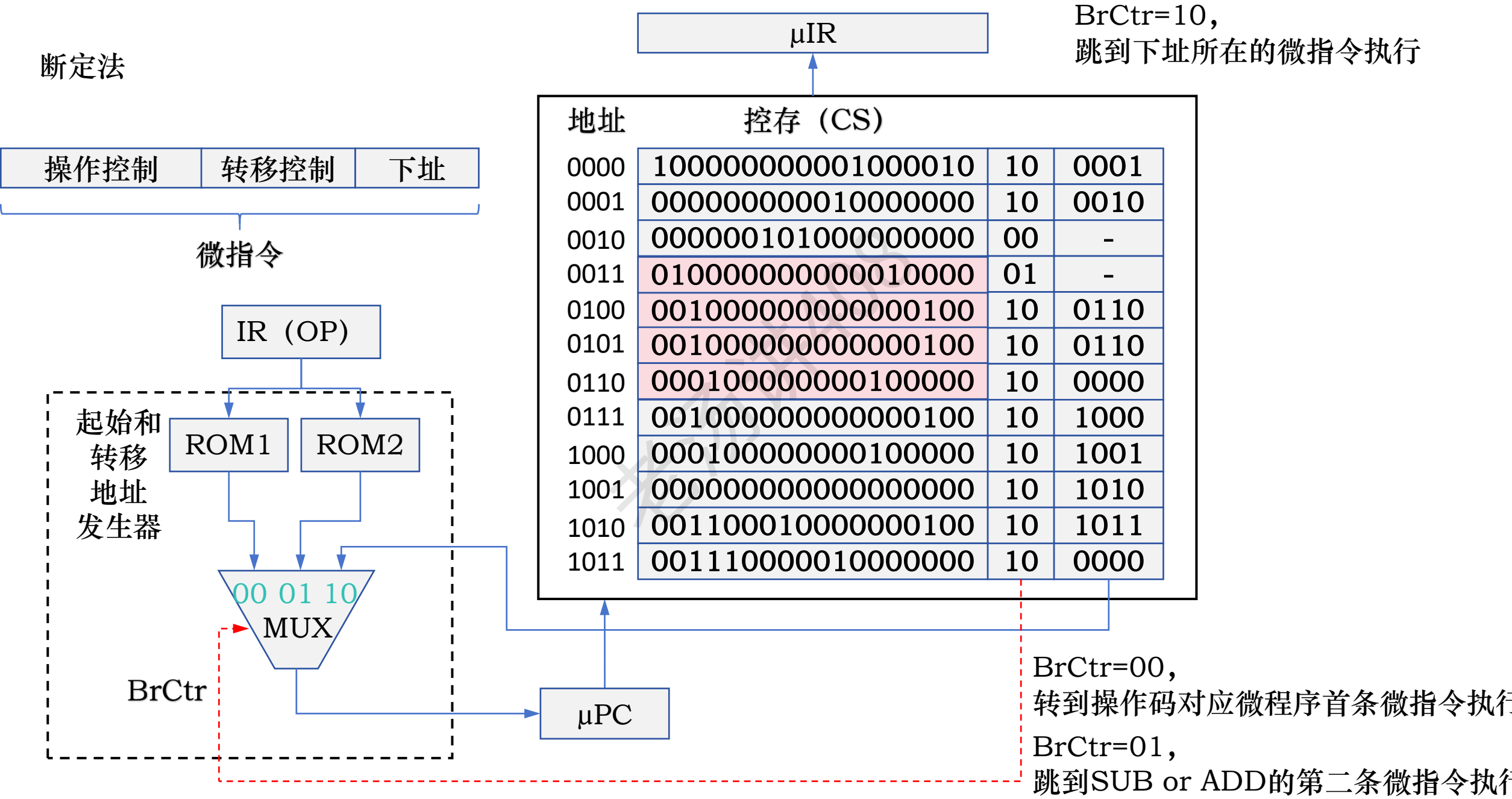


地址	控存 (CS)
0000	100000000001000010
0001	000000000001000000
0010	000000101000000000
0011	010000000000001000
0100	0010000000000000100
0101	0010000000000000100
0110	000100000000100000
0111	000000000000000000
1000	0011000100000000100
1001	001110000010000000
1010	000000010000010100
1011	001010000010011100

ADD: 0011 → 0100 → 0110

SUB: 0011 → 0101 → 0110

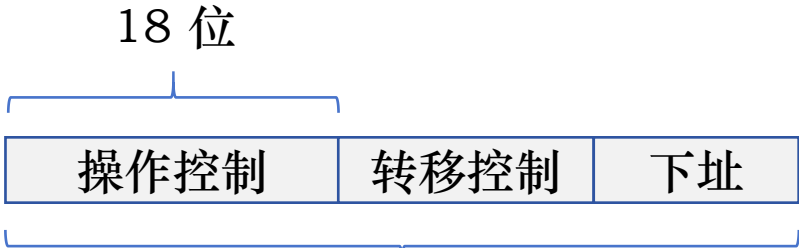
中央处理器（CPU）：控制单元的设计实现 - 微程序控制单元



ADD 指令取指阶段

PCOut=1
MARWr=1
Read=1
R1Out=0
...

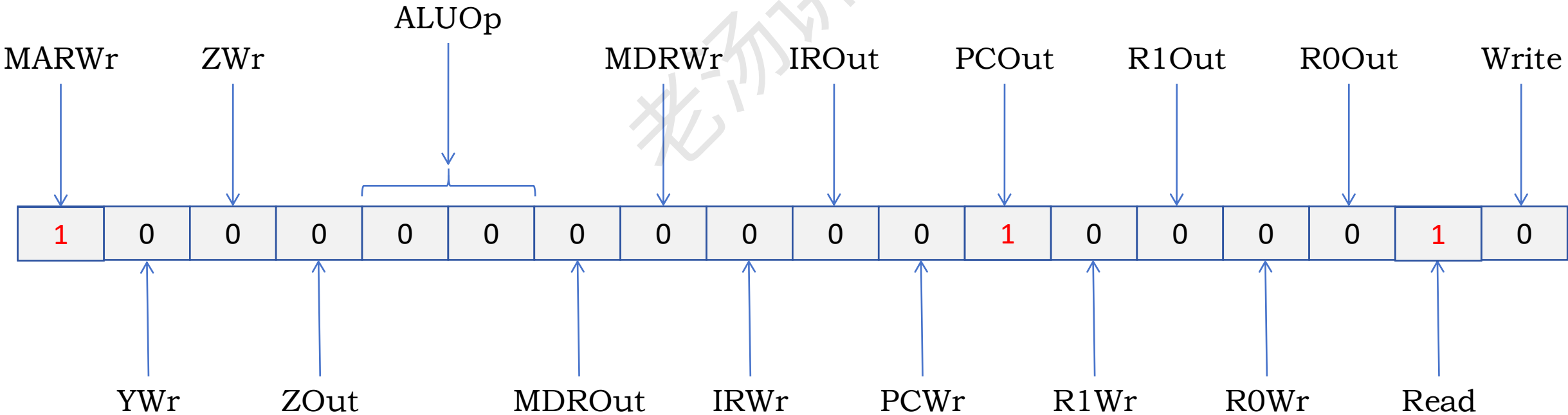
T1: PC → MAR, 1 → Read



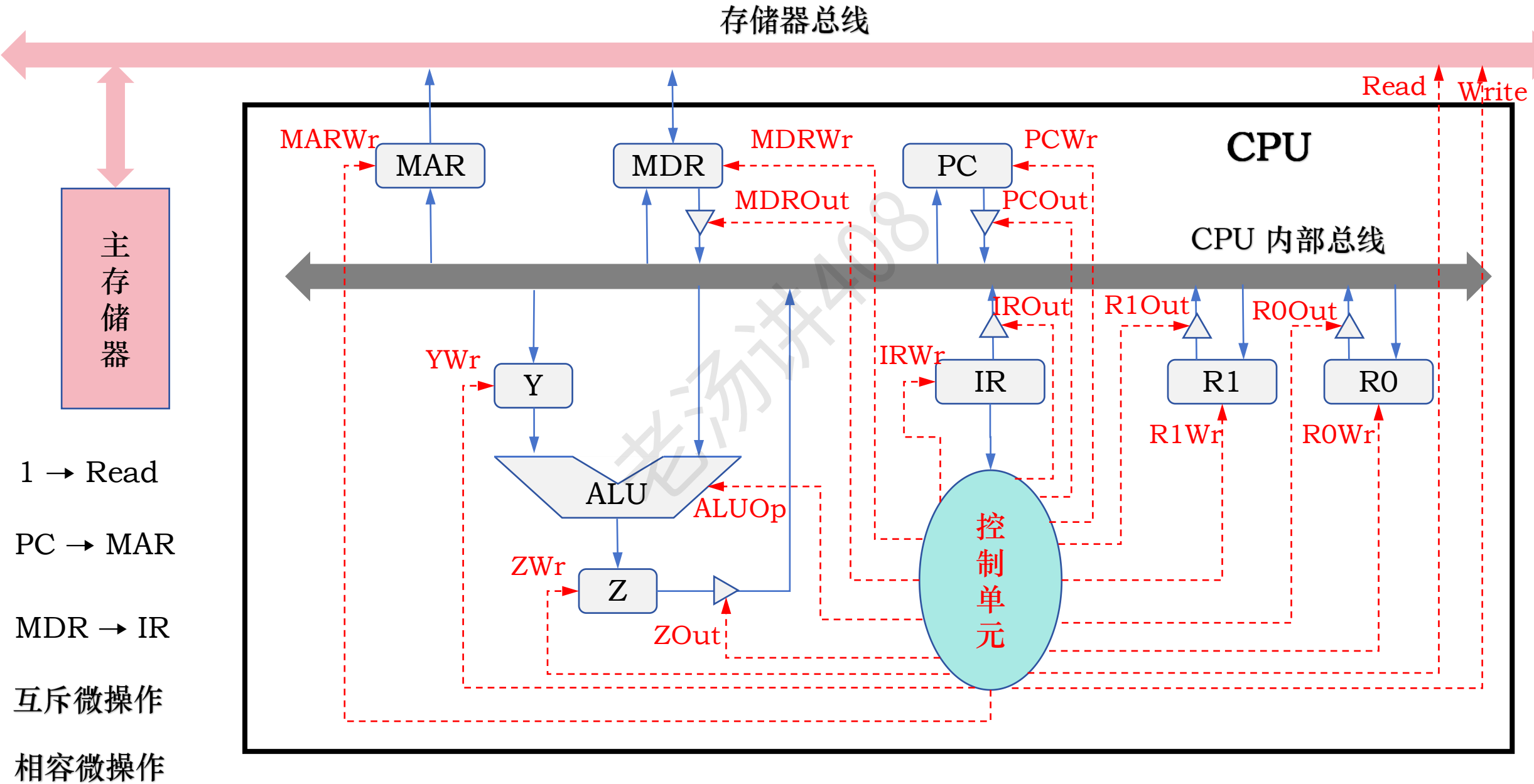
微指令

直接控制方式 控制电路简单，速度快

微指令太长，编码空间利用率低

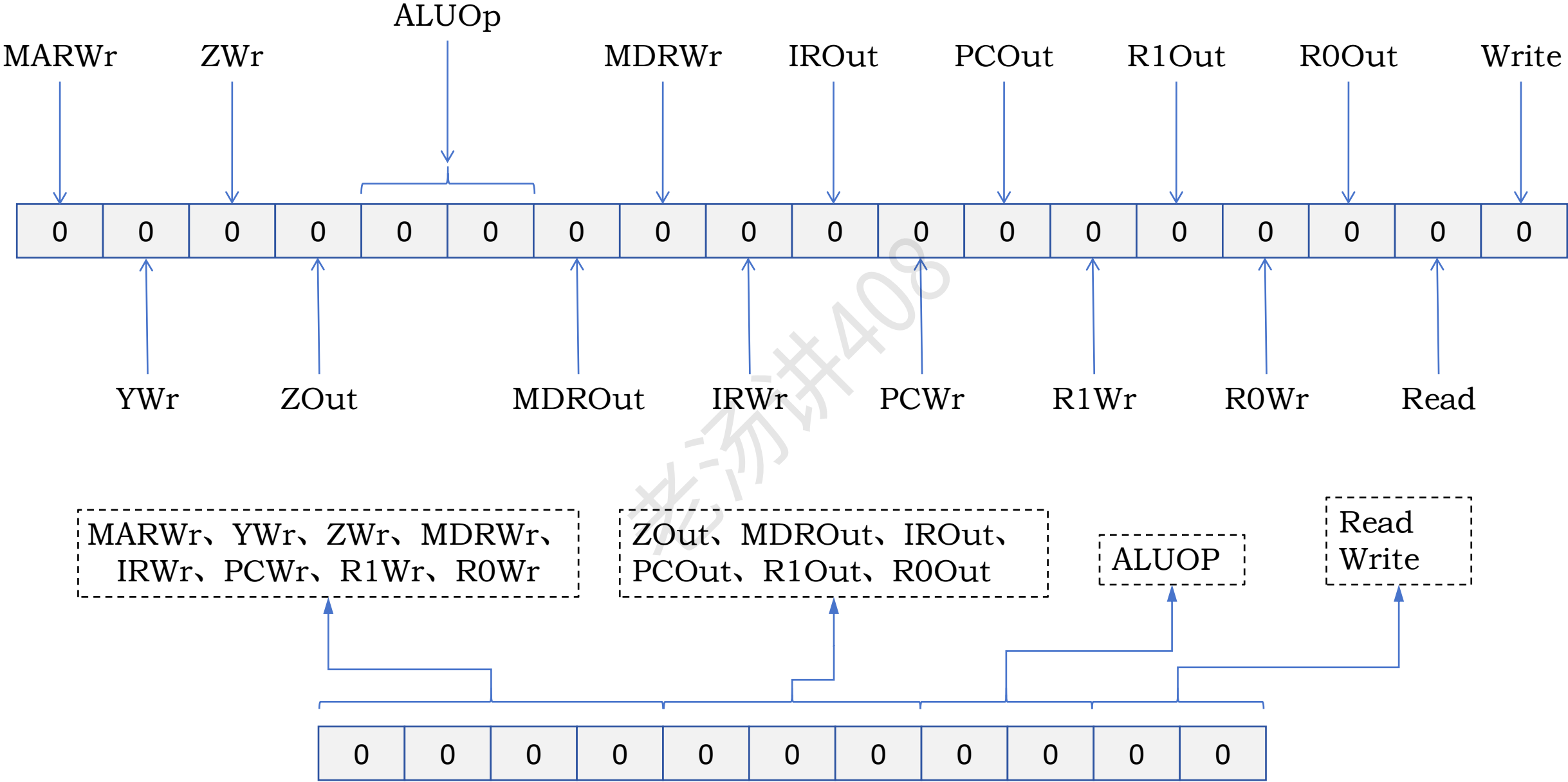


中央处理器（CPU）：控制单元的设计实现 - 微程序控制单元



中央处理器（CPU）：控制单元的设计实现 - 微程序控制单元

微命令编码



- 0000, 表示不发这一组的微命令

0001, 表示发送 MARWr 微命令

0010, 表示发送 YWr 微命令

0011, 表示发送 ZWr 微命令

0100, 表示发送 MDRWr 微命令

0101, 表示发送 IRWr 微命令

0110, 表示发送 PCWr 微命令

0111, 表示发送 R1Wr 微命令

1000, 表示发送 R0Wr 微命令
- 000, 表示不发这一组的微命令

001, 表示发送 ZOut 微命令

010, 表示发送 MDROut 微命令

011, 表示发送 IROut 微命令

100, 表示发送 PCOut 微命令

101, 表示发送 R1Out 微命令

110, 表示发送 R0Out 微命令
- 00, 加法

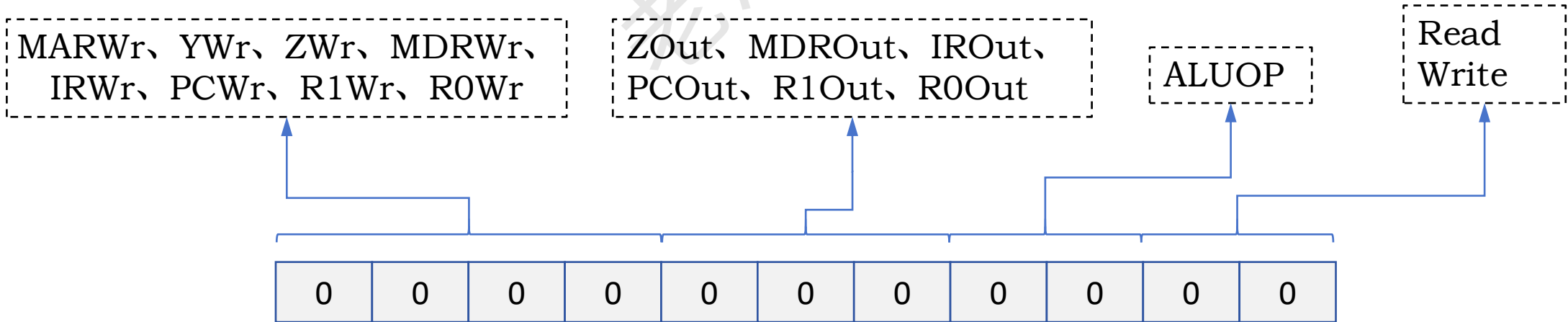
01, 减法

10, 逻辑或

00, 表示不发这一组的微命令

01, 表示发送 Read 微命令

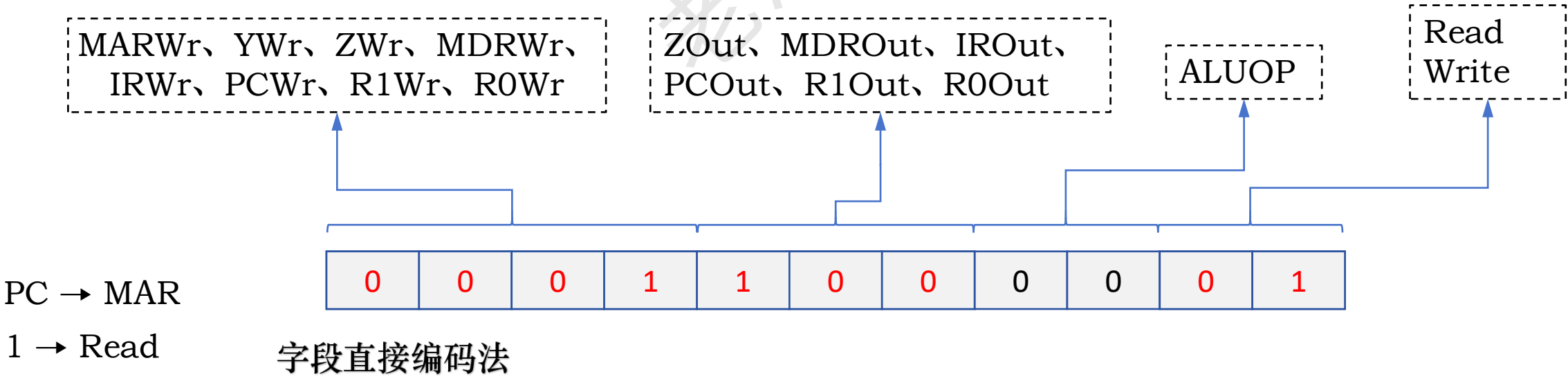
10, 表示发送 Write 微命令



中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微命令编码

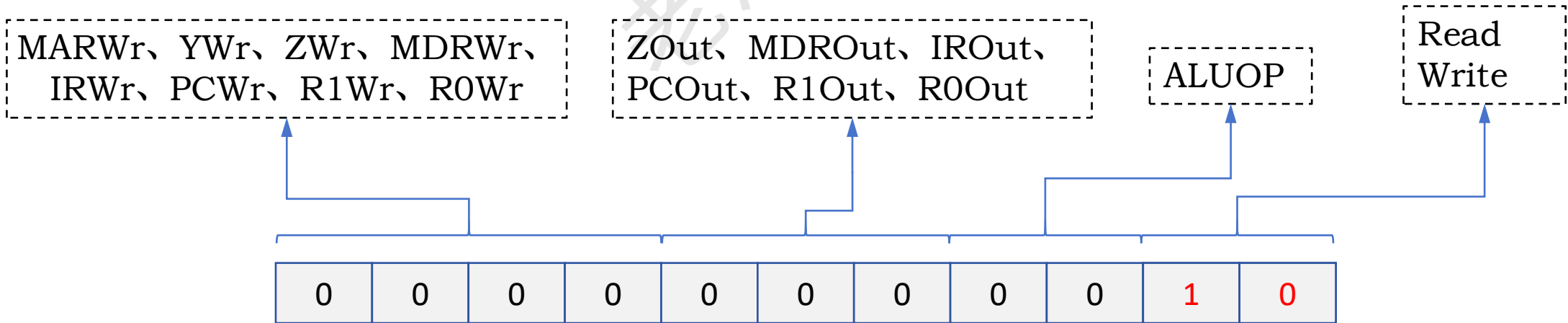
0000, 表示不发这一组的微命令	000, 表示不发这一组的微命令	00, 加法
0001, 表示发送 MARWr 微命令	001, 表示发送 ZOut 微命令	01, 减法
0010, 表示发送 YWr 微命令	010, 表示发送 MDROut 微命令	10, 逻辑或
0011, 表示发送 ZWr 微命令	011, 表示发送 IROut 微命令	
0100, 表示发送 MDRWr 微命令	100, 表示发送 PCOut 微命令	
0101, 表示发送 IRWr 微命令	101, 表示发送 R1Out 微命令	00, 表示不发这一组的微命令
0110, 表示发送 PCWr 微命令	110, 表示发送 R0Out 微命令	01, 表示发送 Read 微命令
0111, 表示发送 R1Wr 微命令		10, 表示发送 Write 微命令
1000, 表示发送 R0Wr 微命令		



中央处理器 (CPU)：控制单元的设计实现 - 微程序控制单元

微命令编码

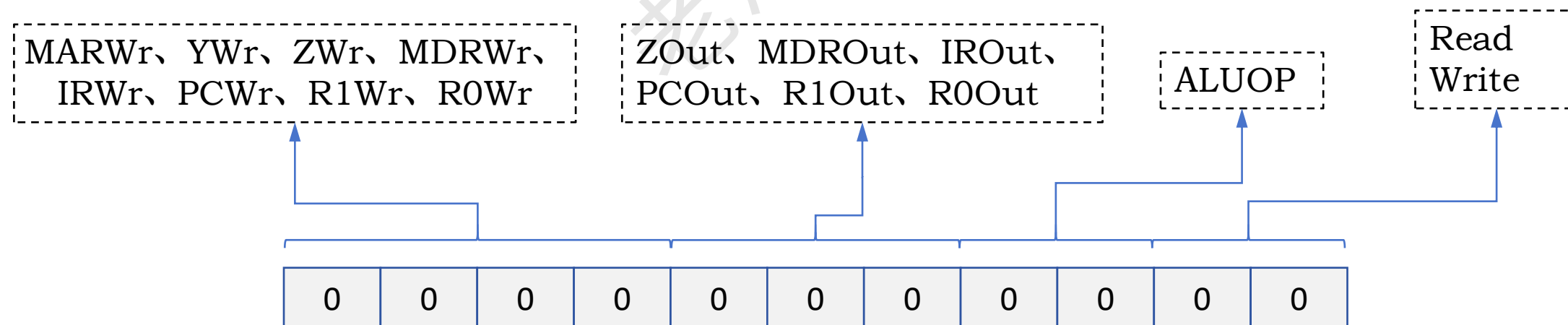
0000, 表示不发这一组的微命令	000, 表示不发这一组的微命令	00, 加法
0001, 表示发送 MARWr 微命令	001, 表示发送 ZOut 微命令	01, 减法
0010, 表示发送 YWr 微命令	010, 表示发送 MDROut 微命令	10, 逻辑或
0011, 表示发送 ZWr 微命令	011, 表示发送 IROut 微命令	
0100, 表示发送 MDRWr 微命令	100, 表示发送 PCOut 微命令	
0101, 表示发送 IRWr 微命令	101, 表示发送 R1Out 微命令	00, 表示不发这一组的微命令
0110, 表示发送 PCWr 微命令	110, 表示发送 R0Out 微命令	01, 表示发送 Read 微命令
0111, 表示发送 R1Wr 微命令		10, 表示发送 Write 微命令
1000, 表示发送 R0Wr 微命令		



1 → Write

其基本思想就是，将微指令分成若干字段，每个字段包含若干微命令，把互斥微命令组合在同一个字段，相容微命令组合在不同的字段，对每一个字段内的微命令进行编码

缩短了微指令的长度，节省了控存空间 增加了译码线路，加大了一些成本，并多出了一部分的译码时间



中央处理器 (CPU) 设计

单周期处理器 → 多周期处理器 → 流水线处理器

32 位 MIPS 指令系统

R- 型指令:

add rd, rs, rt

sub rd, rs, rt

or rd, rs, rt

slt rd, rs, rt

I-型指令:

lw rt, imm16(rs)

sw rt, imm16(rs)

beq rs, rt, imm16

J-型指令:

j target

中央处理器 (CPU) 设计的五个步骤

第一步：分析每条指令的功能

第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路

中央处理器 (CPU) 设计的五个步骤

第一步：分析每条指令的功能

第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

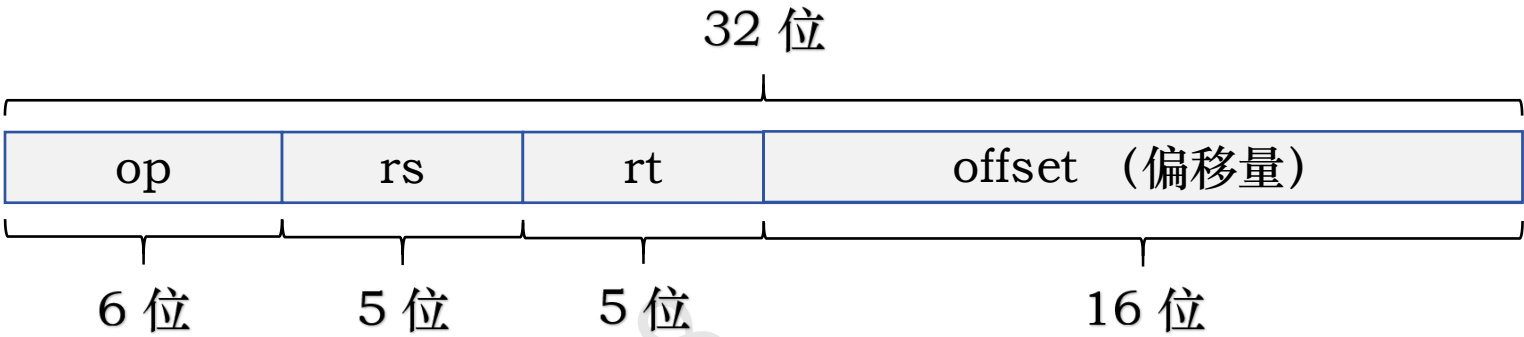
第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路

中央处理器 (CPU) 设计: 分析每条指令的功能

add rd, rs, rt	$R[rd] \leftarrow R[rs] + R[rt]$	从寄存器 rs、rt 中取数，相加、减、或， 结果送到寄存器 rd 中
sub rd, rs, rt	$R[rd] \leftarrow R[rs] - R[rt]$	
or rd, rs, rt	$R[rd] \leftarrow R[rs] \mid R[rt]$	
slt rd, rs, rt	$\text{if } (R[rs] < R[rt]) \ R[rd] \leftarrow 1$ $\text{else } R[rd] \leftarrow 0$	从 rs、rt 取数后，按有符号整数来判断两数大小， 如果小于，rd 设置为 1，否则，rd 设置为 0
lw rt, imm16(rs)	$\text{Addr} \leftarrow R[rs] + \text{SignExt}(\text{imm16})$ $R[rt] \leftarrow M[\text{Addr}]$	从 rs 取数，将 imm16 符号扩展，然后相加， 结果作为访存地址 Addr，从 Addr 取数并送 rt
sw rt, imm16(rs)	$\text{Addr} \leftarrow R[rs] + \text{SignExt}(\text{imm16})$ $M[\text{Addr}] \leftarrow R[rt]$	从 rs 取数，将 imm16 符号扩展，然后相加， 结果作为访存地址 Addr，将 rt 送 Addr 中
beq rs, rt, imm16	$\text{Cond} \leftarrow R[rs] - R[rt]$ $\text{if } (\text{Cond} == 0)$ $\text{PC} \leftarrow \text{PC} + 4 + (\text{SignExt}(\text{imm16}) \ll 4)$	做减法以比较 rs 和 rt 中内容的大小，并计 算下条指令地址，然后根据比较结果修改 PC

MIPS 指令系统：I-型指令格式（分支指令 beq 和 bne） - 第 5 章第 9 小节

I-型指令格式



4
beq \$4, \$5, 4



beq 指令的下一条指令的内存地址
若 $\$rs == \rt , 则 $PC = PC + 4 + (offset \ll 2)$

5
bne \$4, \$5, 4

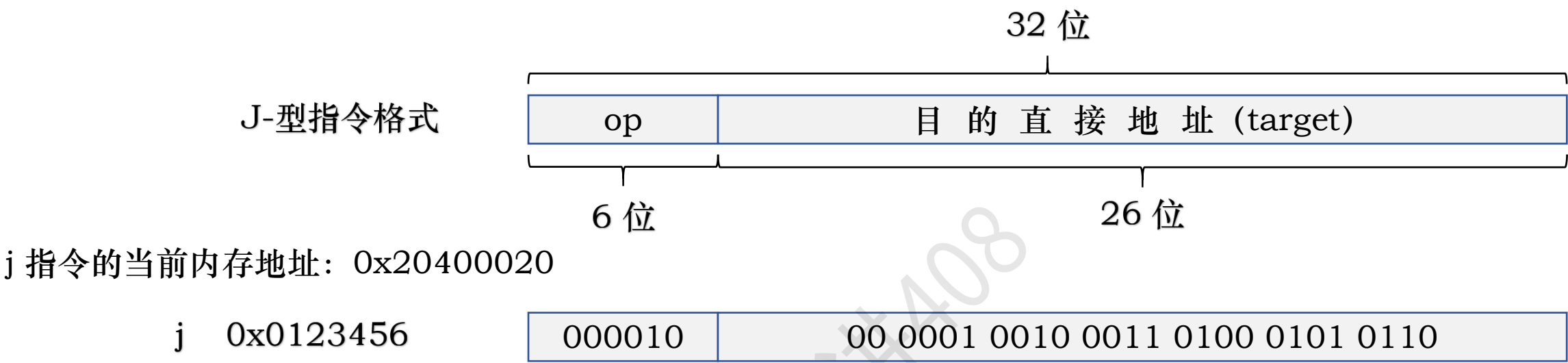


bne 指令的下一条指令的内存地址
若 $\$rs != \rt , 则 $PC = PC + 4 + (offset \ll 2)$

中央处理器 (CPU) 设计: 分析每条指令的功能

add rd, rs, rt	$R[rd] \leftarrow R[rs] + R[rt]$	从寄存器 rs、rt 中取数，相加、减、或， 结果送到寄存器 rd 中
sub rd, rs, rt	$R[rd] \leftarrow R[rs] - R[rt]$	
or rd, rs, rt	$R[rd] \leftarrow R[rs] \mid R[rt]$	
slt rd, rs, rt	$\begin{aligned} &\text{if } (R[rs] < R[rt]) \ R[rd] \leftarrow 1 \\ &\text{else } R[rd] \leftarrow 0 \end{aligned}$	从 rs、rt 取数后，按有符号整数来判断两数大小， 如果小于，rd 设置为 1，否则，rd 设置为 0
lw rt, imm16(rs)	$\begin{aligned} &\text{Addr} \leftarrow R[rs] + \text{SignExt}(\text{imm16}) \\ &R[rt] \leftarrow M[\text{Addr}] \end{aligned}$	从 rs 取数，将 imm16 符号扩展，然后相加， 结果作为访存地址 Addr，从 Addr 取数并送 rt
sw rt, imm16(rs)	$\begin{aligned} &\text{Addr} \leftarrow R[rs] + \text{SignExt}(\text{imm16}) \\ &M[\text{Addr}] \leftarrow R[rt] \end{aligned}$	从 rs 取数，将 imm16 符号扩展，然后相加， 结果作为访存地址 Addr，将 rt 送 Addr 中
beq rs, rt, imm16	$\begin{aligned} &\text{Cond} \leftarrow R[rs] - R[rt] \\ &\text{if } (\text{Cond} == 0) \\ &\quad PC \leftarrow PC + 4 + (\text{SignExt}(\text{imm16}) \times 4) \end{aligned}$	做减法以比较 rs 和 rt 中内容的大小，并计 算下条指令地址，然后根据比较结果修改 PC
j target	$PC[31:0] \leftarrow PC[31:28] \mid \mid (\text{target}[25:0] \ll 2)$	无条件跳转到指定的目的地址所在指令

MIPS 指令系统：J-型指令格式（无条件跳转指令 j） - 第 5 章的第 16 小节



步骤 1：扩展目标地址 $0x0123456 \ll 2 = 0x048D158$

步骤 2：拼接地址 $0x20400020 \& 0xF0000000 = 0x20000000$

目的地址 = $0x20000000 \mid 0x048D158 = 0x2048D158$

目的地址 = (当前指令地址 & 0xF0000000) | (target << 2)

中央处理器 (CPU) 设计的五个步骤

第一步：分析每条指令的功能

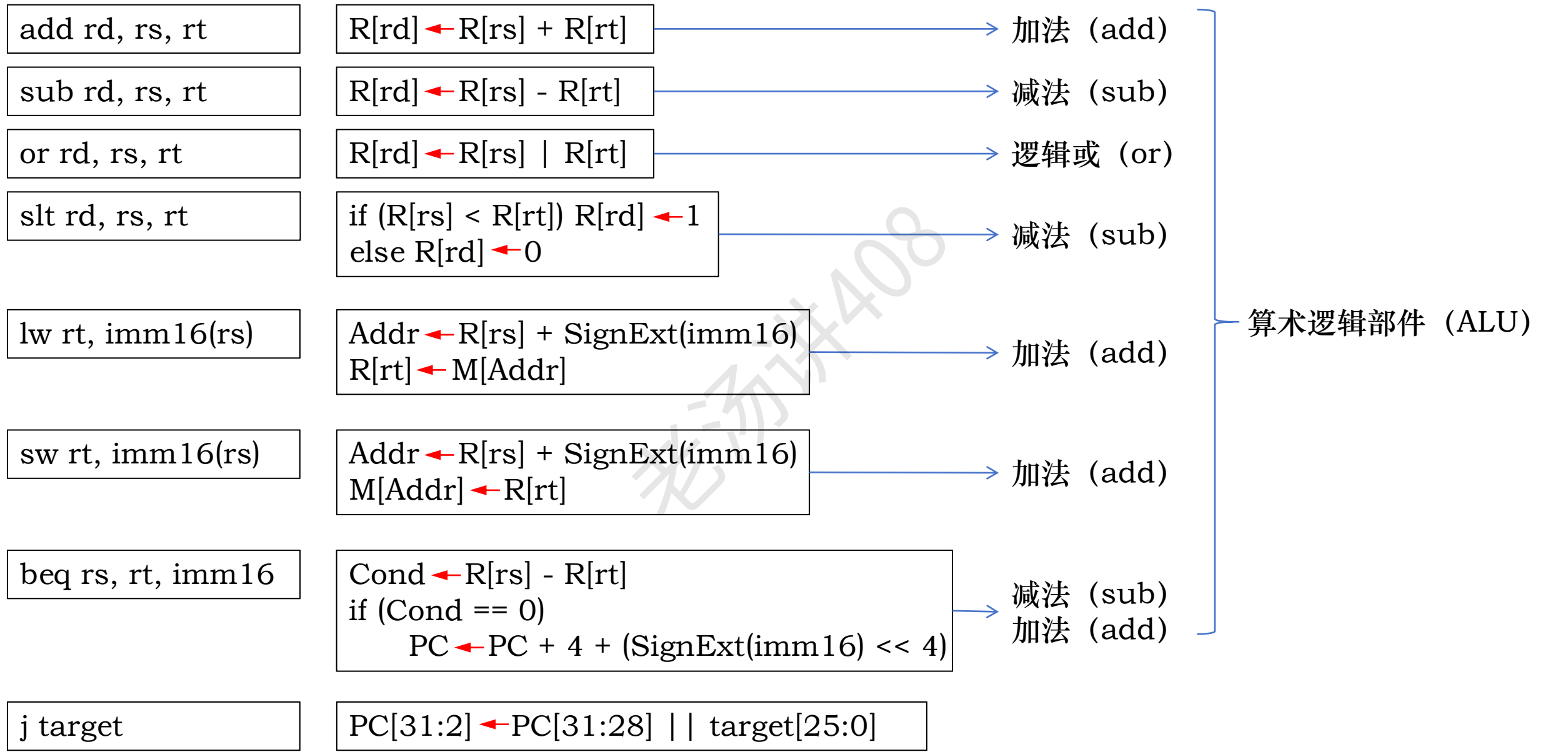
第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

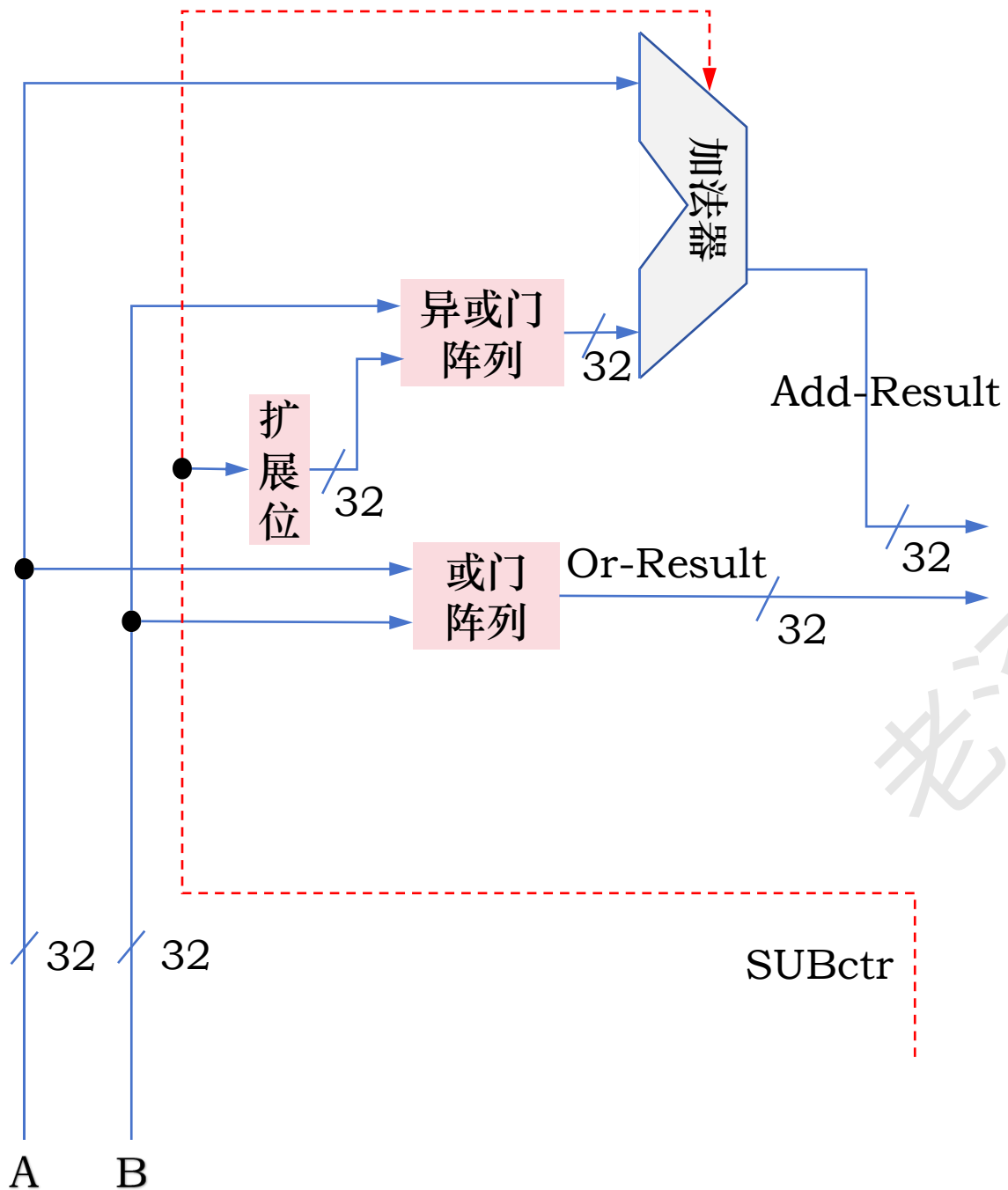
第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路

中央处理器 (CPU) 设计: 给出所需的元件



设计 ALU 部件



加法 (add)

减法 (sub)

逻辑或 (or)

	0	1
$\wedge$	0	0
	0	1

$$A - B = A + [-B]_{\text{补}} = A + \overline{B} + 1$$

	0	1
$\wedge$	1	1
	1	0

设计 ALU 部件

加法 (add)

减法 (sub)

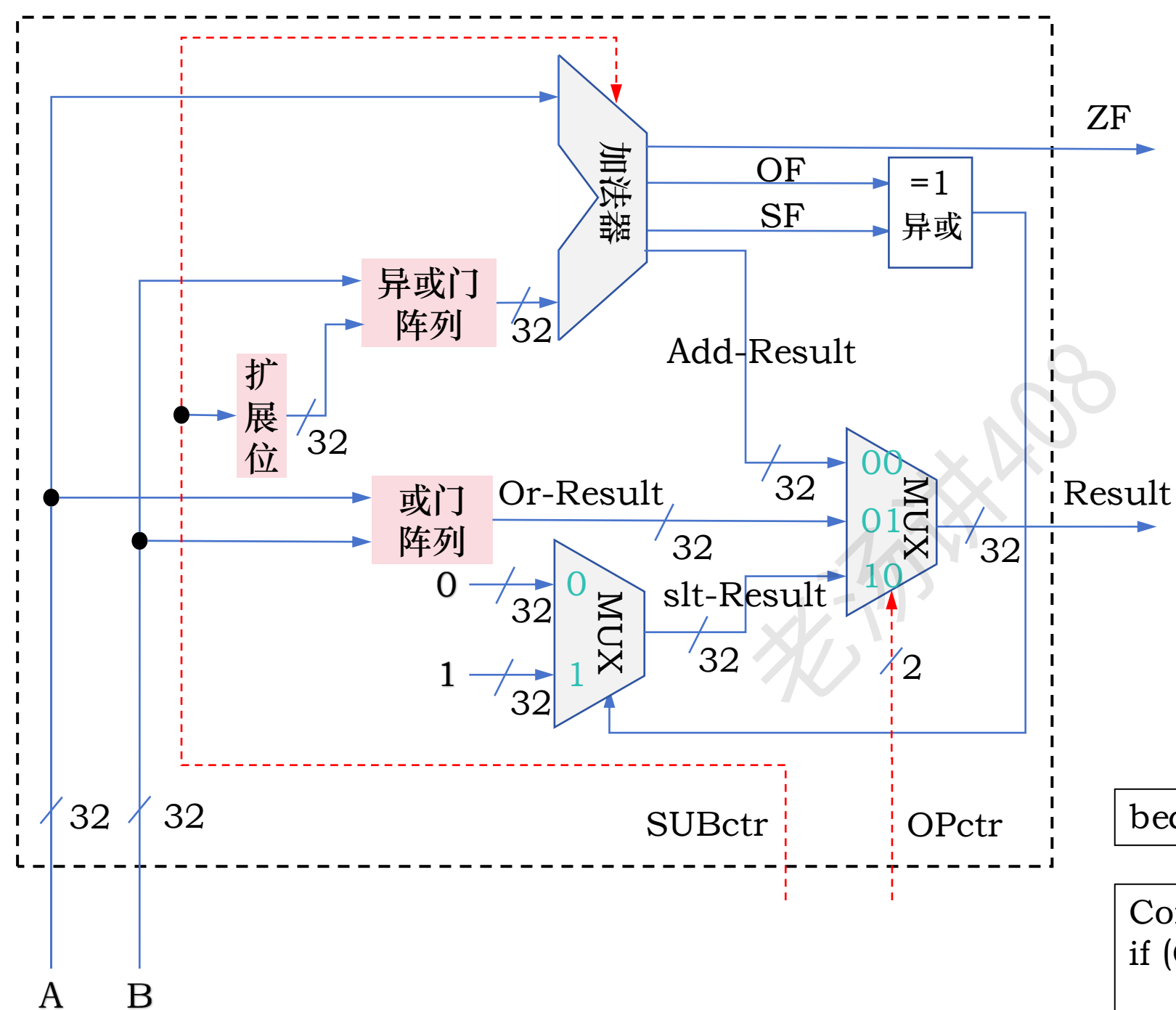
逻辑或 (or)

slt rd, rs, rt

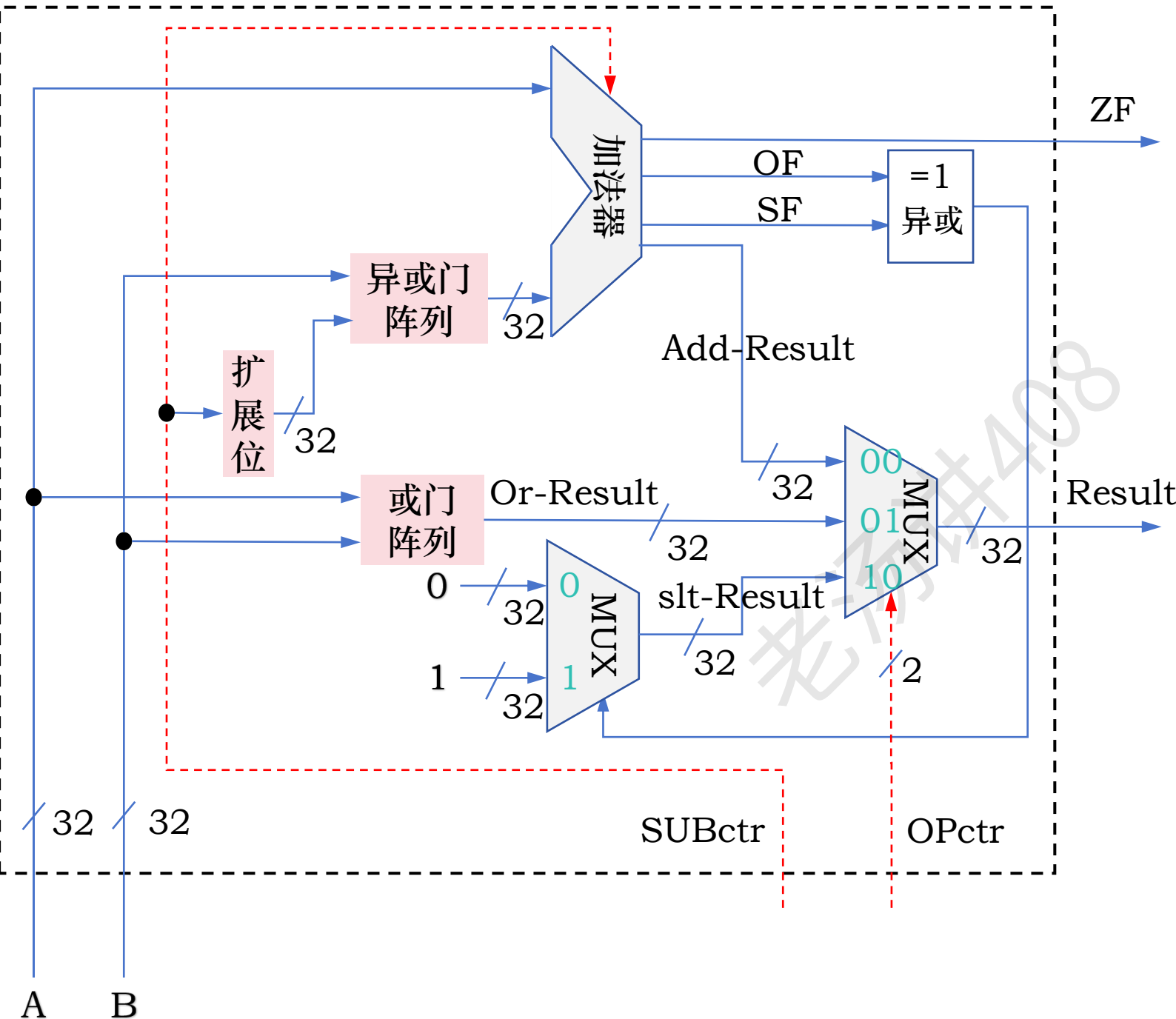
if (R[rs] < R[rt]) R[rd] $\leftarrow$ 1
else R[rd] $\leftarrow$ 0

beq rs, rt, imm16

Cond $\leftarrow$ R[rs] - R[rt]
if (Cond == 0)
 PC $\leftarrow$ PC + 4 + (SignExt(imm16) $\times$ 4)



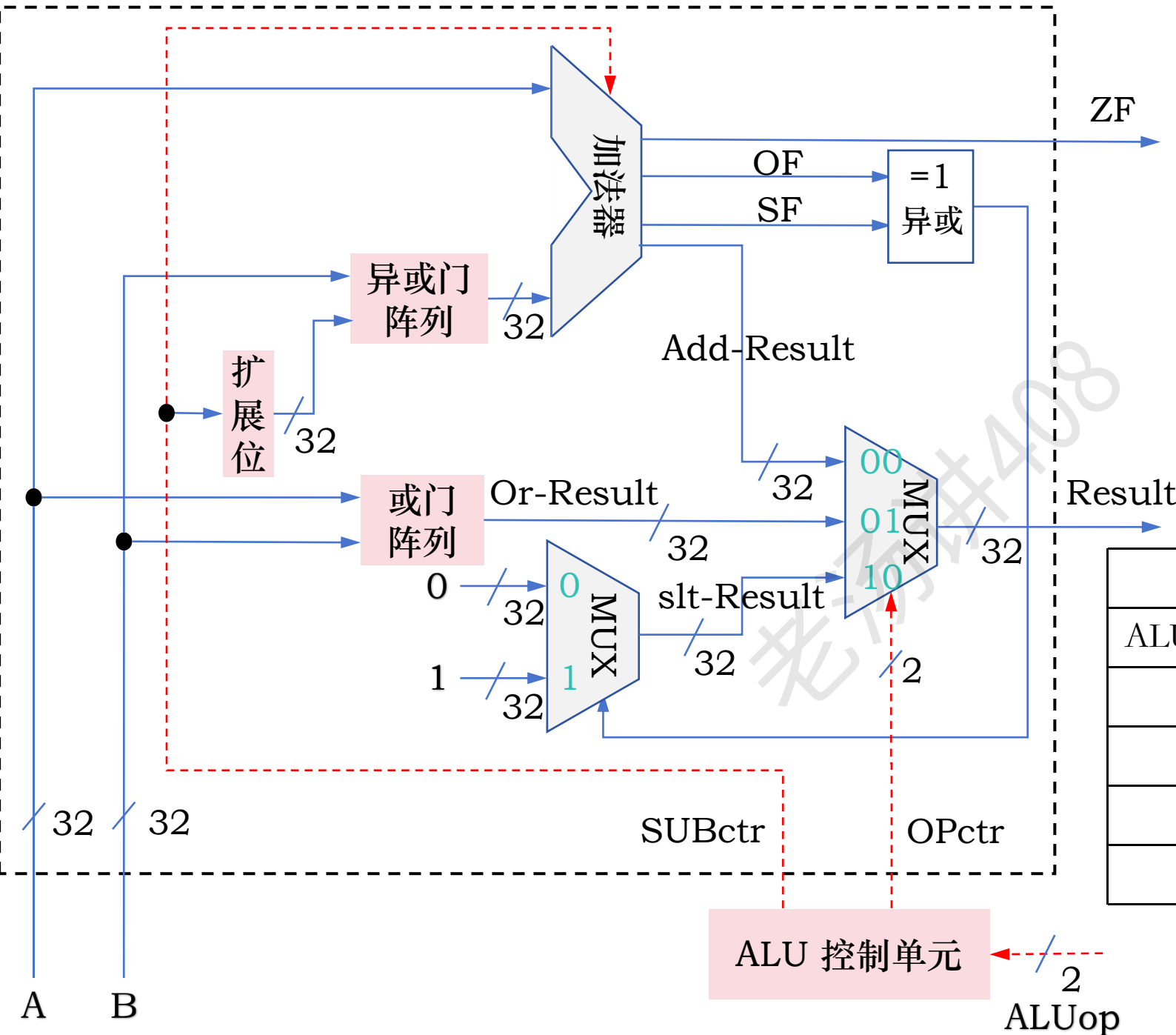
设计 ALU 部件



操作	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0

SUBctr	OPctr	
0	00	加法 (add)
1	00	减法 (sub)
×	01	逻辑或 (or)
1	10	小于置 1 (slt)

设计 ALU 部件



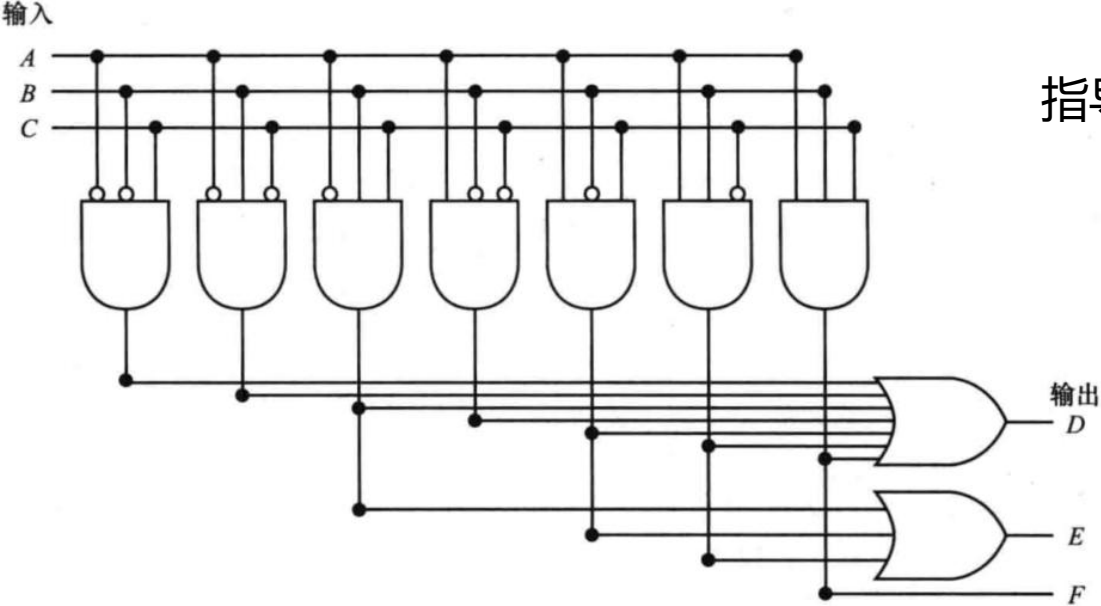
操作	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0

输入		输出		
ALUop1	ALUop0	SUBctr	OPctr1	OPctr0
0	0	0	0	0
0	1	1	0	0
1	0	×	0	1
1	1	1	1	0

真值表

补一补数字电路基础：数字电路实现

可编程逻辑阵列 (PLA)



指导实现数字电路



输出 $D = A + B + C$

输出 $E = A \cdot B \cdot \overline{C} + A \cdot C \cdot \overline{B} + B \cdot C \cdot \overline{A}$

输出 $F = A \cdot B \cdot C$



输出逻辑表达式

数字电路实现的功能是：

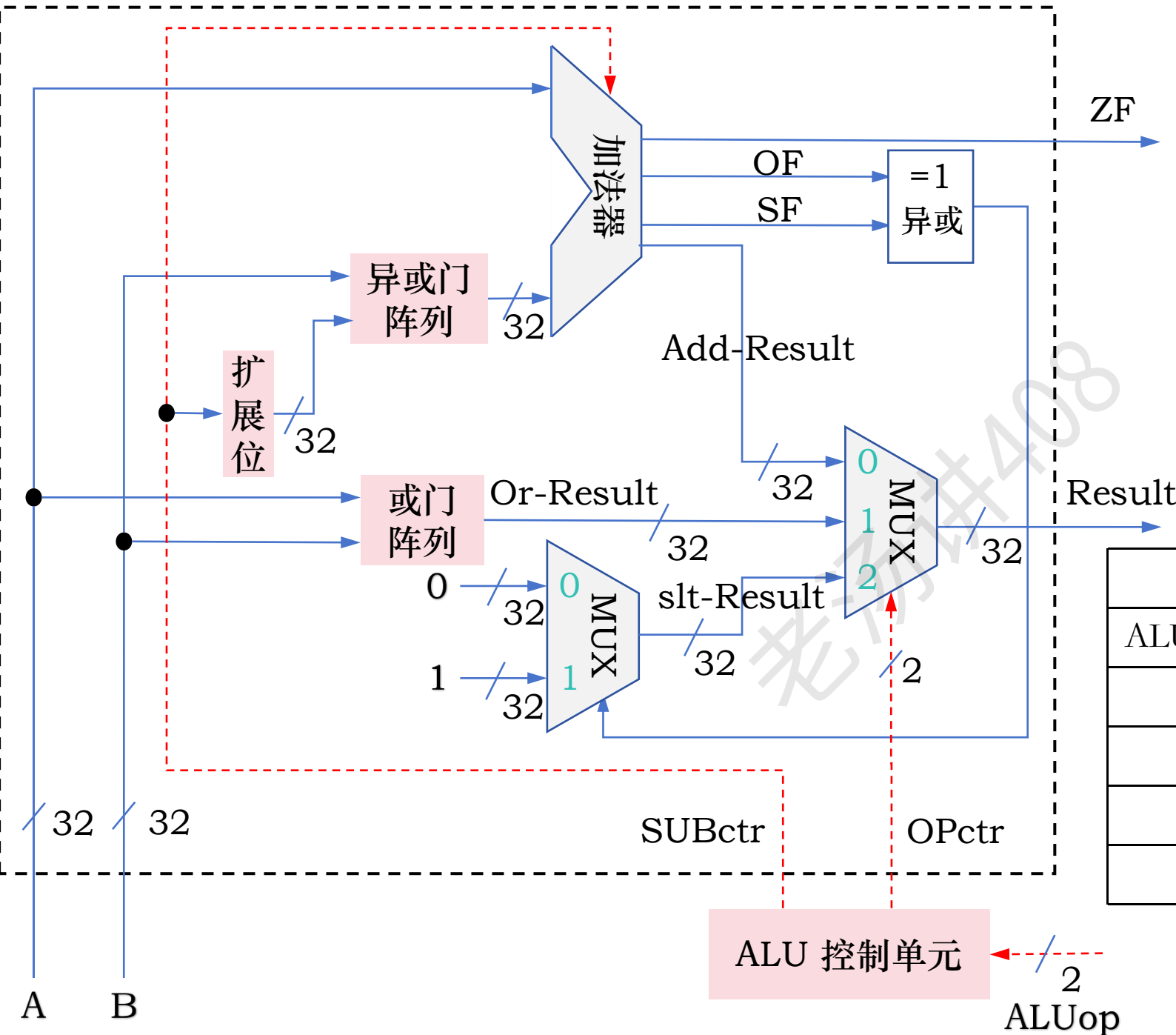
- 如果有一个输入为 1，则输出 D 为 1；
- 如果只有两个输入为 1，则输出 E 为 1；
- 如果三个输入都是 1，则输出 F 为 1

真值表



输入			输出		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

设计 ALU 部件

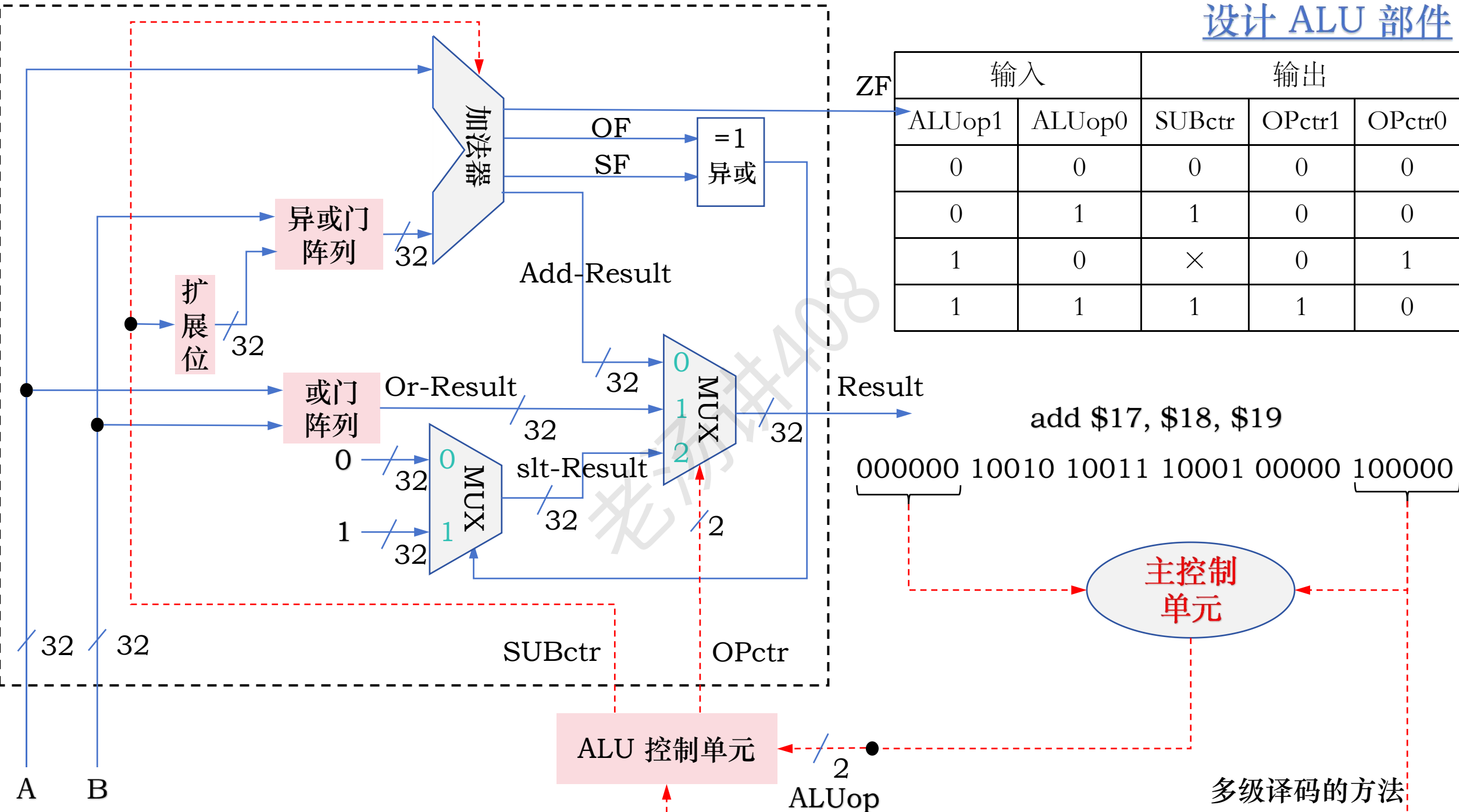


操作	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0

输入		输出		
ALUop1	ALUop0	SUBctr	OPctr1	OPctr0
0	0	0	0	0
0	1	1	0	0
1	0	×	0	1
1	1	1	1	0

真值表

设计 ALU 部件



指令	ALUop	funct 字段	ALU 控制信号	ALU 动作
lw	00	× × × × × ×	000	add
sw	00	× × × × × ×	000	add
beq	01	× × × × × ×	100	sub
add	10	100000	000	add
sub	10	100010	100	sub
or	10	100101	× 01	or
slt	10	101010	110	slt

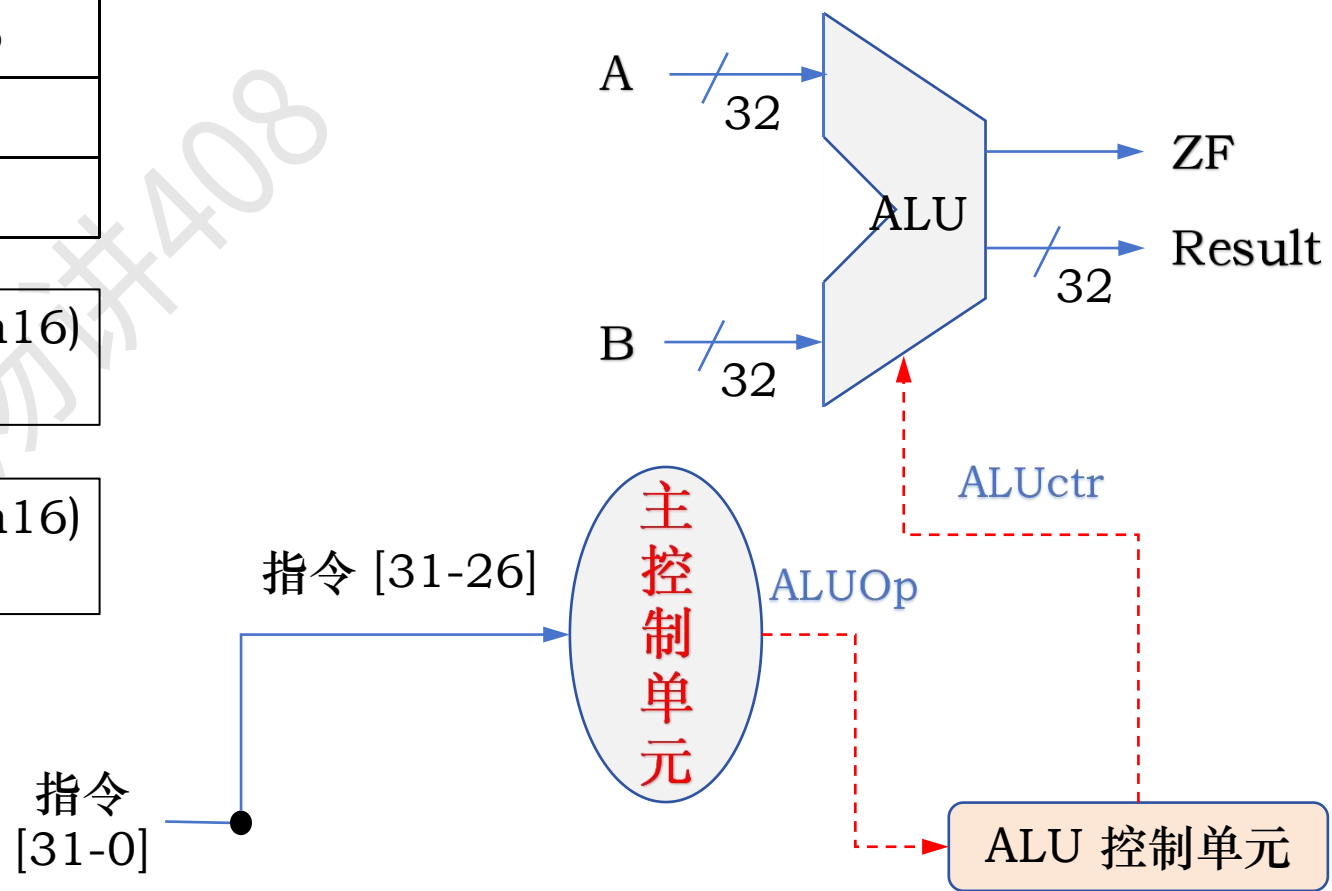
lw rt, imm16(rs) $\text{Addr} \leftarrow \text{R}[\text{rs}] + \text{SignExt}(\text{imm16})$
 $\text{R}[\text{rt}] \leftarrow \text{M}[\text{Addr}]$

sw rt, imm16(rs) $\text{Addr} \leftarrow \text{R}[\text{rs}] + \text{SignExt}(\text{imm16})$
 $\text{M}[\text{Addr}] \leftarrow \text{R}[\text{rt}]$

beq rs, rt, imm16

Cond $\text{R}[\text{rs}] - \text{R}[\text{rt}]$
if (Cond == 0)
PC $\text{PC} + 4 + (\text{SignExt}(\text{imm16}) \times 4)$

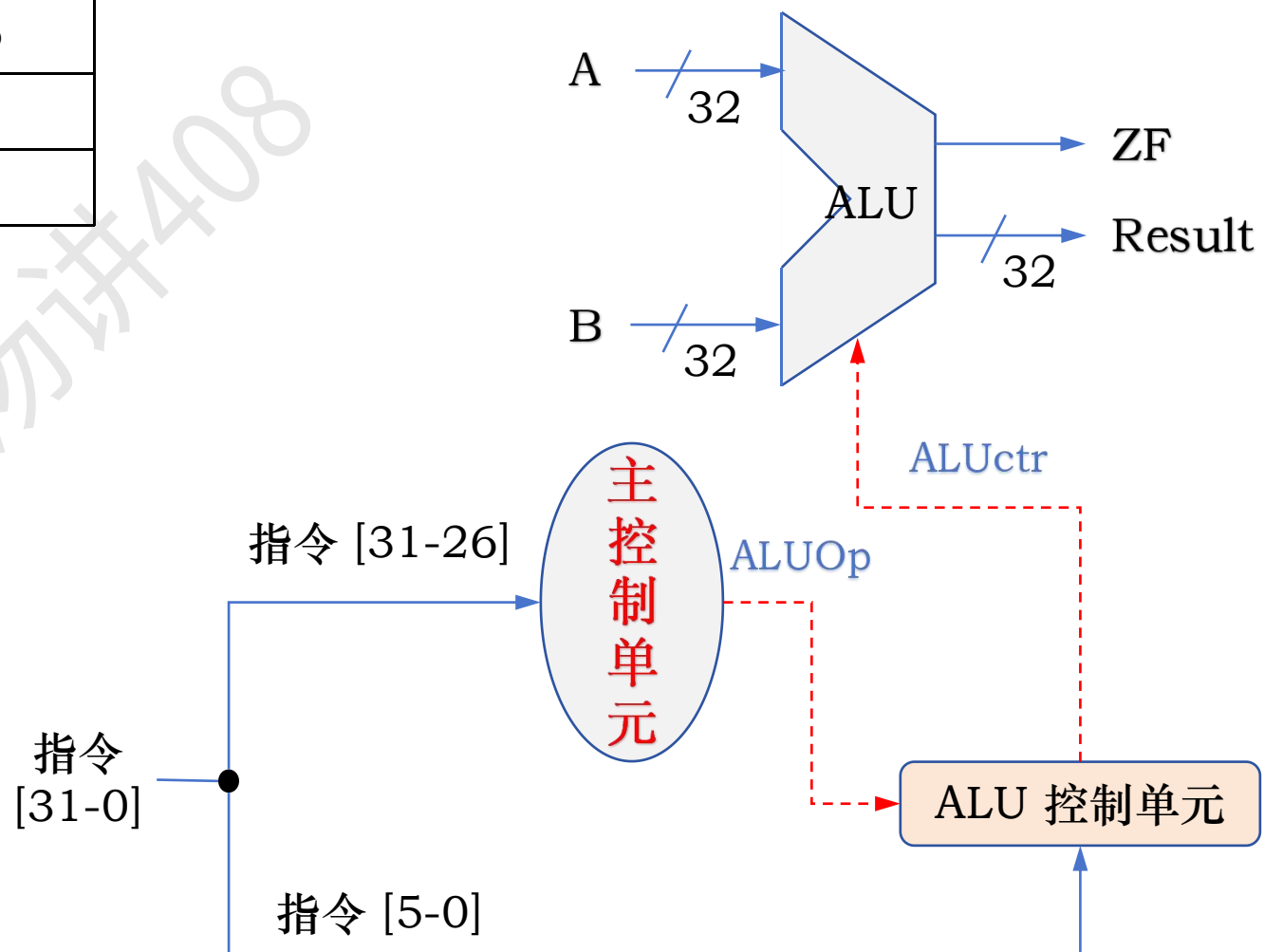
操作类型	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0



指令	ALUop	funct 字段	ALU 控制信号	ALU 动作
lw	00	× × × × × ×	000	add
sw	00	× × × × × ×	000	add
beq	01	× × × × × ×	100	sub
add	10	100000	000	add
sub	10	100010	100	sub
or	10	100101	× 01	or
slt	10	101010	110	slt

操作类型	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0

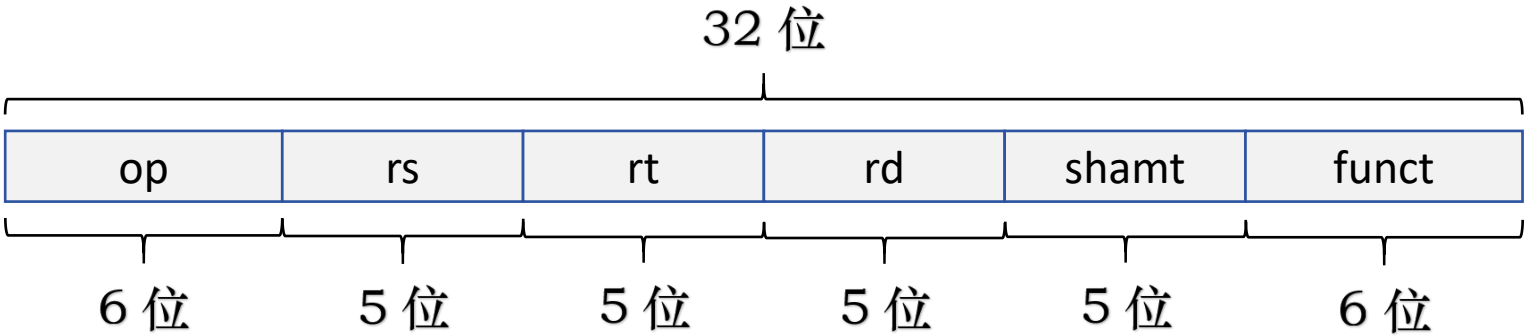
add rd, rs, rt	$R[rd] \leftarrow R[rs] + R[rt]$
sub rd, rs, rt	$R[rd] \leftarrow R[rs] - R[rt]$
or rd, rs, rt	$R[rd] \leftarrow R[rs] \mid R[rt]$



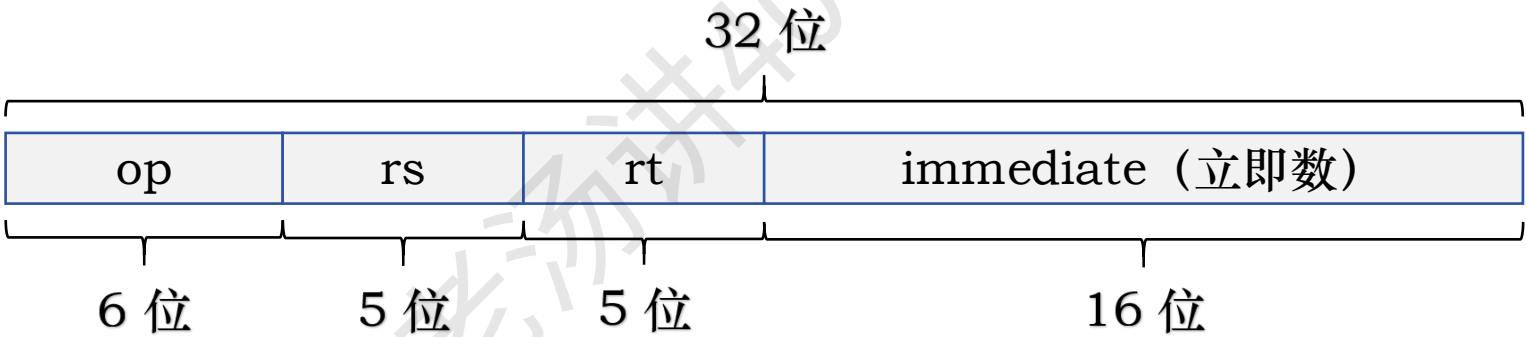
MIPS 指令系统：总结 (2、指令格式)

指令字长 = 32 位

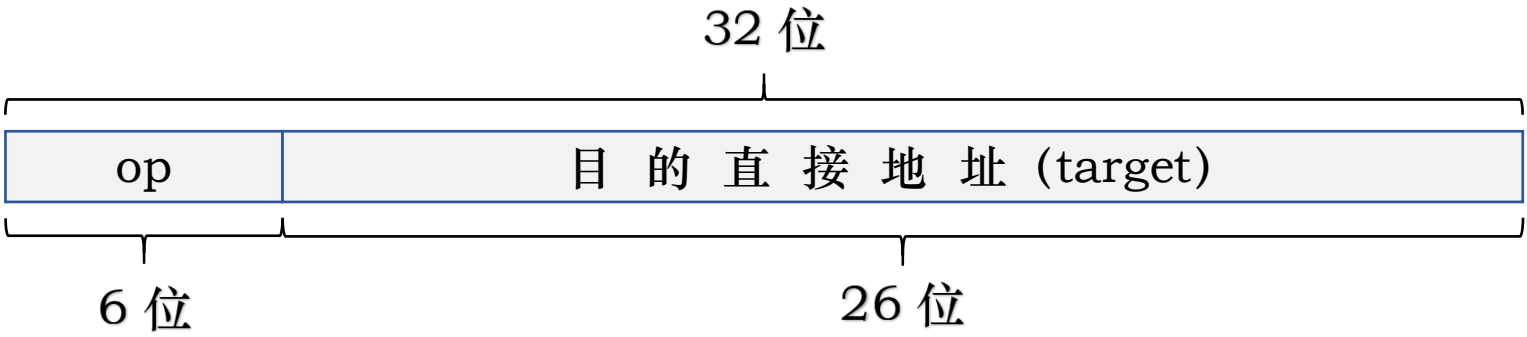
R-型指令格式



I-型指令格式



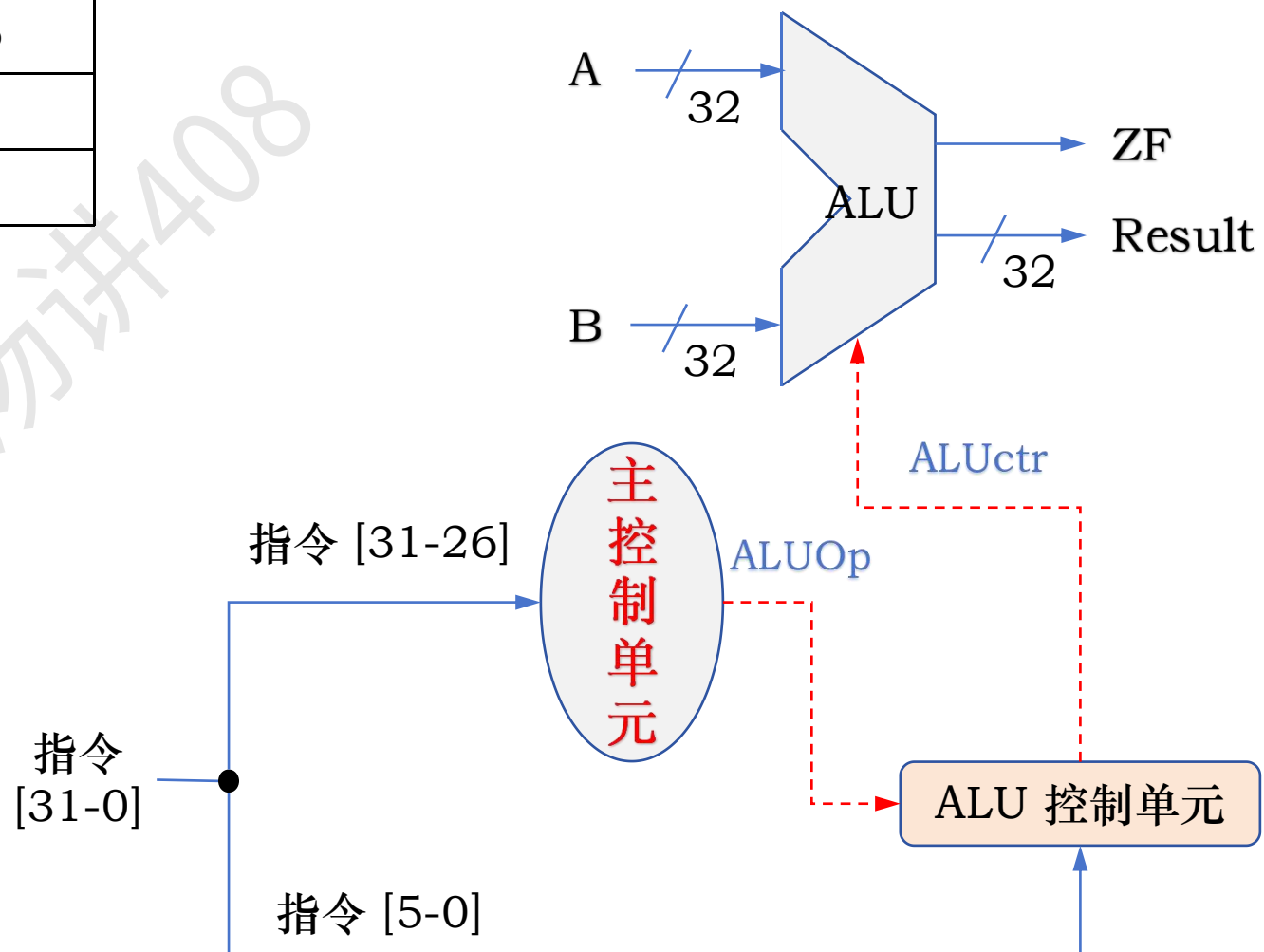
J-型指令格式



指令	ALUop	funct 字段	ALU 控制信号	ALU 动作
lw	00	× × × × × ×	000	add
sw	00	× × × × × ×	000	add
beq	01	× × × × × ×	100	sub
add	10	100000	000	add
sub	10	100010	100	sub
or	10	100101	× 01	or
slt	10	101010	110	slt

操作类型	SUBctr	OPctr1	OPctr0
add	0	0	0
sub	1	0	0
or	×	0	1
slt	1	1	0

add rd, rs, rt	$R[rd] \leftarrow R[rs] + R[rt]$
sub rd, rs, rt	$R[rd] \leftarrow R[rs] - R[rt]$
or rd, rs, rt	$R[rd] \leftarrow R[rs] \mid R[rt]$
slt rd, rs, rt	if ($R[rs] < R[rt]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$



ALU 控制单元电路的输入输出真值表

指令	ALUop	funct 字段	ALU 控制信号	ALU 动作
lw	00	× × × × × ×	000	add
sw	00	× × × × × ×	000	add
beq	01	× × × × × ×	100	sub
add	10	100000	000	add
sub	10	100010	100	sub
or	10	100101	× 01	or
slt	10	101010	110	slt



ALU 控制单元电路真值表

输入								输出		
ALUop1	ALUop0	func5	func4	func3	func2	func1	func0	ALUctr2	ALUctr1	ALUctr0
0	0	×	×	×	×	×	×	0	0	0
0	0	×	×	×	×	×	×	0	0	0
0	1	×	×	×	×	×	×	1	0	0
1	0	1	0	0	0	0	0	0	0	0
1	0	1	0	0	0	1	0	1	0	0
1	0	1	0	0	1	0	1	×	0	1
1	0	1	0	1	0	1	0	1	1	0

中央处理器 (CPU) 设计的五个步骤

第一步：分析每条指令的功能

第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

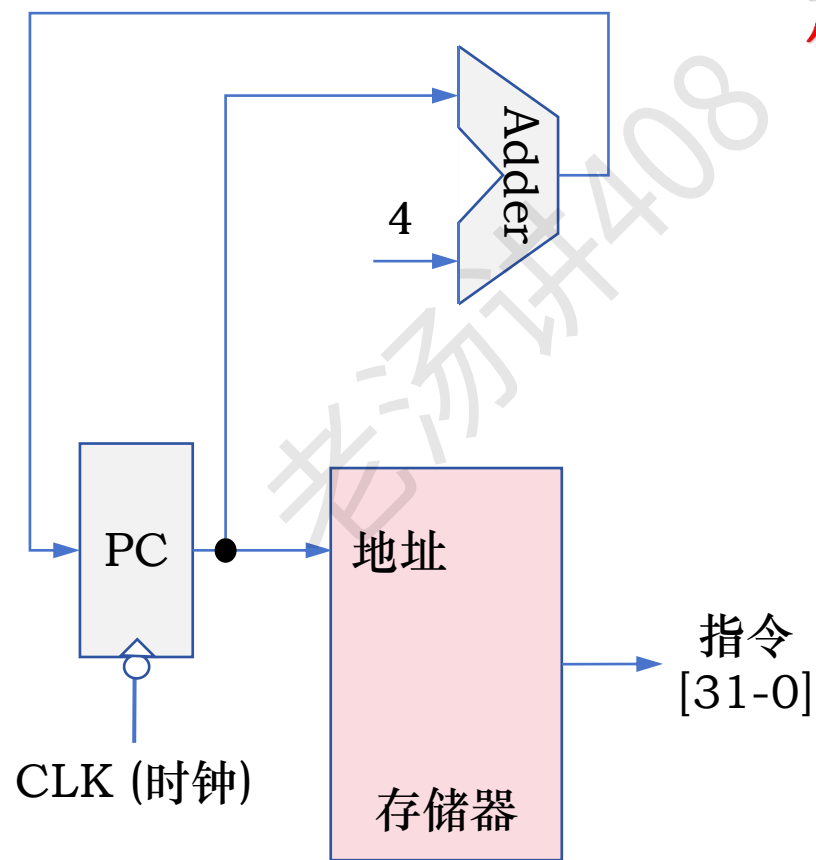
第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路

取指令部件的设计

单周期 CPU

每个时钟周期执行一条指令

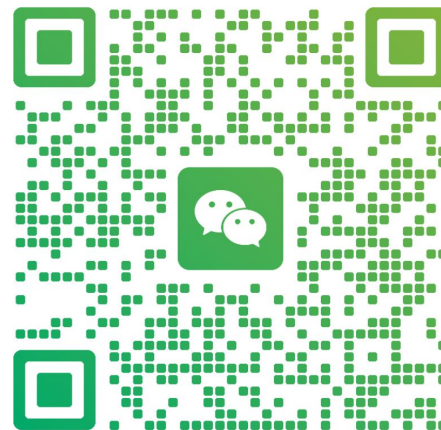
CPI (一条指令需要的时钟周期) = 1



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

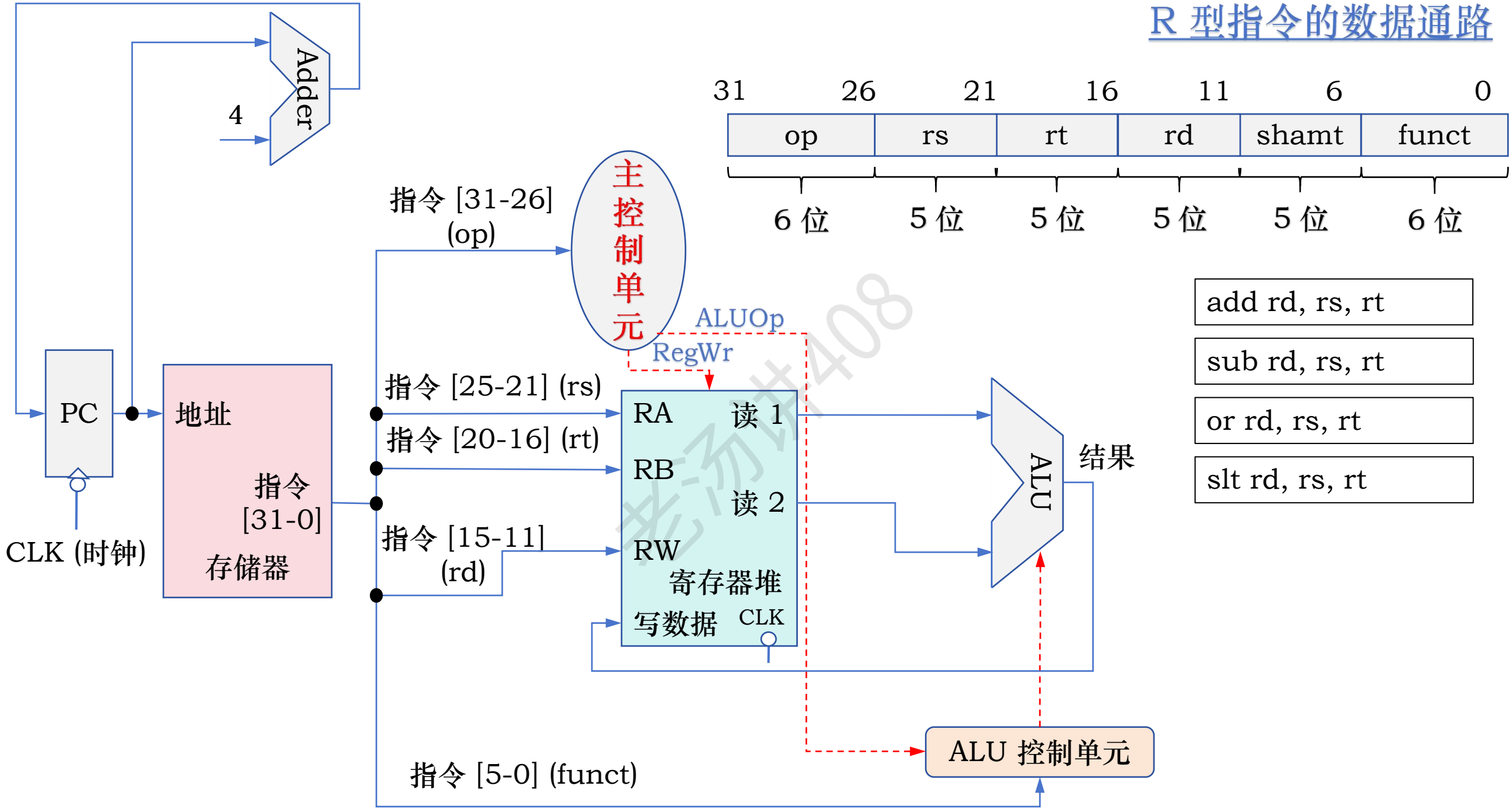


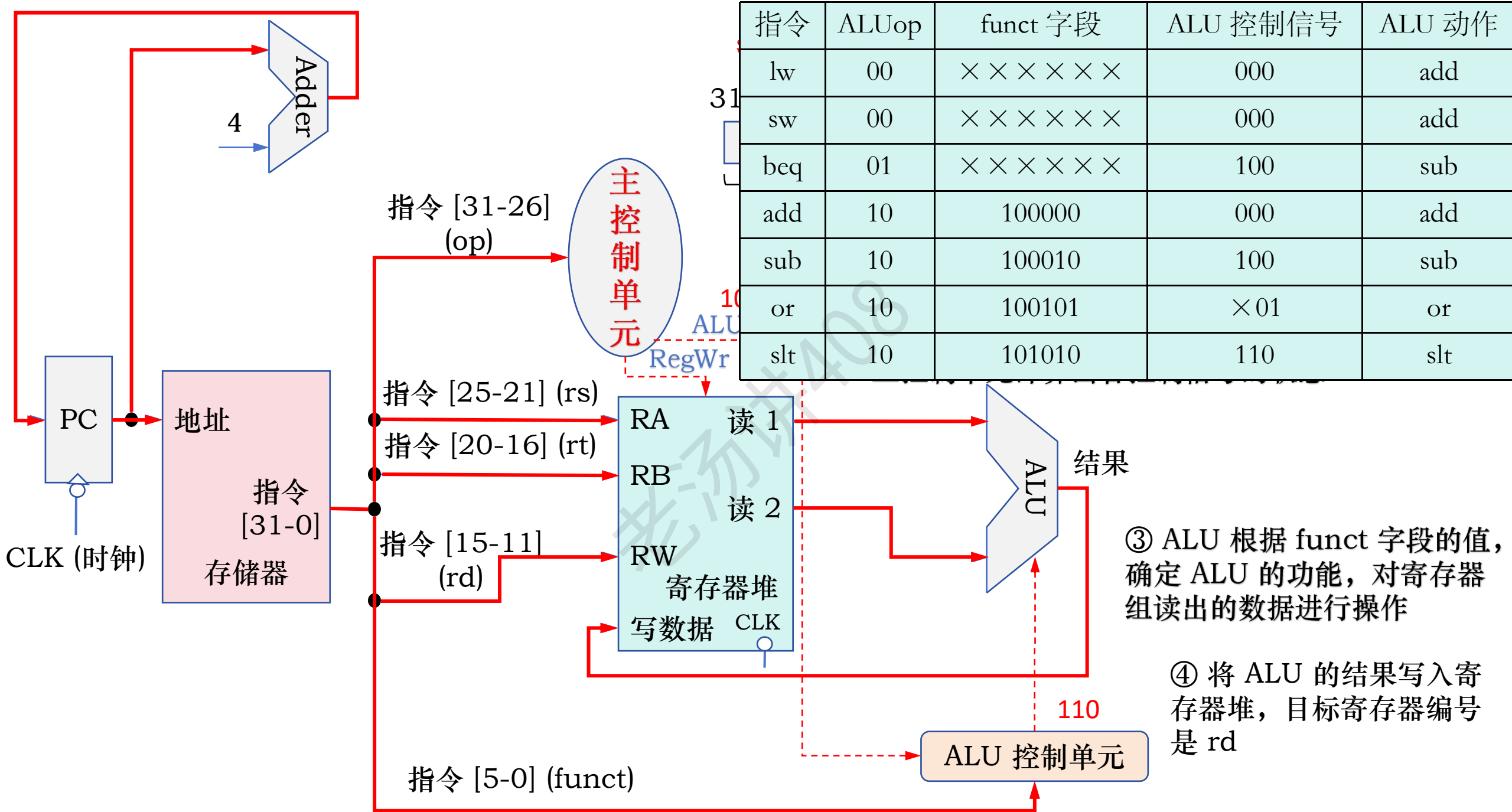
老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

R 型指令的数据通路



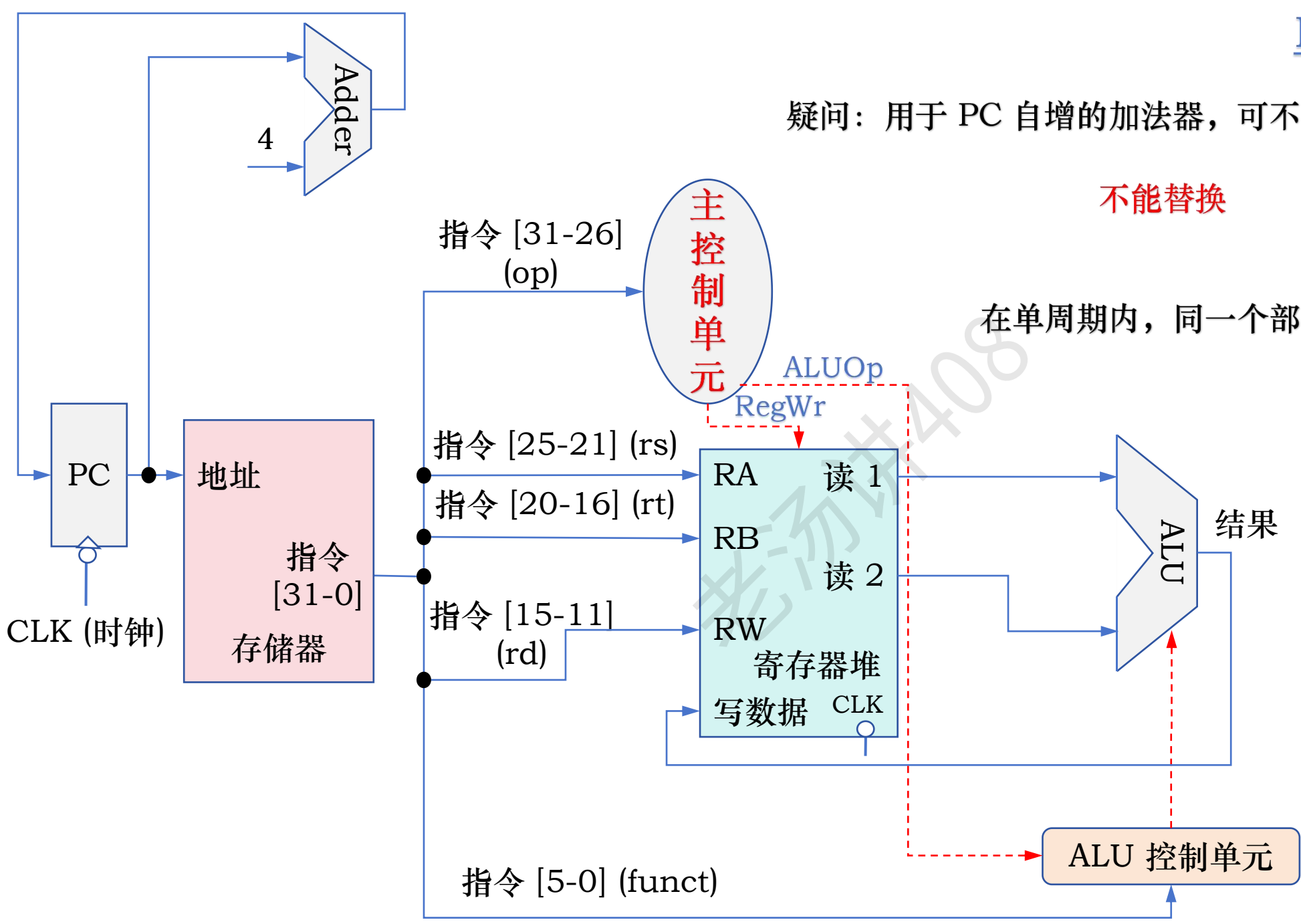


R 型指令的数据通路

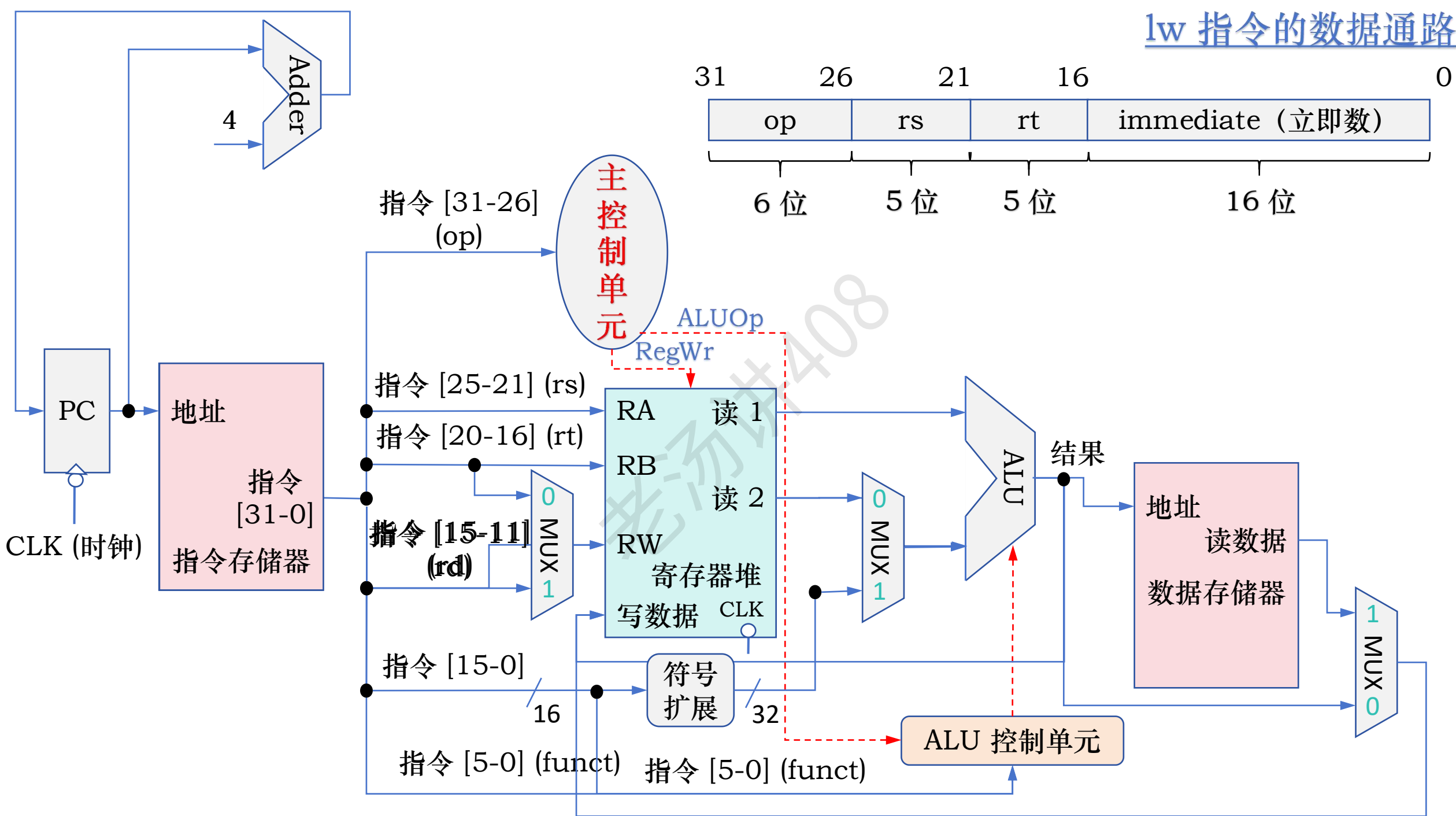
疑问：用于 PC 自增的加法器，可不可以使用 ALU 来代替？

不能替换

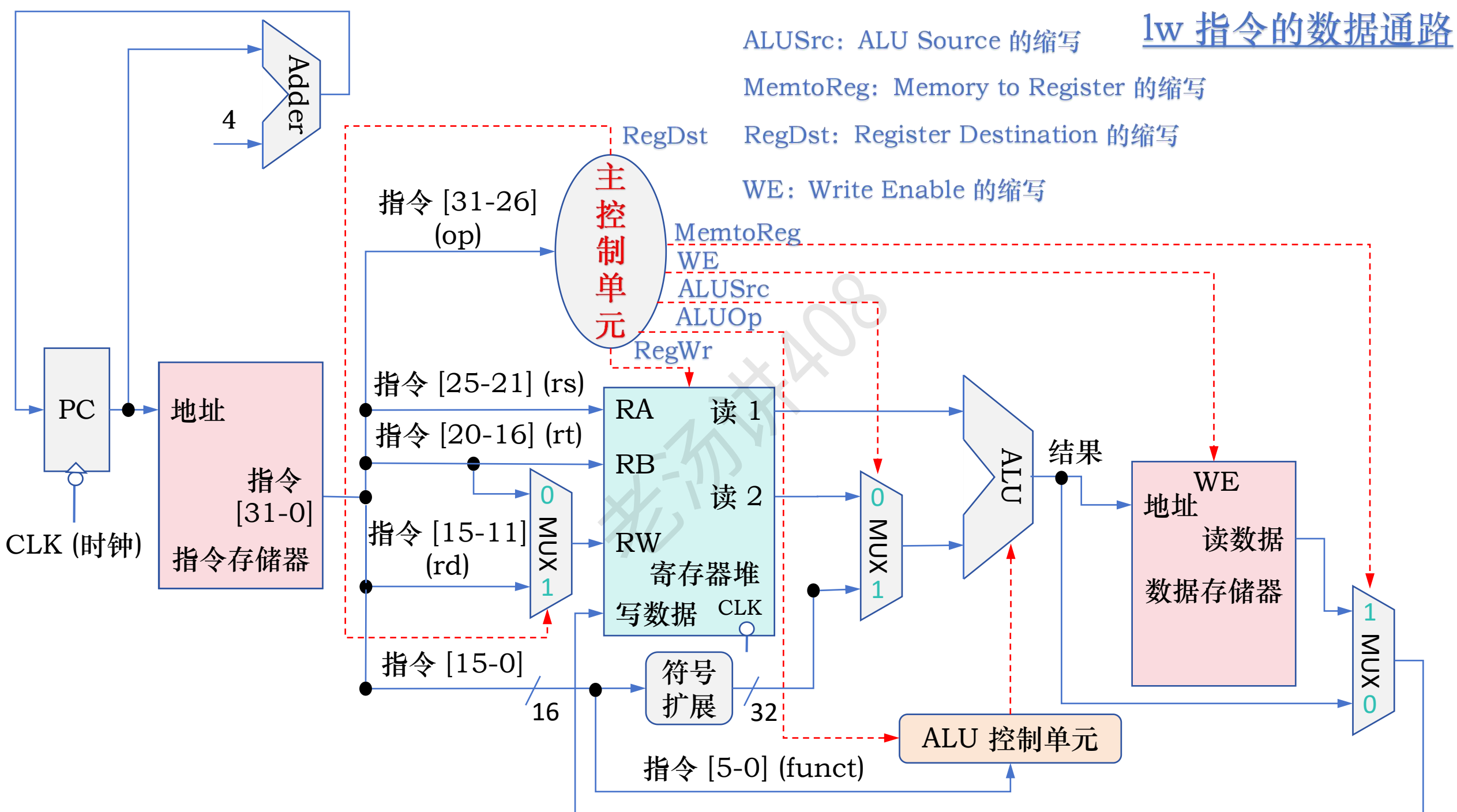
在单周期内，同一个部件就不能被使用两次

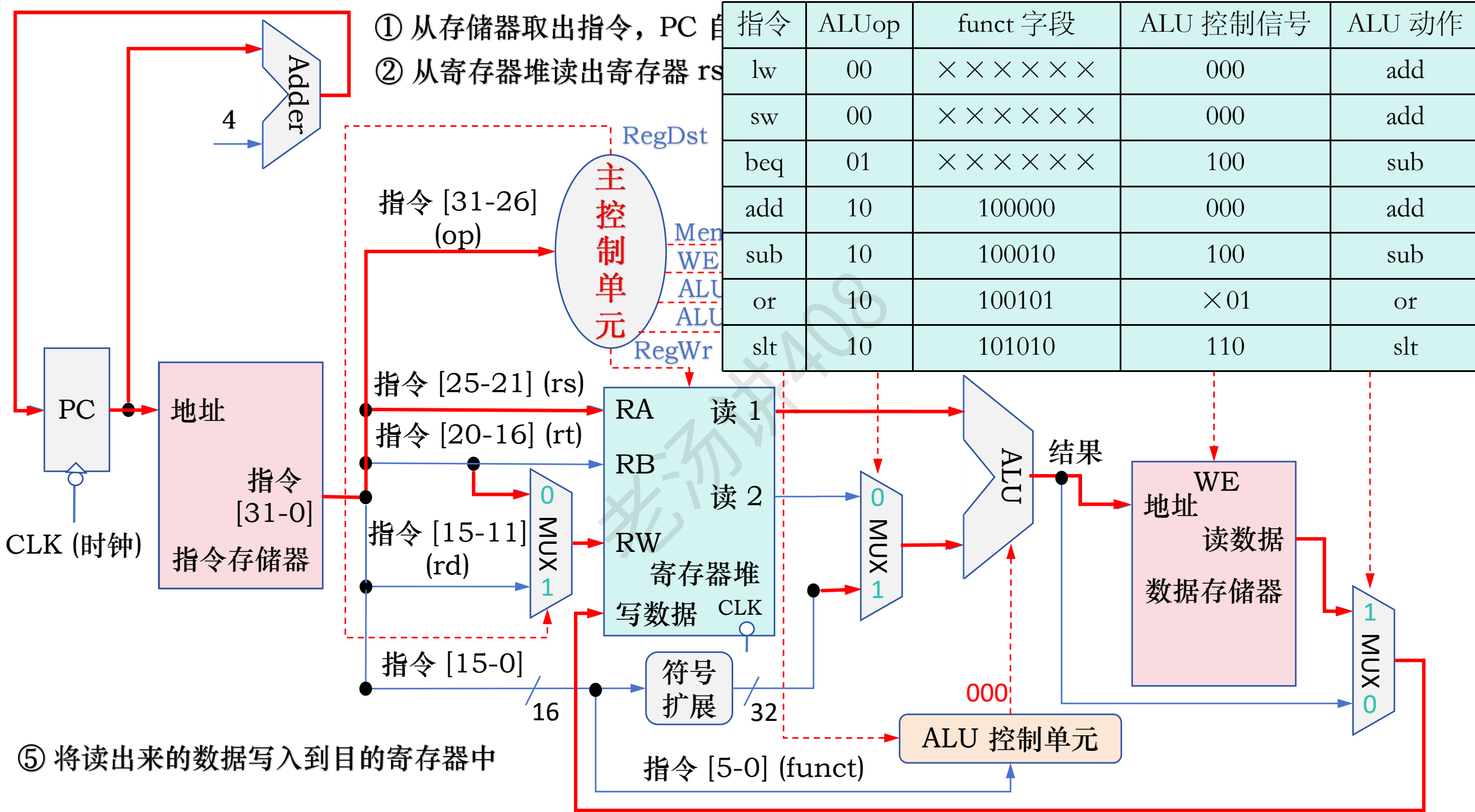


lw 指令的数据通路



lw 指令的数据通路





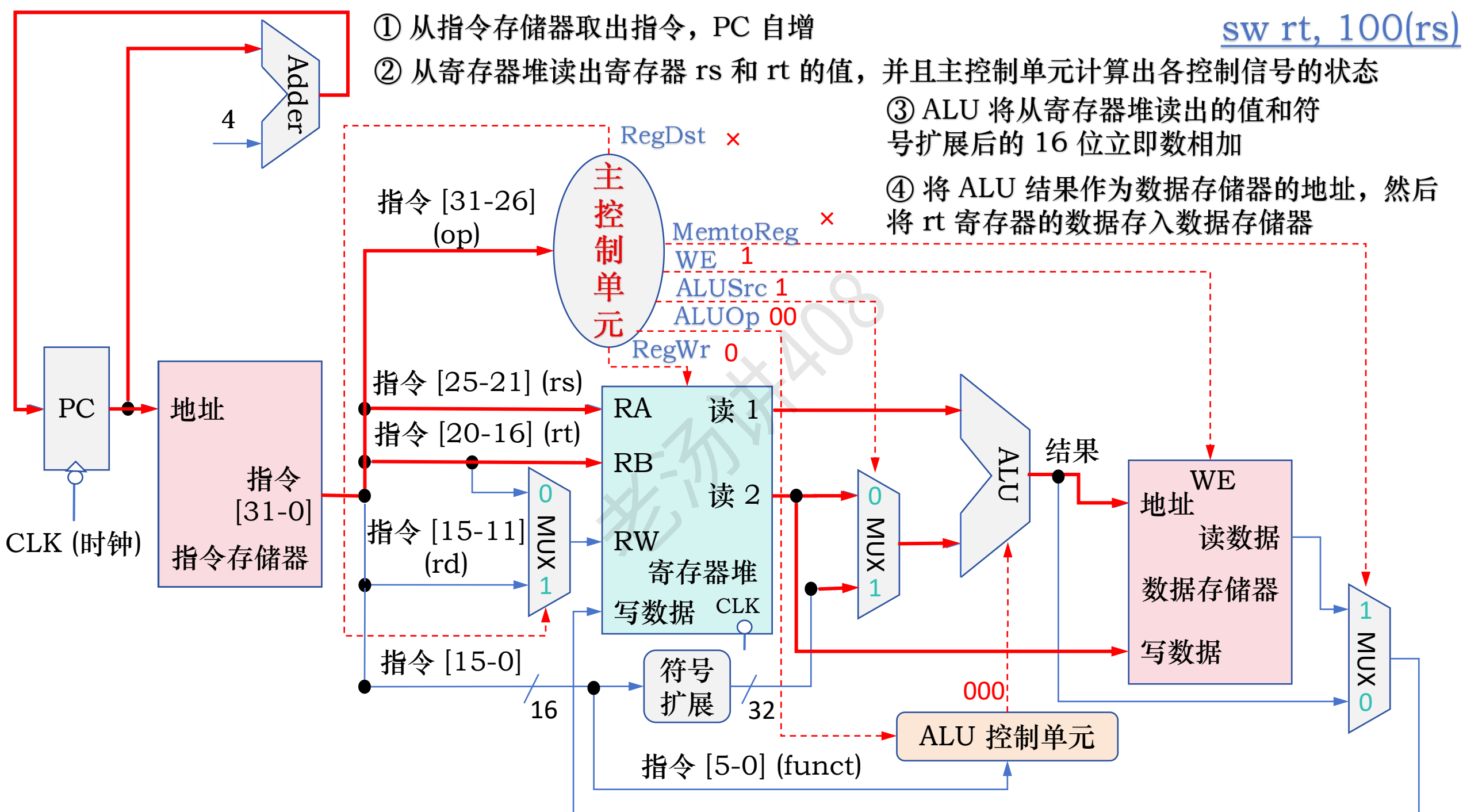
sw rt, 100(rs)

① 从指令存储器取出指令，PC 自增

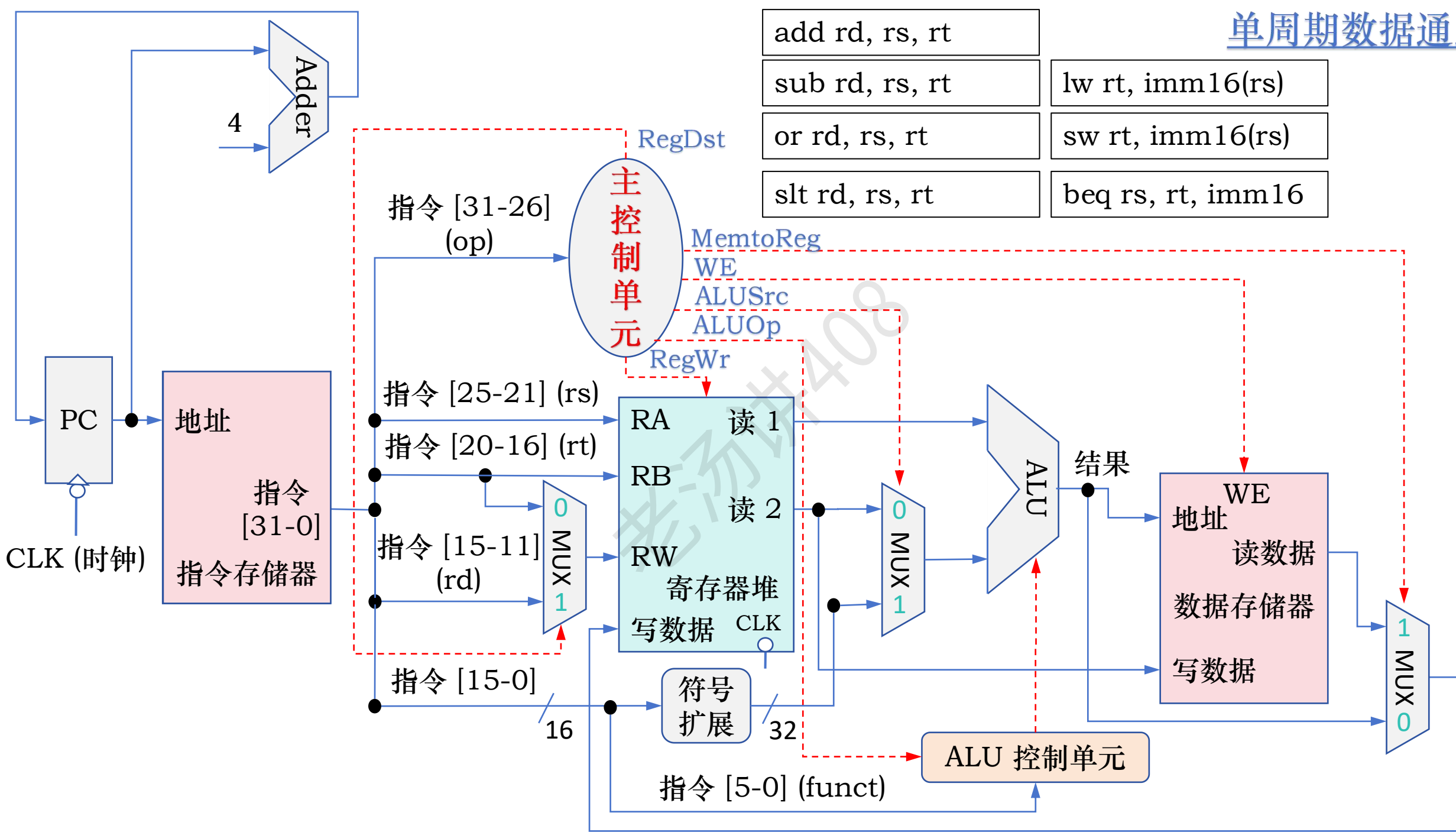
② 从寄存器堆读出寄存器 rs 和 rt 的值，并且主控制单元计算出各控制信号的状态

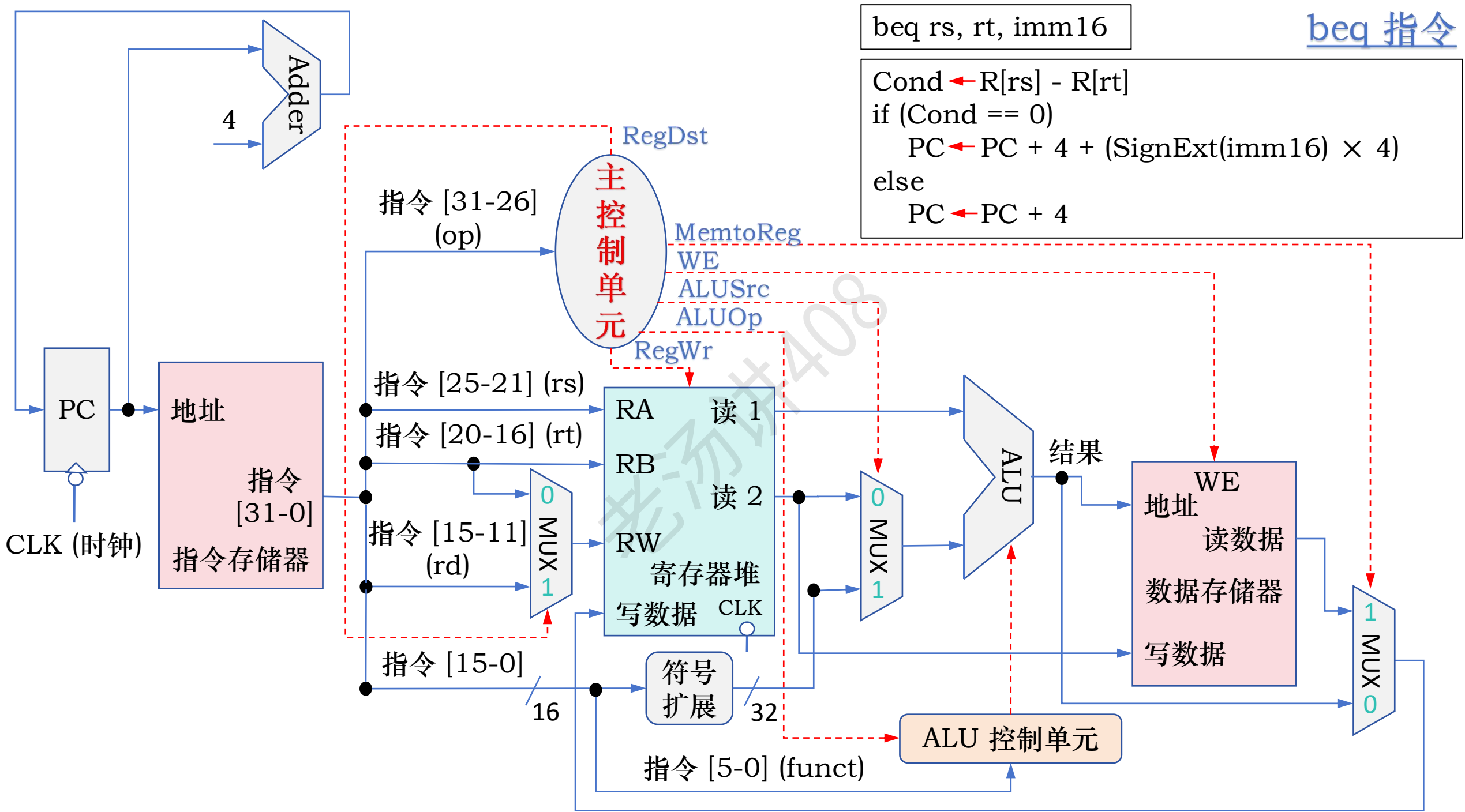
③ ALU 将从寄存器堆读出的值和符号扩展后的 16 位立即数相加

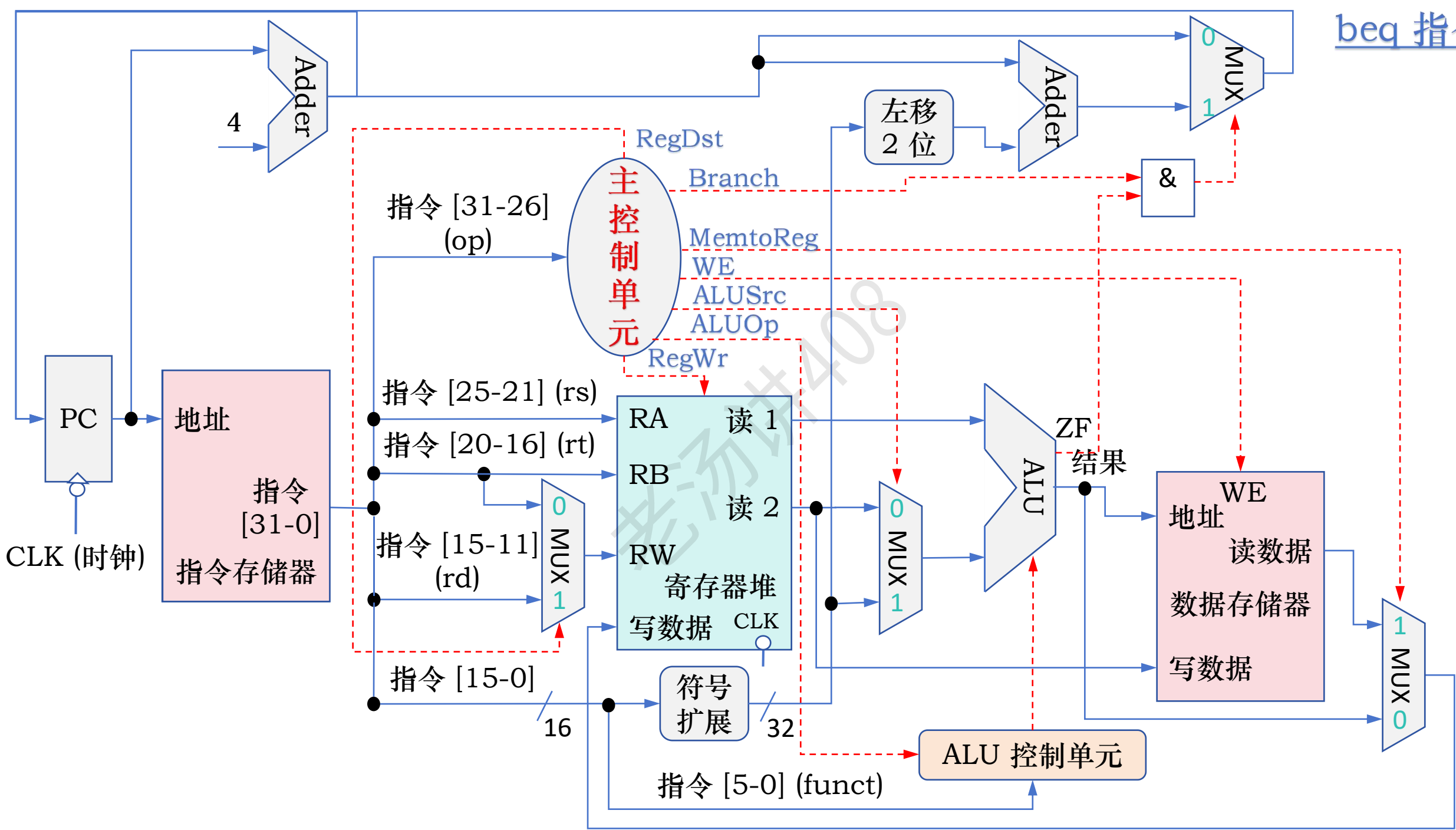
④ 将 ALU 结果作为数据存储器的地址，然后将 rt 寄存器的数据存入数据存储器

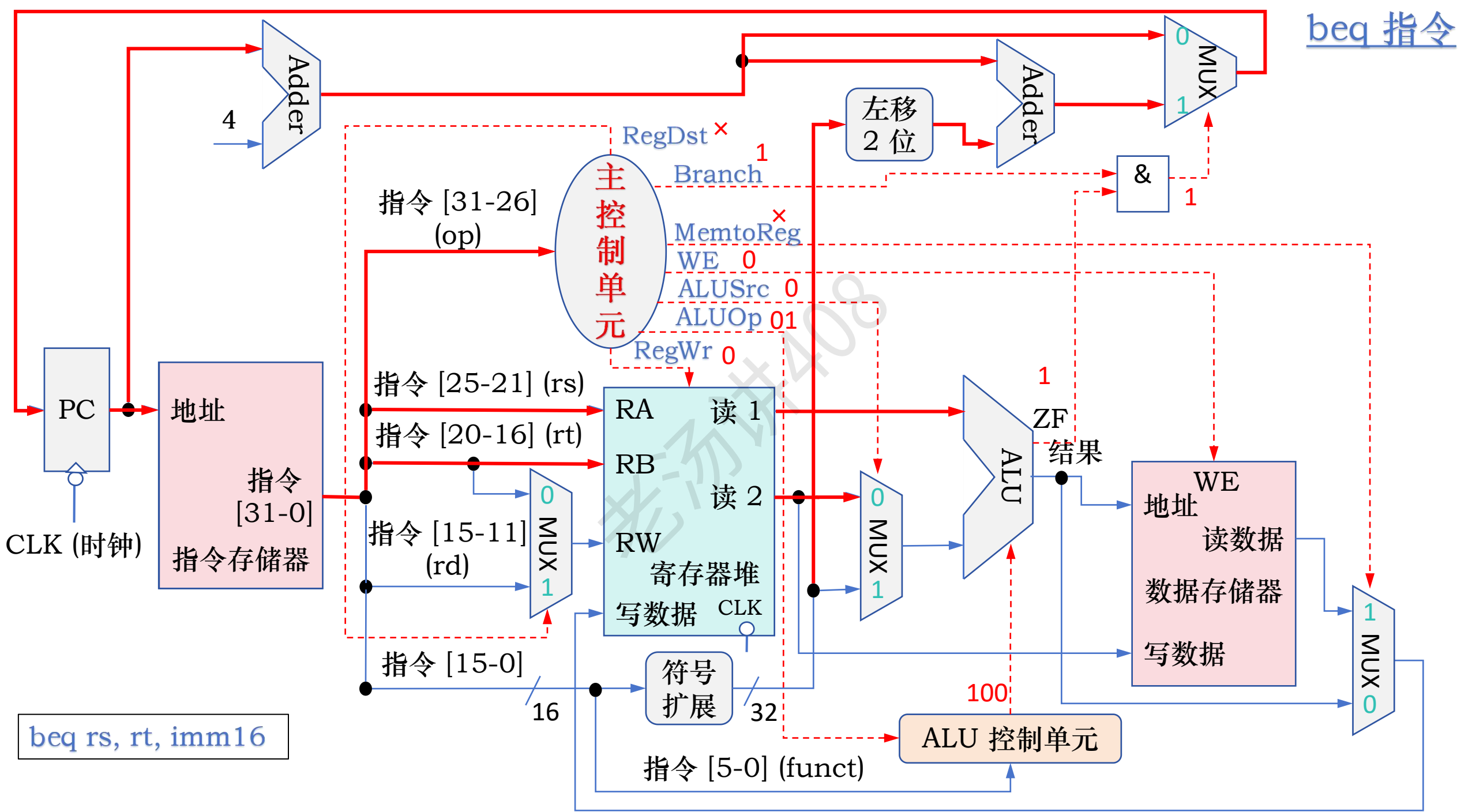


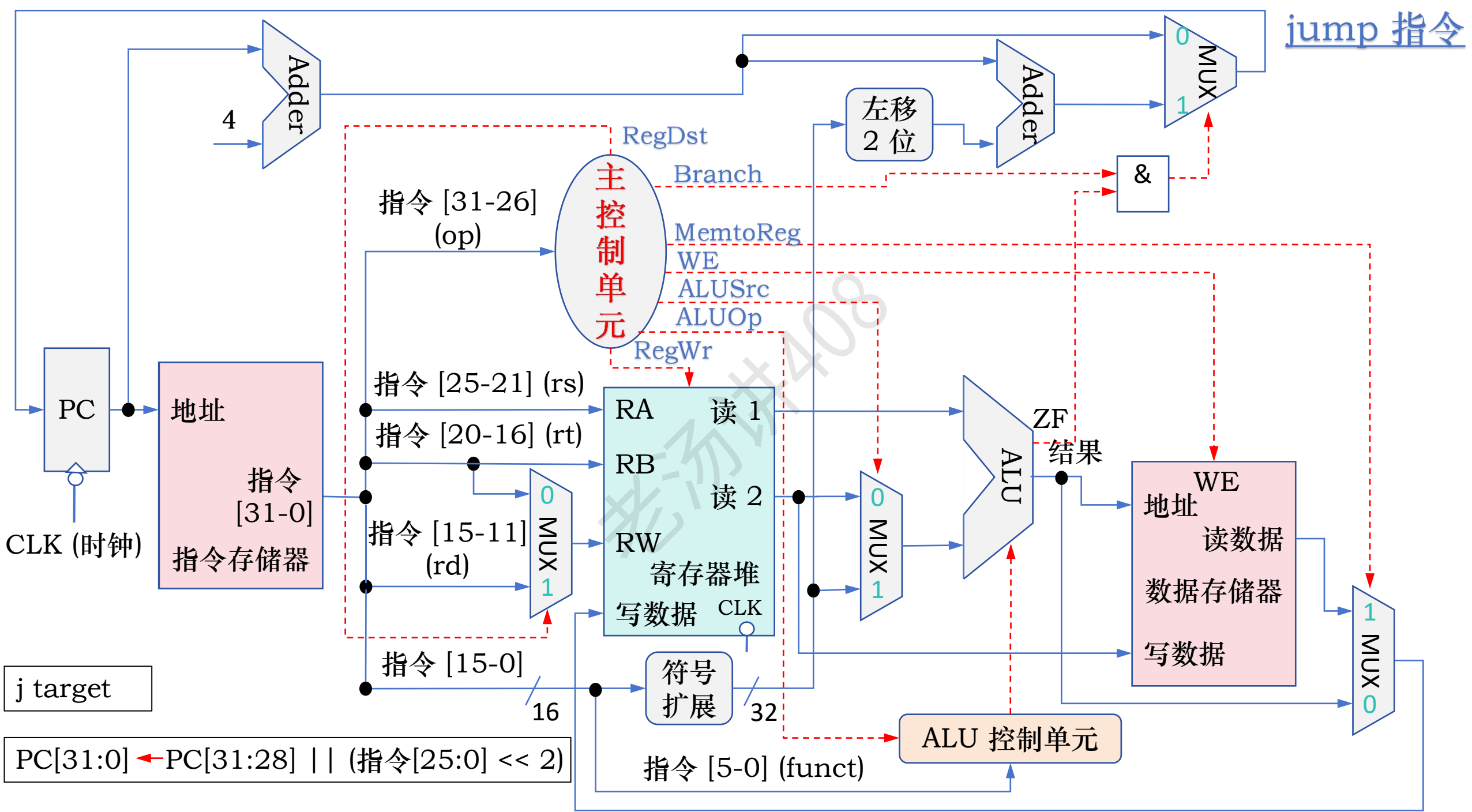
单周期数据通路

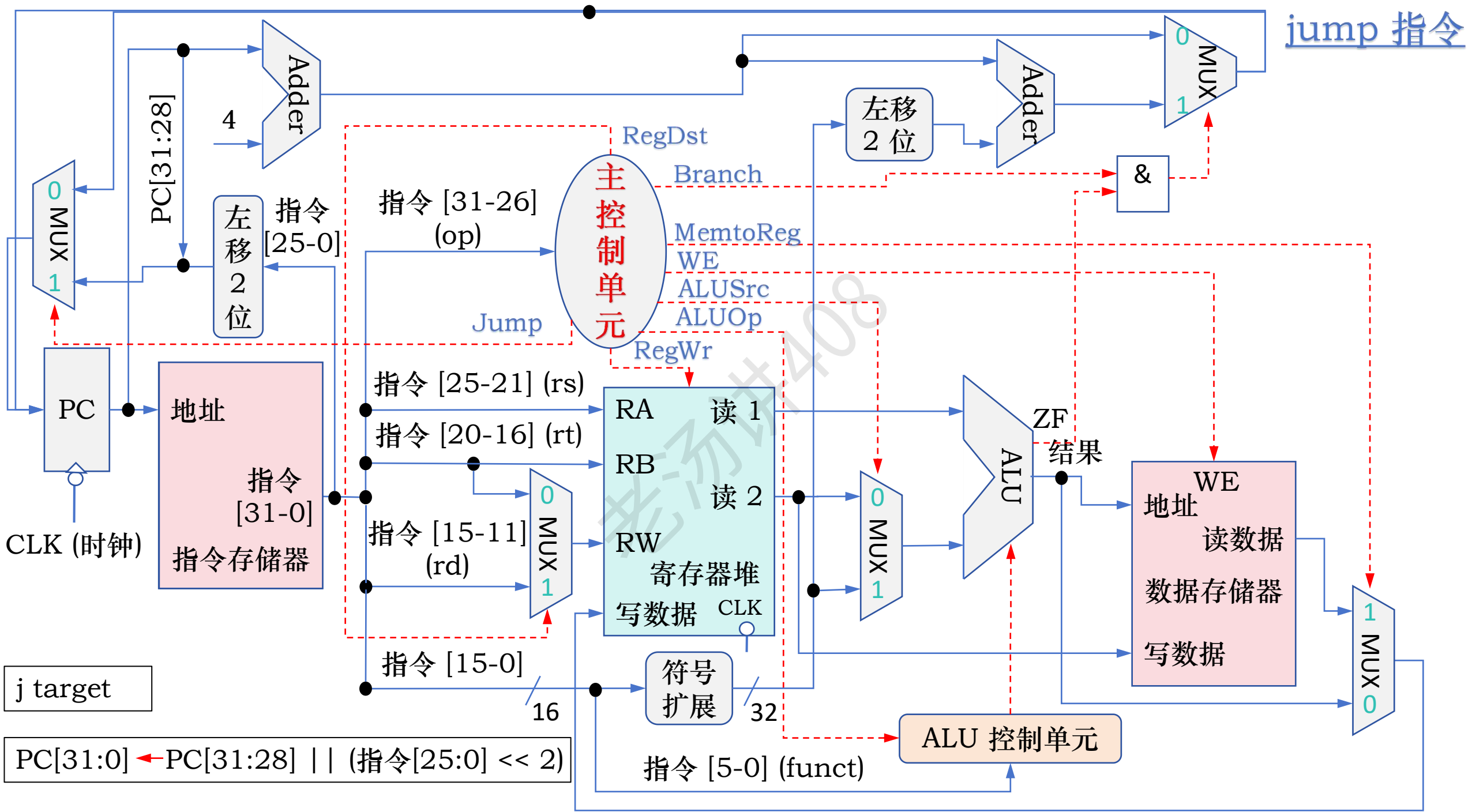




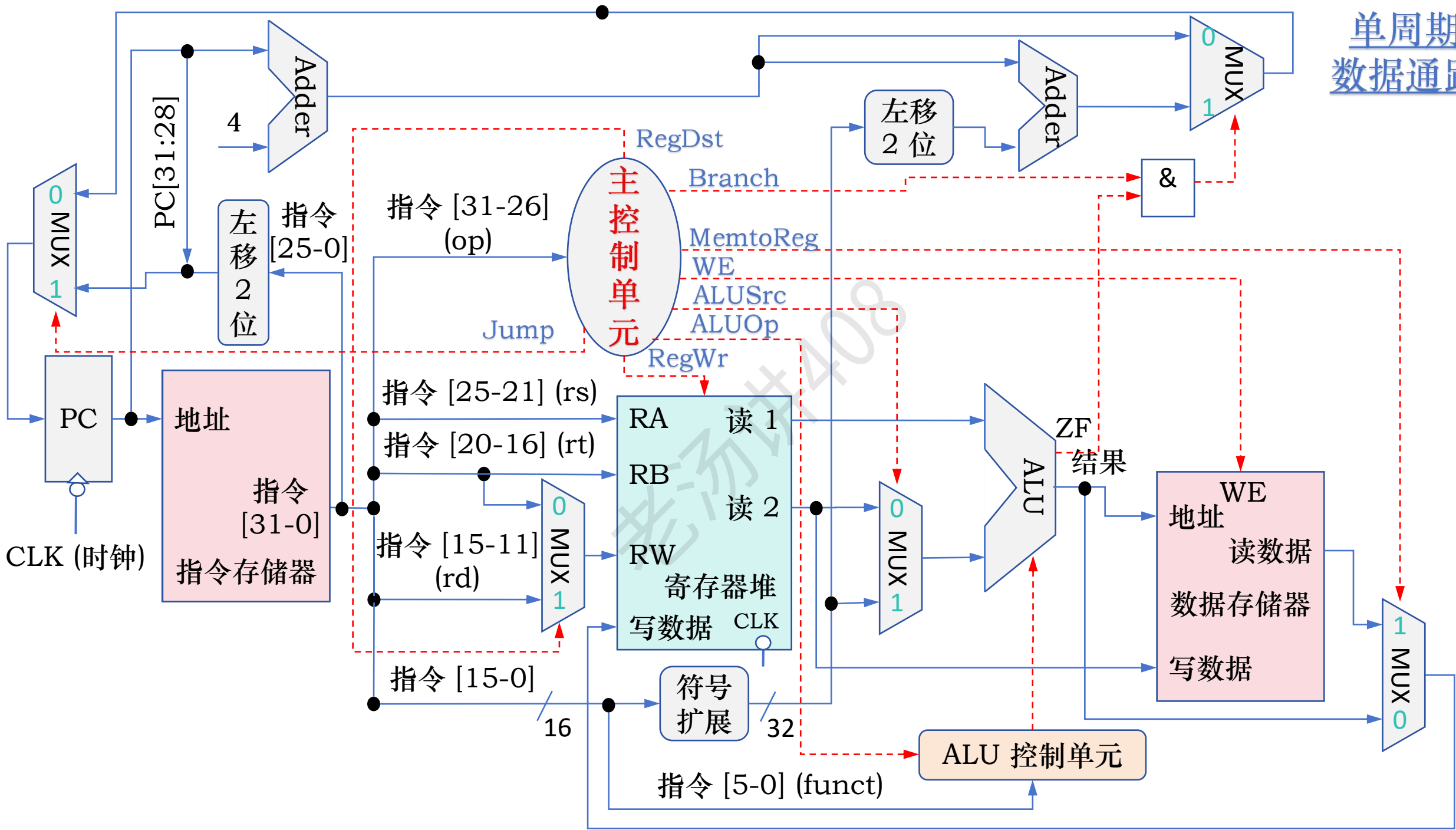








单周期
数据通路



中央处理器 (CPU) 设计的五个步骤

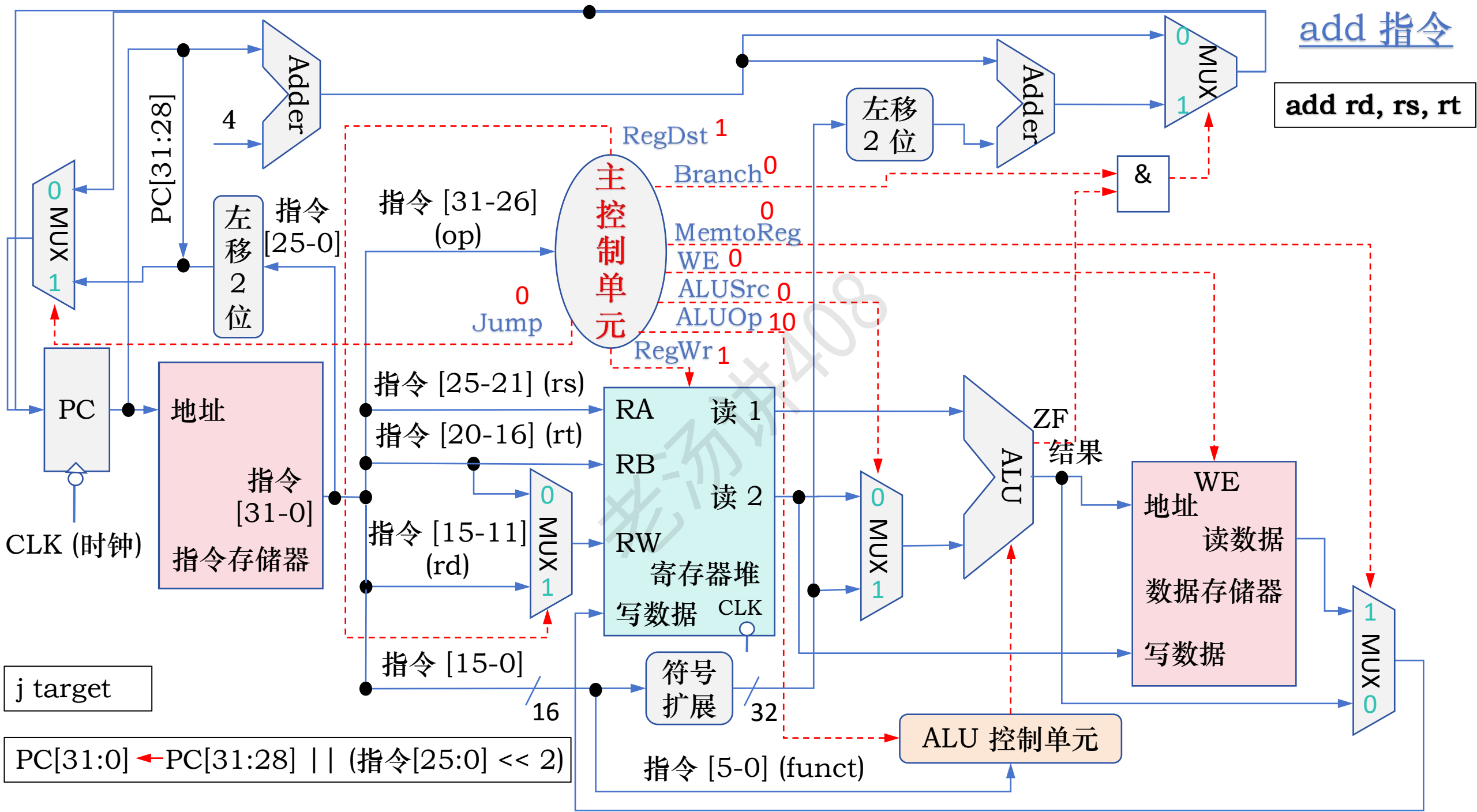
第一步：分析每条指令的功能

第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路



汇总所有指令的控制信号

add rd, rs, rt

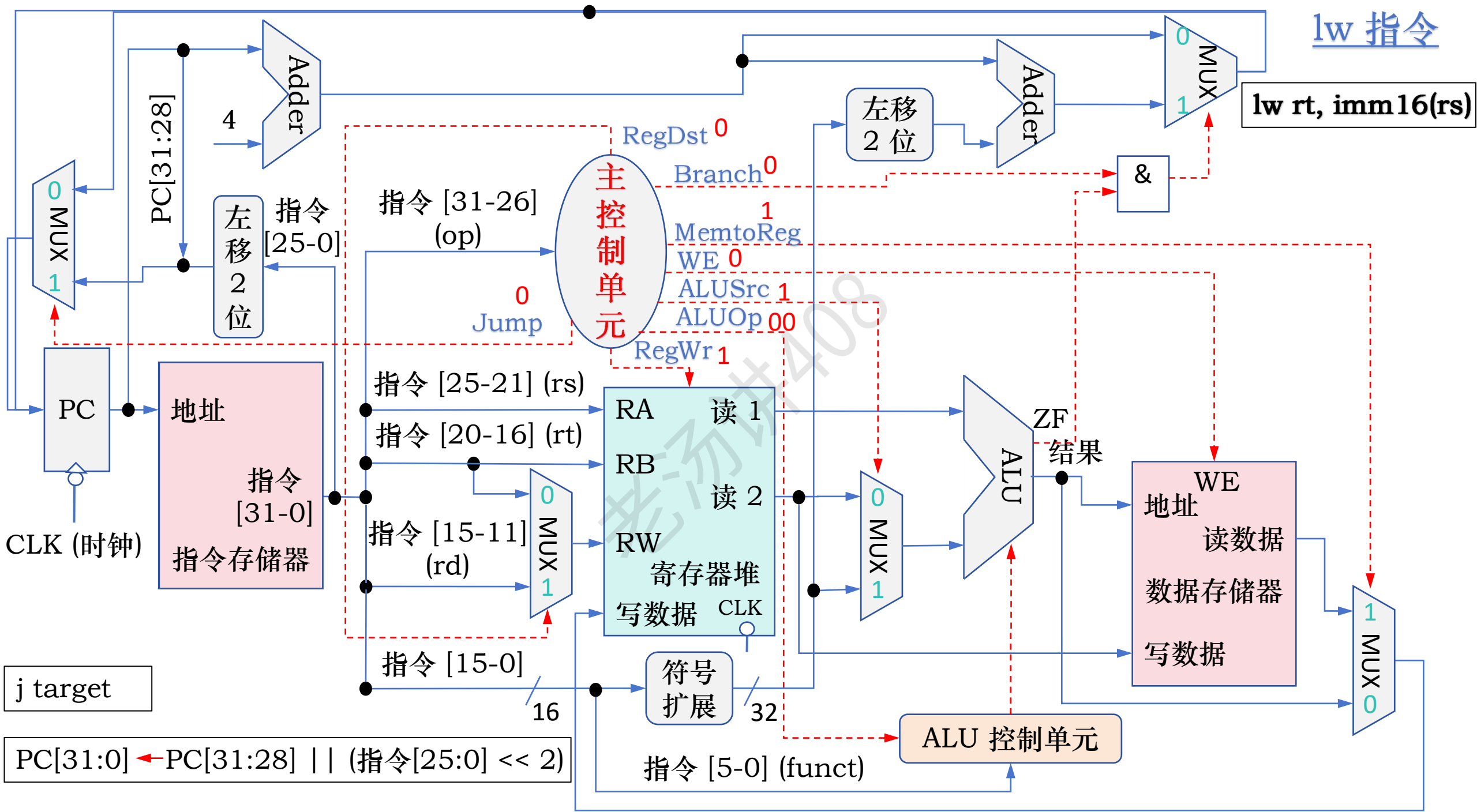
sub rd, rs, rt

or rd, rs, rt

slt rd, rs, rt

控制信号	add 指令
RegDst	1
Branch	0
MemtoReg	0
WE	0
ALUSrc	0
ALUOp1	1
ALUOp0	0
RegWr	1
Jump	0

控制信号	R 型指令
RegDst	1
Branch	0
MemtoReg	0
WE	0
ALUSrc	0
ALUOp1	1
ALUOp0	0
RegWr	1
Jump	0



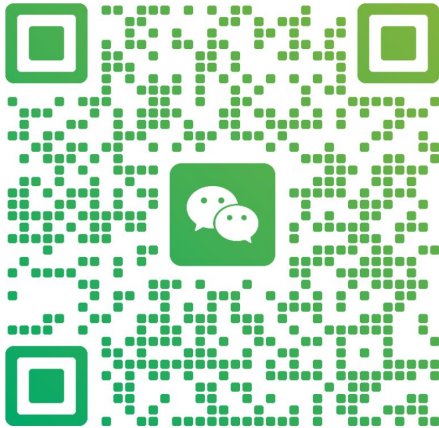
汇总所有指令的控制信号

控制信号	R 型指令	lw 指令
RegDst	1	1
Branch	0	0
MemtoReg	0	1
WE	0	0
ALUSrc	0	1
ALUOp1	1	0
ALUOp0	0	0
RegWr	1	1
Jump	0	0

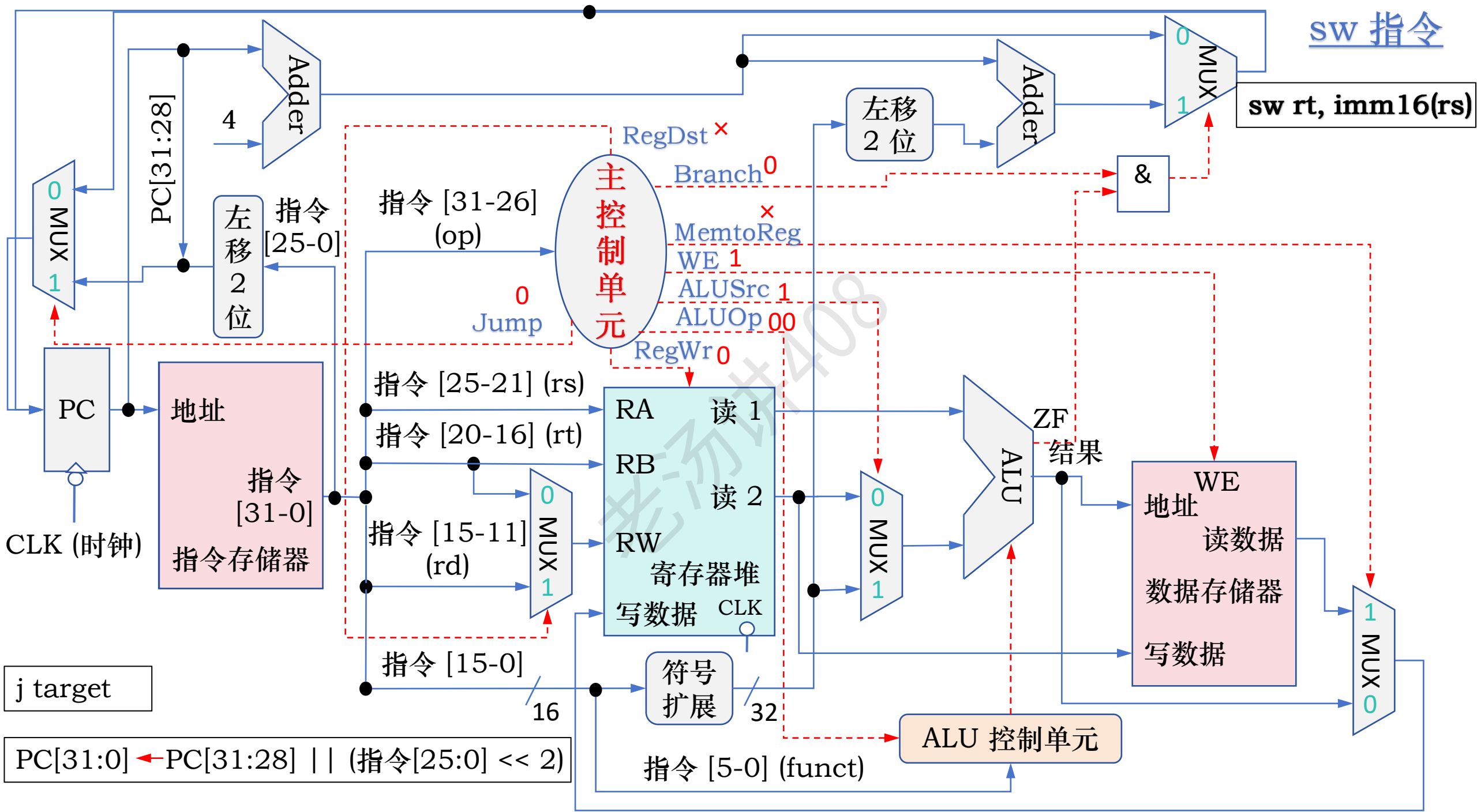
加老汤微信，帮你 408 冲 120+
咨询 408 高分特训营



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



汇总所有指令的控制信号

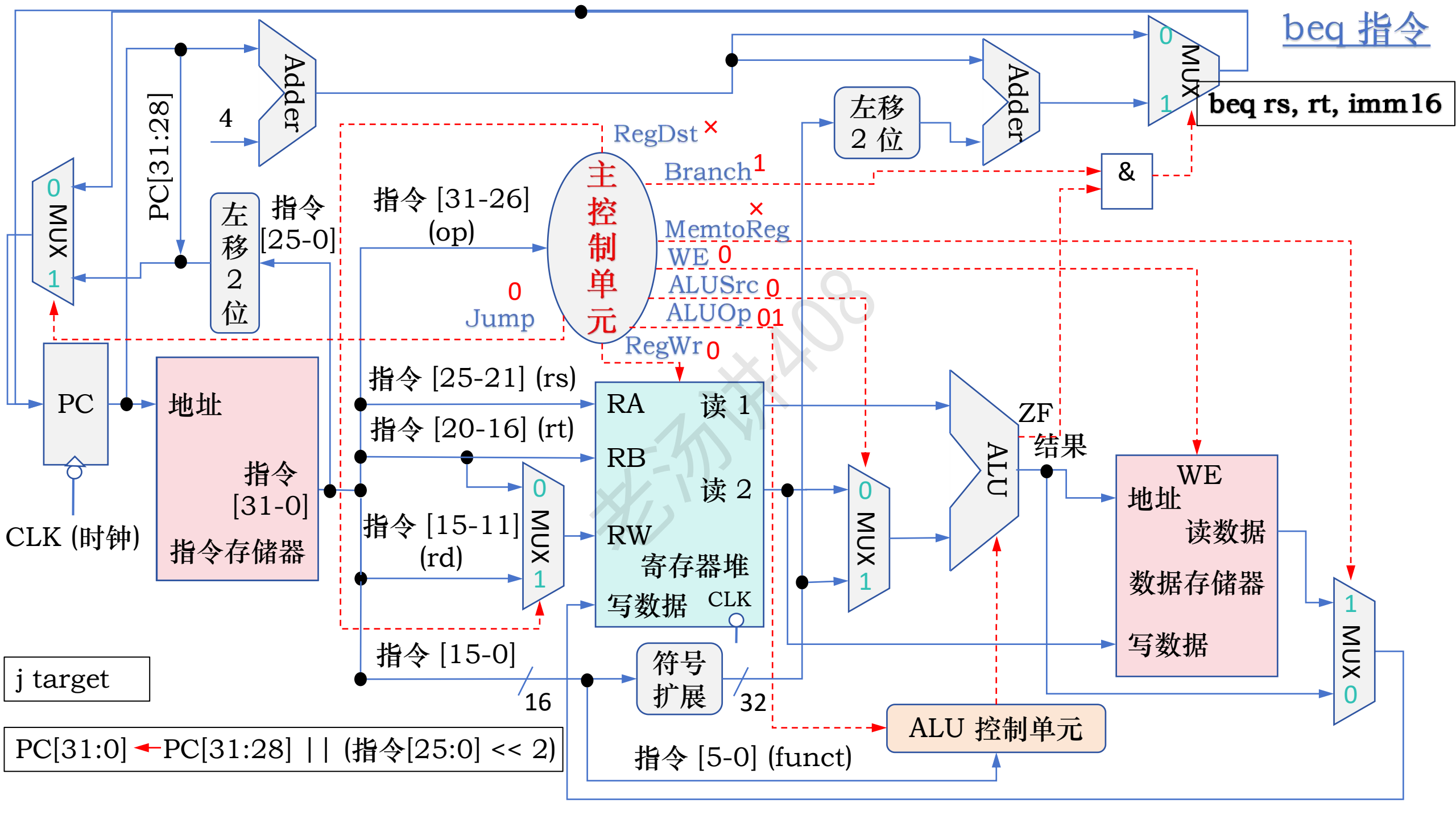
控制信号	R 型指令	lw 指令	sw 指令
RegDst	1	1	×
Branch	0	0	0
MemtoReg	0	1	×
WE	0	0	1
ALUSrc	0	1	1
ALUOp1	1	0	0
ALUOp0	0	0	0
RegWr	1	1	0
Jump	0	0	0

加老汤微信，帮你 408 冲 120+
咨询 408 高分特训营



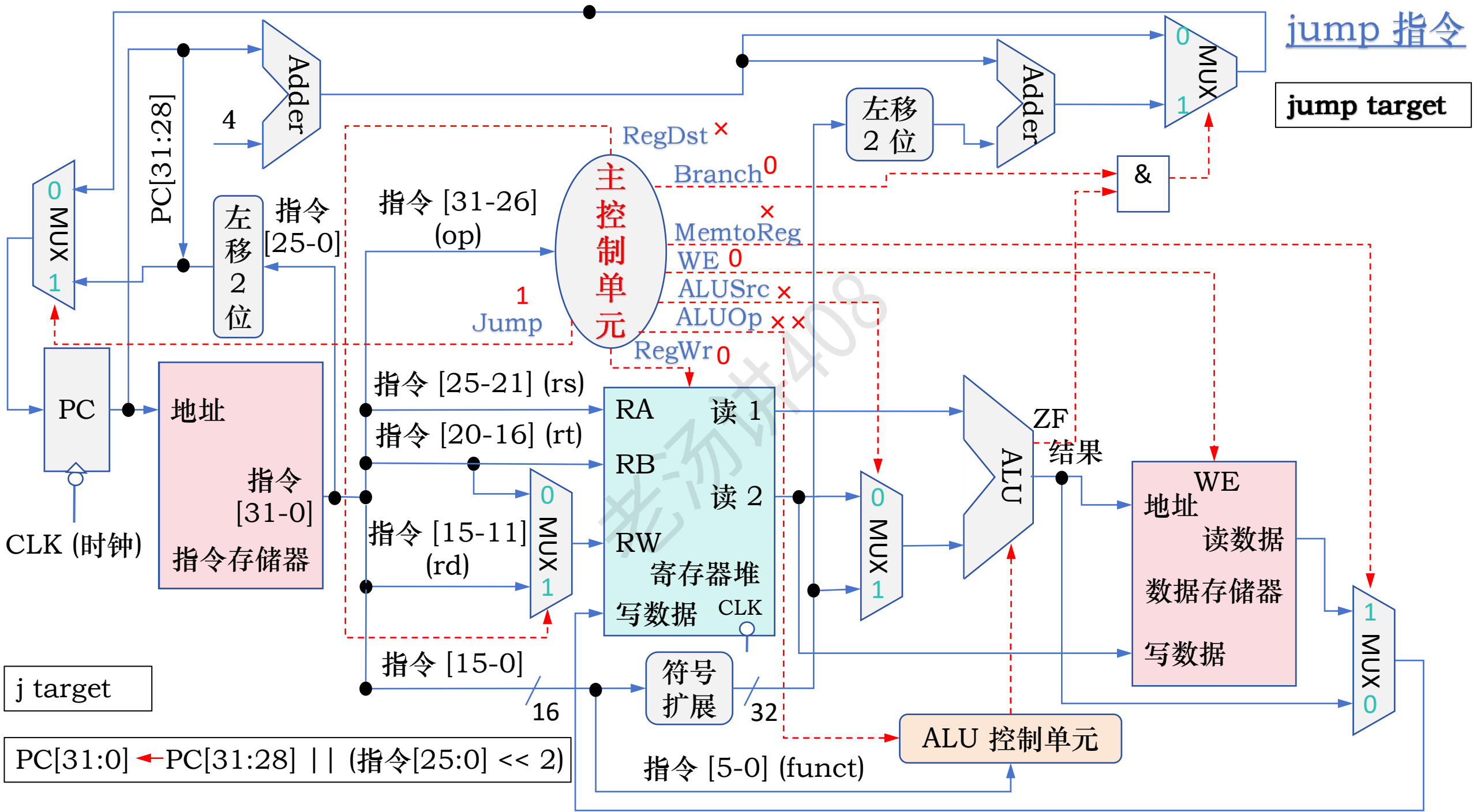
扫一扫上面的二维码图案，加我为朋友。

beq 指令



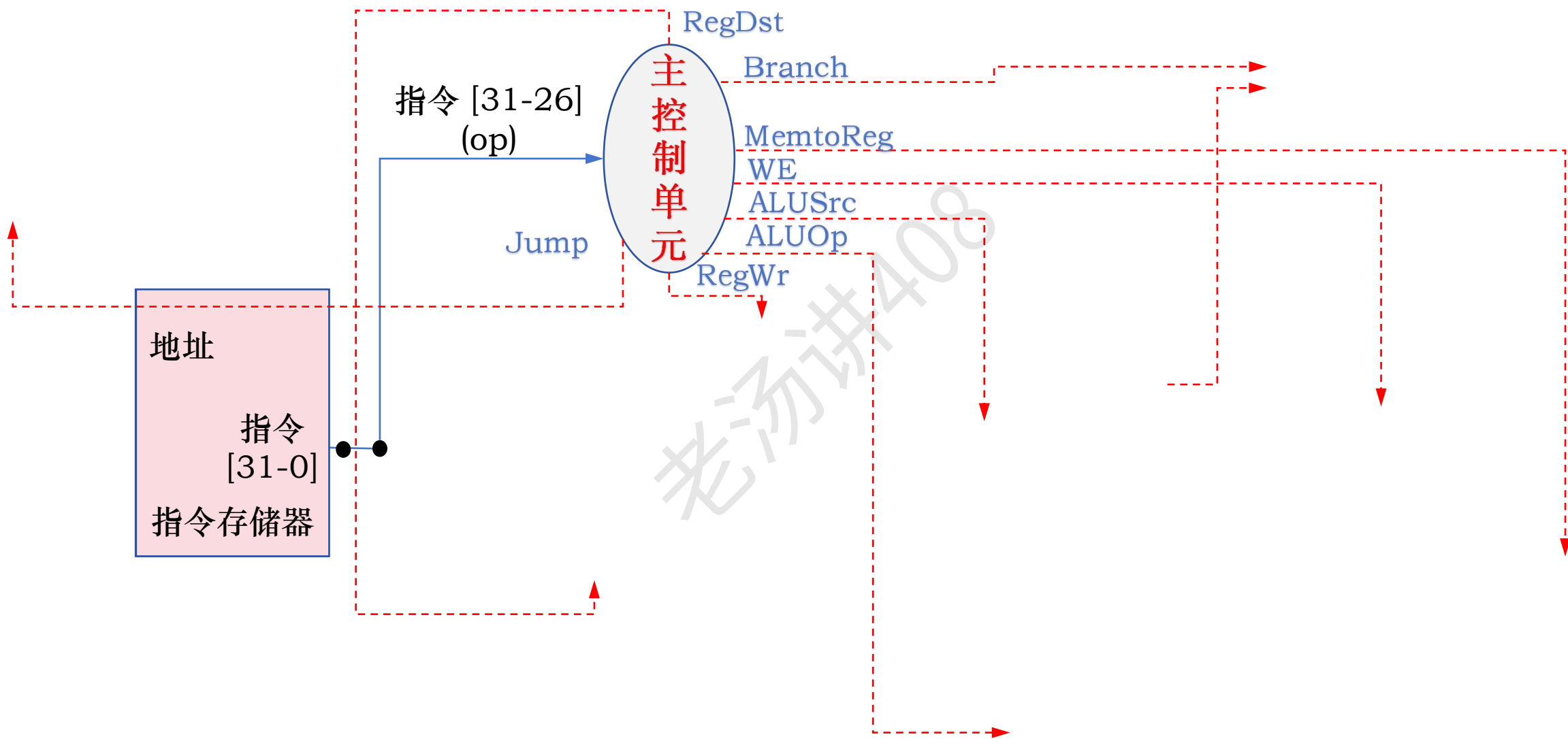
汇总所有指令的控制信号

控制信号	R 型指令	lw 指令	sw 指令	beq 指令
RegDst	1	1	×	×
Branch	0	0	0	0
MemtoReg	0	1	×	×
WE	0	0	1	0
ALUSrc	0	1	1	1
ALUOp1	1	0	0	0
ALUOp0	0	0	0	1
RegWr	1	1	0	0
Jump	0	0	0	0



汇总所有指令的控制信号

控制信号	R 型指令	lw 指令	sw 指令	beq 指令	jump 指令
RegDst	1	1	×	×	×
Branch	0	0	0	0	0
MemtoReg	0	1	×	×	×
WE	0	0	1	0	0
ALUSrc	0	1	1	1	×
ALUOp1	1	0	0	0	×
ALUOp0	0	0	0	1	×
RegWr	1	1	0	0	0
Jump	0	0	0	0	1



生成指令和控制信号之间的关系表

指令	输入（6 位操作码）						输出（控制信号）								
	OP5	OP4	OP3	OP2	OP1	OP0	RegDst	Branch	MemtoReg	WE	ALUSrc	ALUOp1	ALUOp0	RegWr	Jump
R 型	0	0	0	0	0	0	1	0	0	0	0	1	0	1	0
lw	1	0	0	0	1	1	1	0	1	0	1	0	0	1	0
sw	1	0	1	0	1	1	×	0	×	1	1	0	0	0	0
beq	0	0	0	1	0	0	×	0	×	0	1	0	1	0	0
jump	0	0	0	0	1	0	×	0	×	0	×	×	×	0	1



主控制单元电路设计

中央处理器 (CPU) 设计的五个步骤

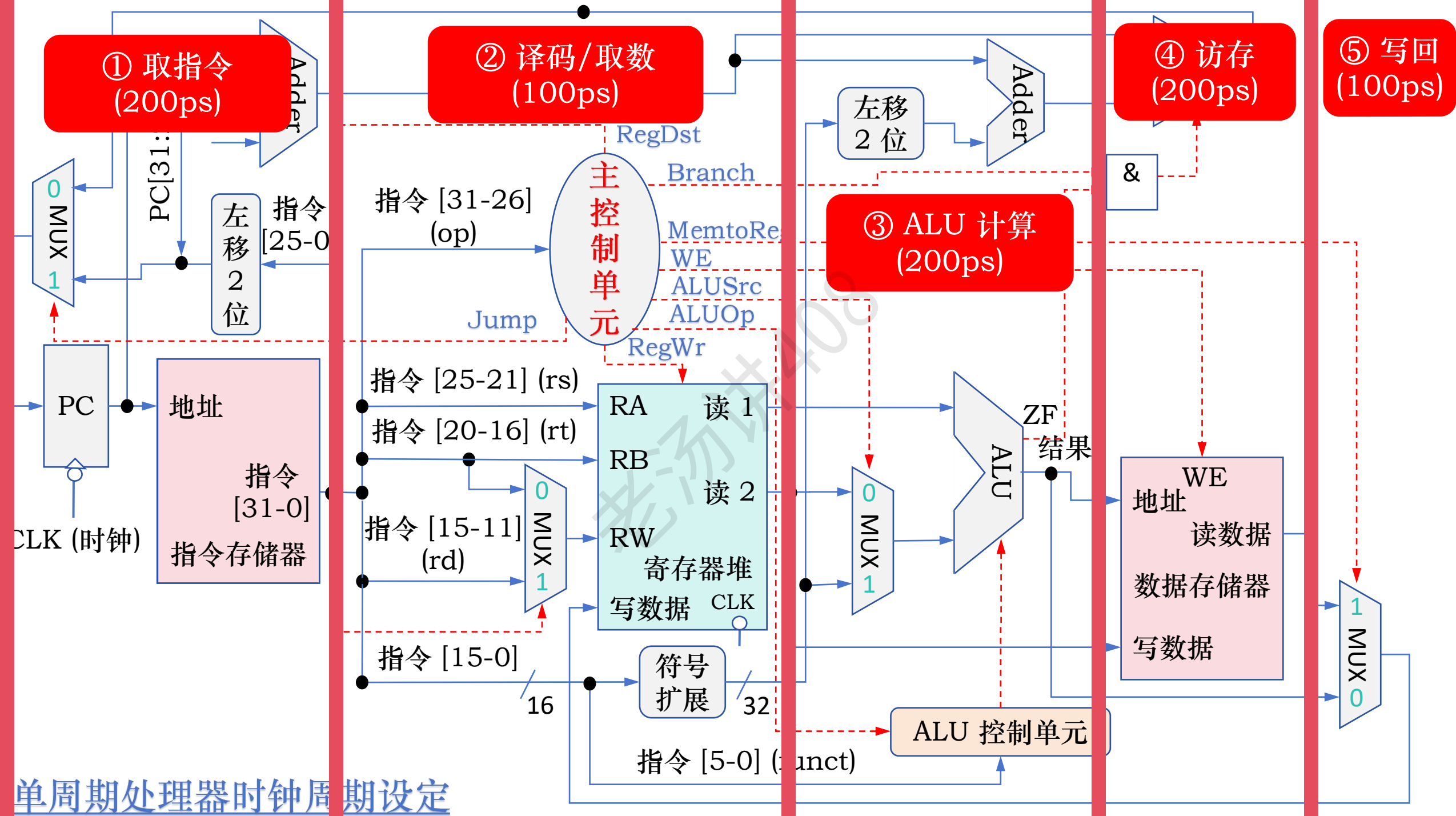
第一步：分析每条指令的功能

第二步：根据指令的功能给出所需的元件，并考虑如何将它们互连

第三步：确定每个元件所需控制信号的取值

第四步：汇总所有指令涉及的控制信号，生成反应指令与控制信号之间的关系表

第五步：根据关系表，得到每个控制信号的逻辑表达式，据此设计控制电路



单周期处理器时钟周期设定

单周期处理器时钟周期设定

$1ps = 10^{-3}ns = 10^{-6}\mu s = 10^{-9}ms = 10^{-12}s$

$1\text{皮秒} = 10^{-3}\text{纳秒} = 10^{-6}\text{微秒} = 10^{-9}\text{毫秒} = 10^{-12}\text{秒}$

指令类型	取指令	译码/取数	ALU 计算	访存	写回	总时间
R 型指令	200ps	100ps	200ps		100ps	600ps
lw 指令	200ps	100ps	200ps	200ps	100ps	800ps
sw 指令	200ps	100ps	200ps	200ps		700ps
beq 指令	200ps	100ps	200ps			500ps
jump 指令	200ps					200ps

时钟周期：800ps

多周期处理器时钟周期设定

$1ps = 10^{-3}ns = 10^{-6}\mu s = 10^{-9}ms = 10^{-12}s$
 $1\text{皮秒} = 10^{-3}\text{纳秒} = 10^{-6}\text{微秒} = 10^{-9}\text{毫秒} = 10^{-12}\text{秒}$

指令类型	取指令	译码/取数	ALU 计算	访存	写回	总时间
R 型指令	200ps	100ps	200ps		100ps	600ps
lw 指令	200ps	100ps	200ps	200ps	100ps	800ps
sw 指令	200ps	100ps	200ps	200ps		700ps
beq 指令	200ps	100ps	200ps			500ps
jump 指令	200ps					200ps

时钟周期: 200ps

多周期处理器时钟周期设定

指令类型	取指令	译码/取数	ALU 计算	访存	写回	总时间
R 型指令	200ps	100ps	200ps		100ps	600ps
lw 指令	200ps	100ps	200ps	200ps	100ps	800ps
sw 指令	200ps	100ps	200ps	200ps		700ps
beq 指令	200ps	100ps	200ps			500ps
jump 指令	200ps					200ps

单周期处理器时钟周期：800ps $7 \times 800ps = 5600ps$

多周期处理器时钟周期：200ps

beq \$4,\$5,200

sub \$2,\$6,\$7

slt \$4,\$2,\$4

add \$2,\$6,\$7

lw \$2,200(\$6)

j 200000

sw \$2,100(\$6)

$3 \times 200ps = 600ps$

$4 \times 200ps = 800ps$

$4 \times 200ps = 800ps$

$4 \times 200ps = 800ps$

$5 \times 200ps = 1000ps$

$1 \times 200ps = 200ps$

$4 \times 200ps = 800ps$

}

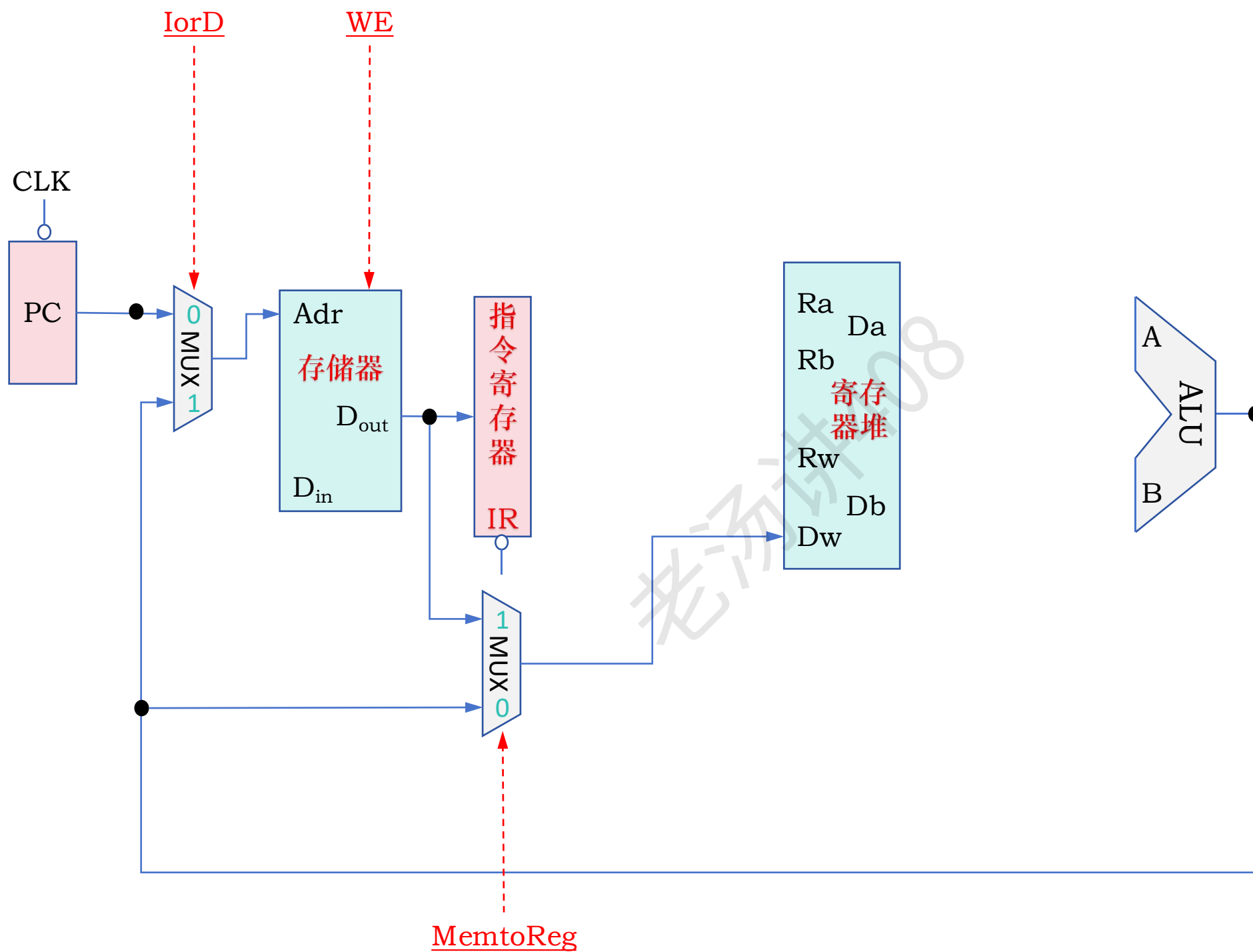
5000ps

多周期处理器的优势

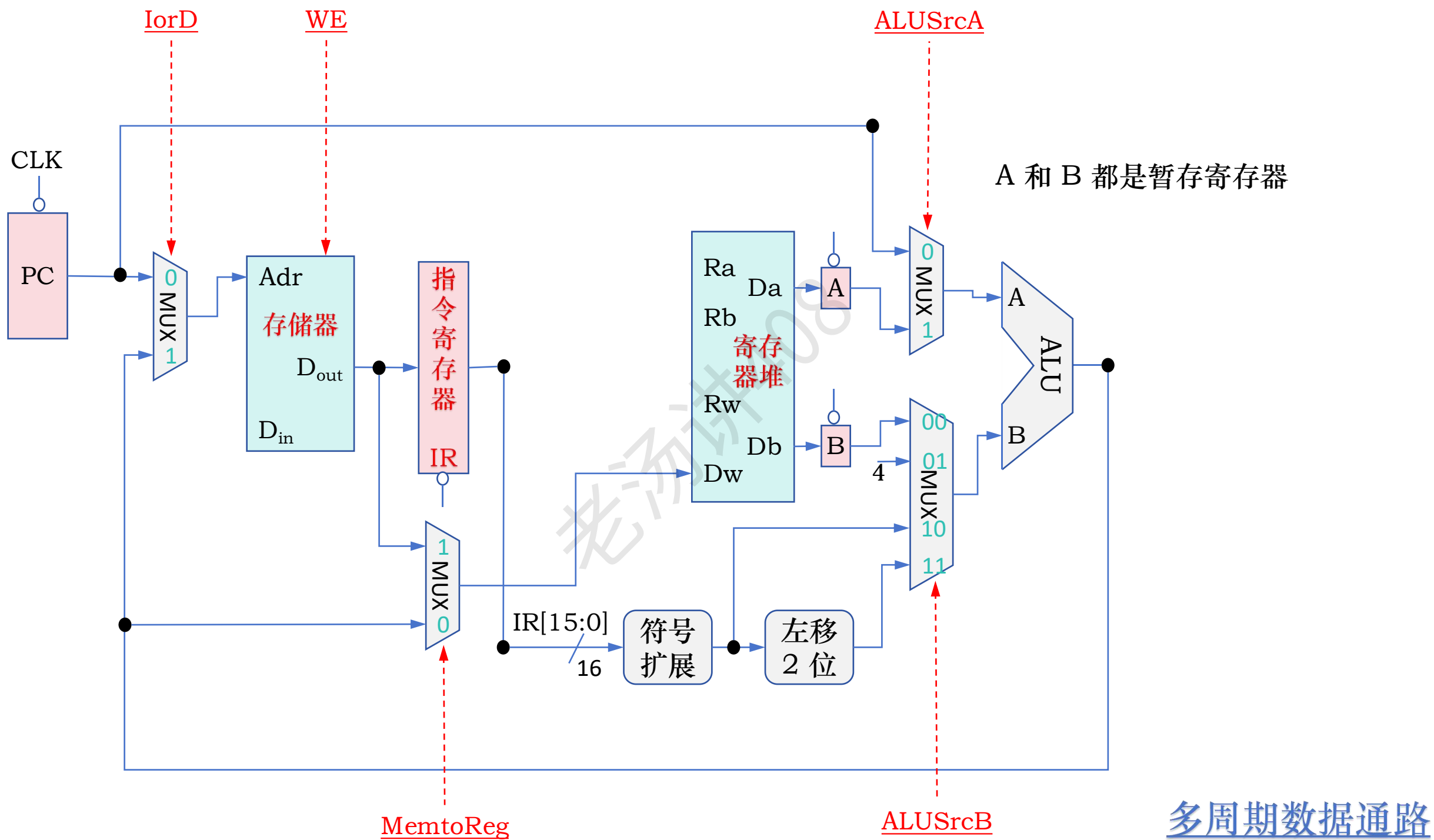
多周期处理器性能比单周期处理器的要好

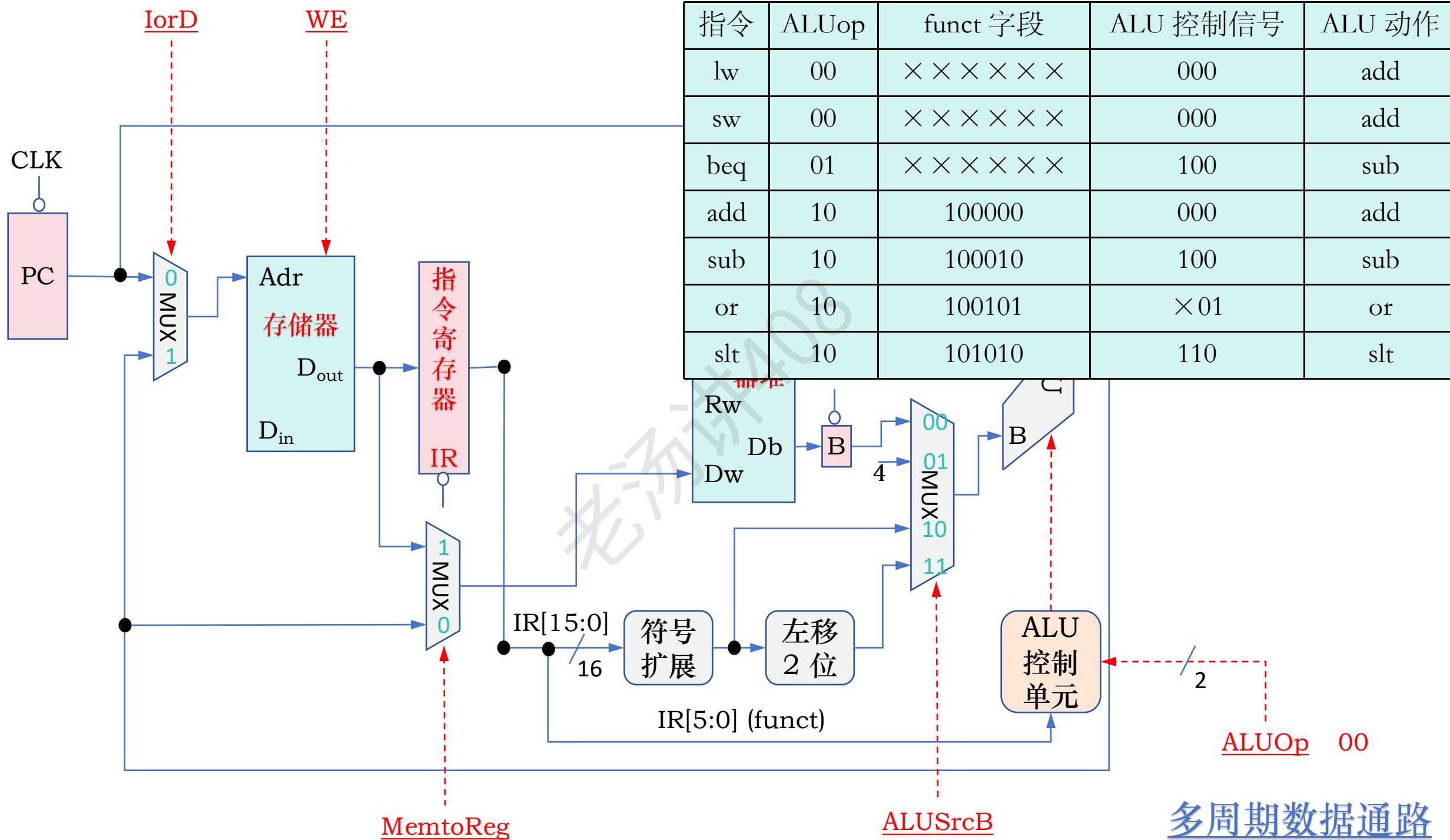
可以降低硬件成本

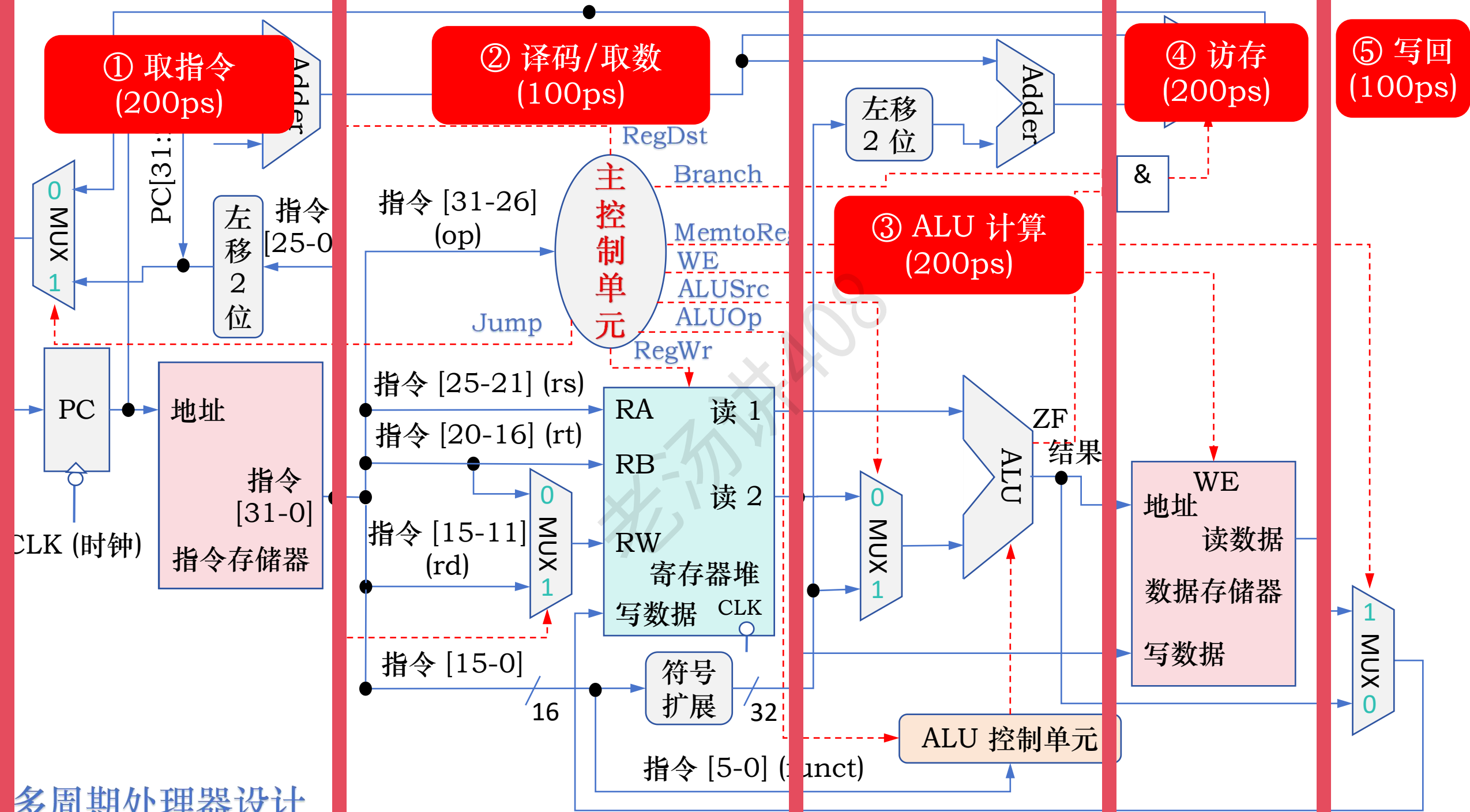
在不同时钟周期可以共享同一个功能部件，使得执行一条指令可以多次使用同一个功能部件



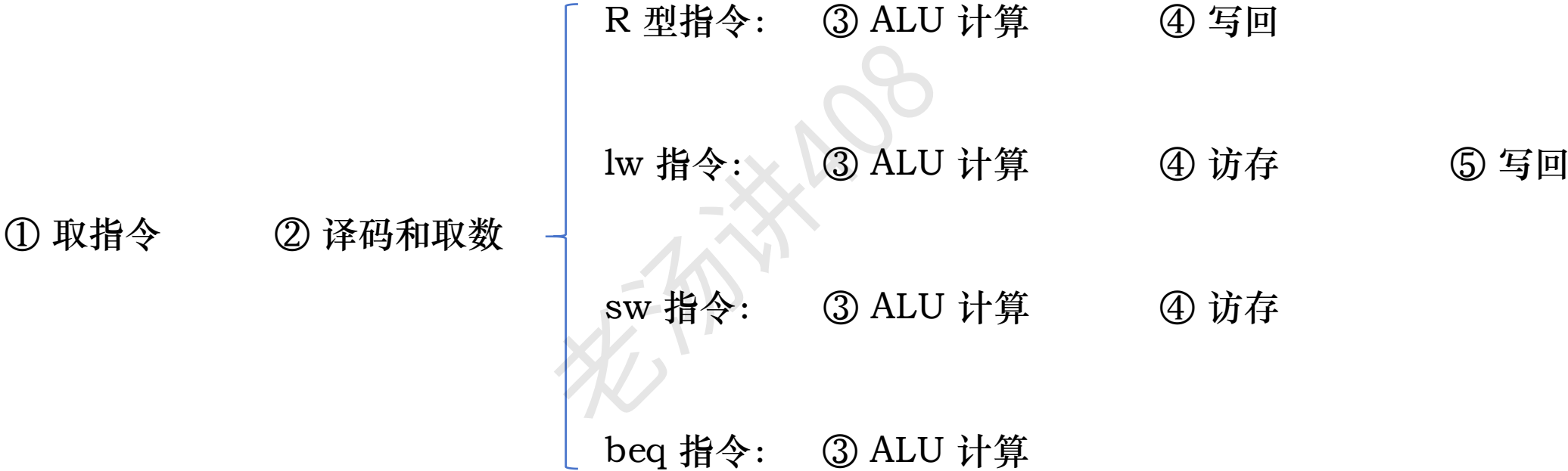
多周期数据通路

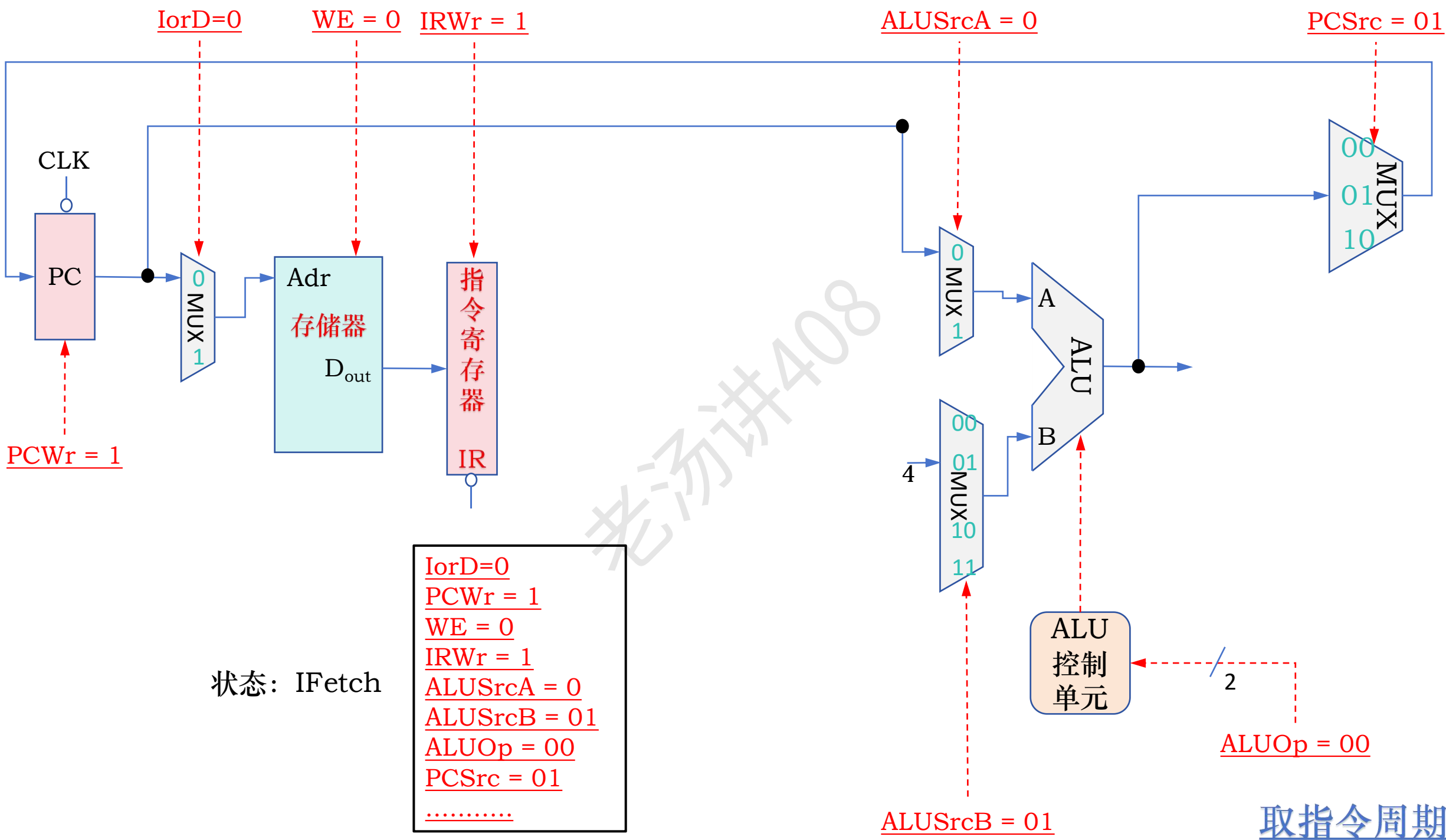


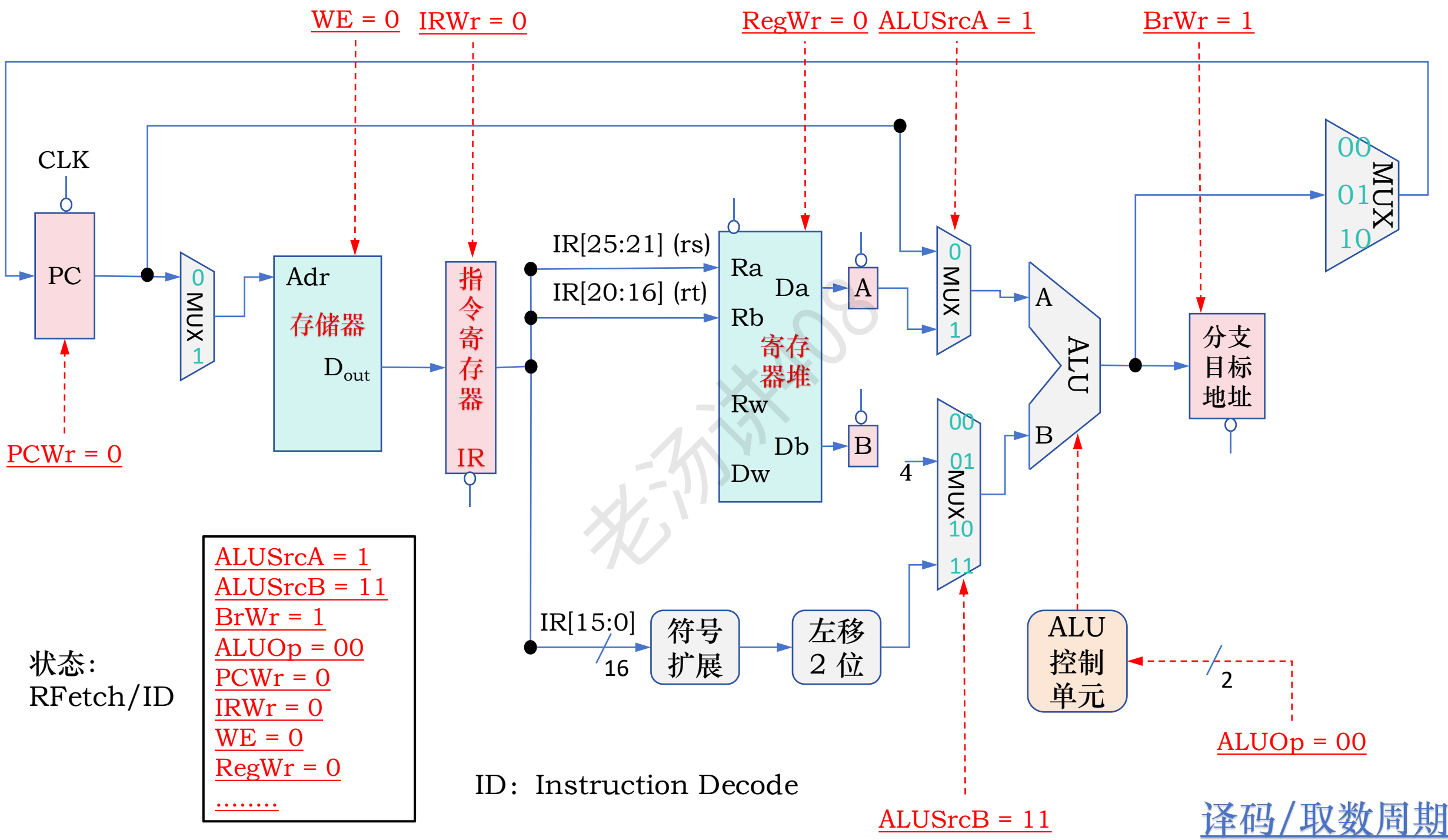


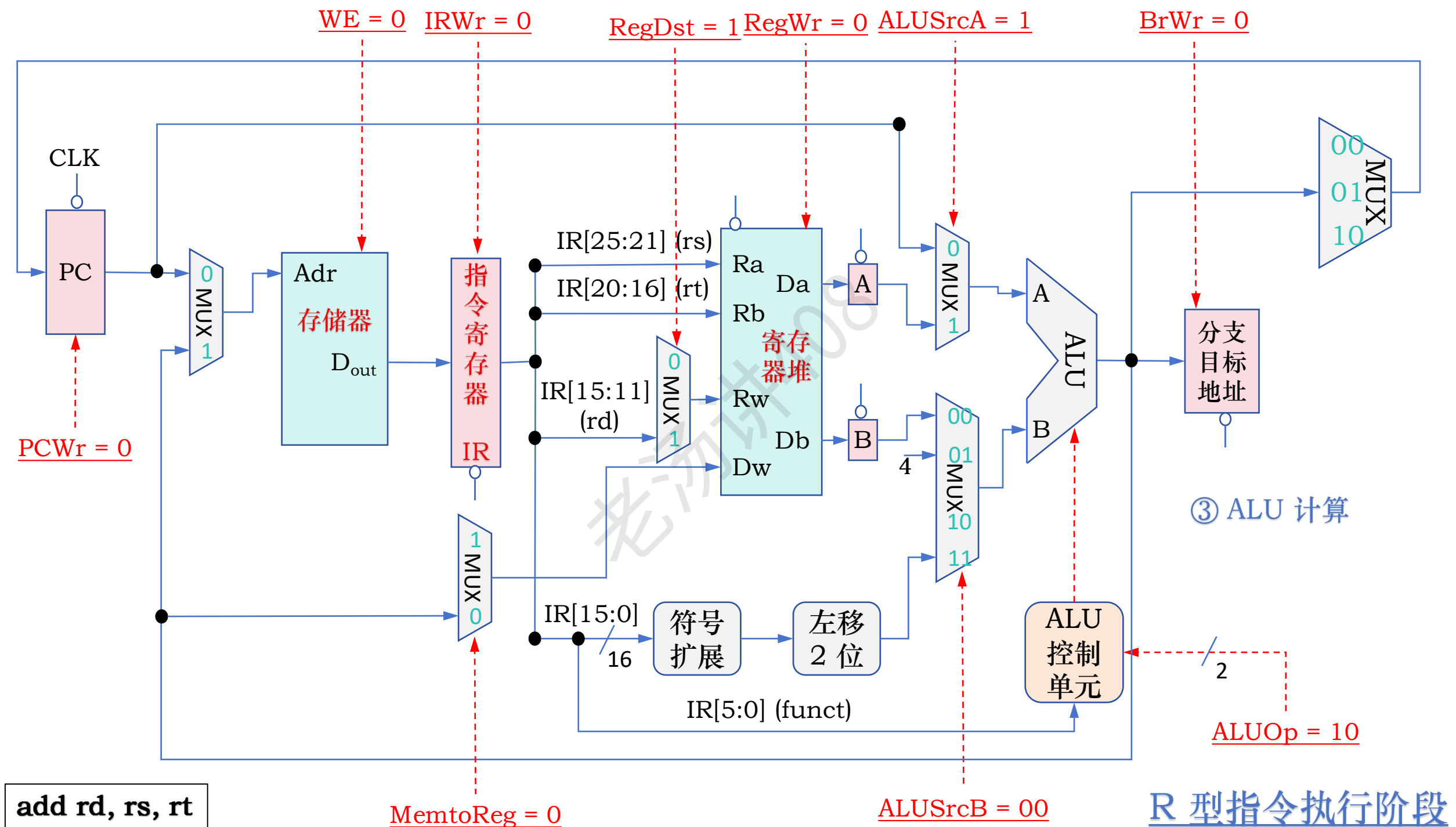


指令在多周期数据通路中的操作









状态: RExec

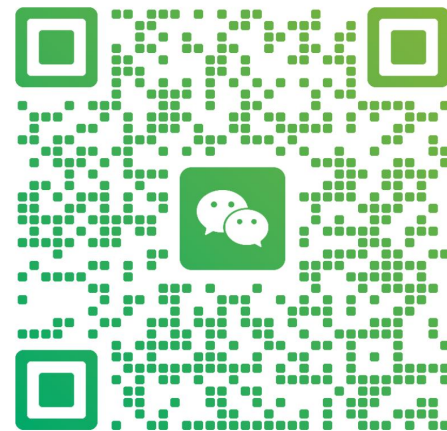
RegDst = 1
ALUSrcA = 1
ALUSrcB = 00
MemtoReg = 0
ALUOp = 10
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



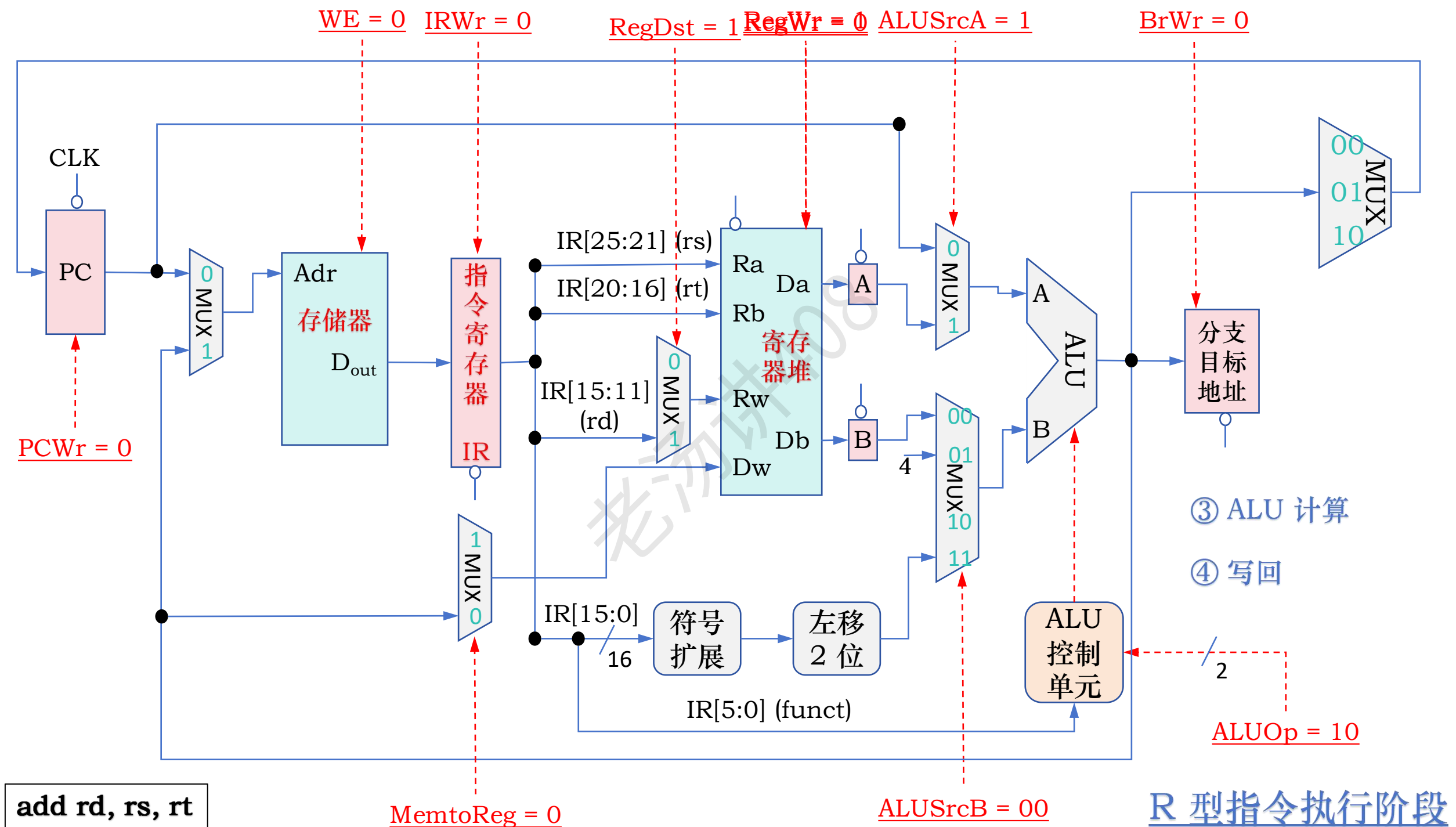
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

R 型指令执行阶段



状态: RExec

RegDst = 1
ALUSrcA = 1
ALUSrcB = 00
MemtoReg = 0
ALUOp = 10
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

状态: RFinish

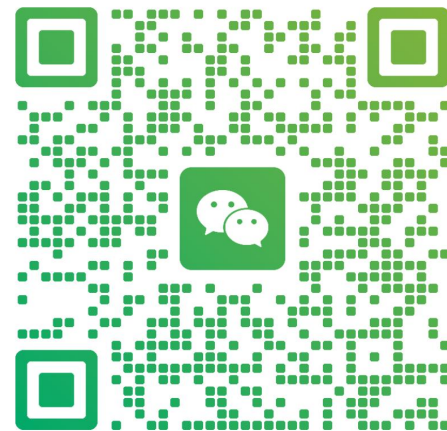
RegDst = 1
ALUSrcA = 1
ALUSrcB = 00
MemtoReg = 0
ALUOp = 10
PCWr = 0
IRWr = 0
WE = 0
RegWr = 1
BrWr = 0
.....

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



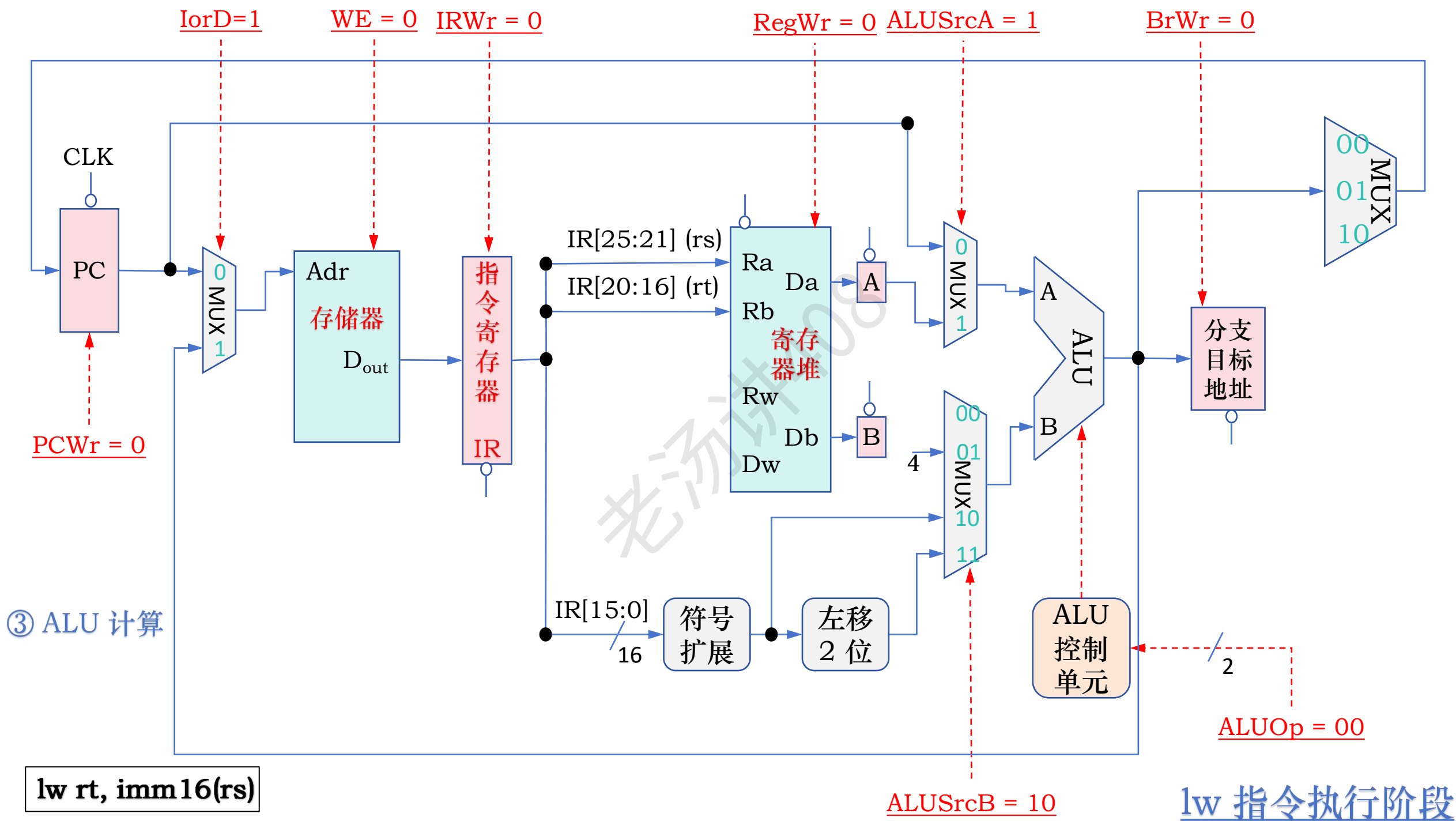
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

R 型指令执行阶段



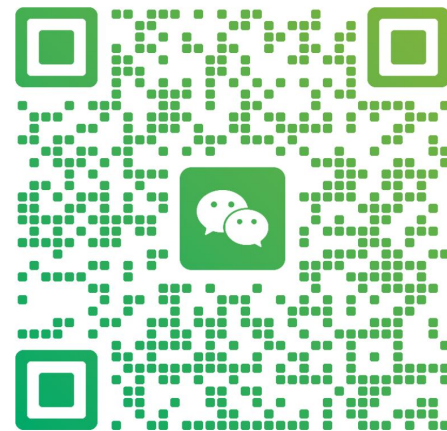
状态: MemAdr

ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

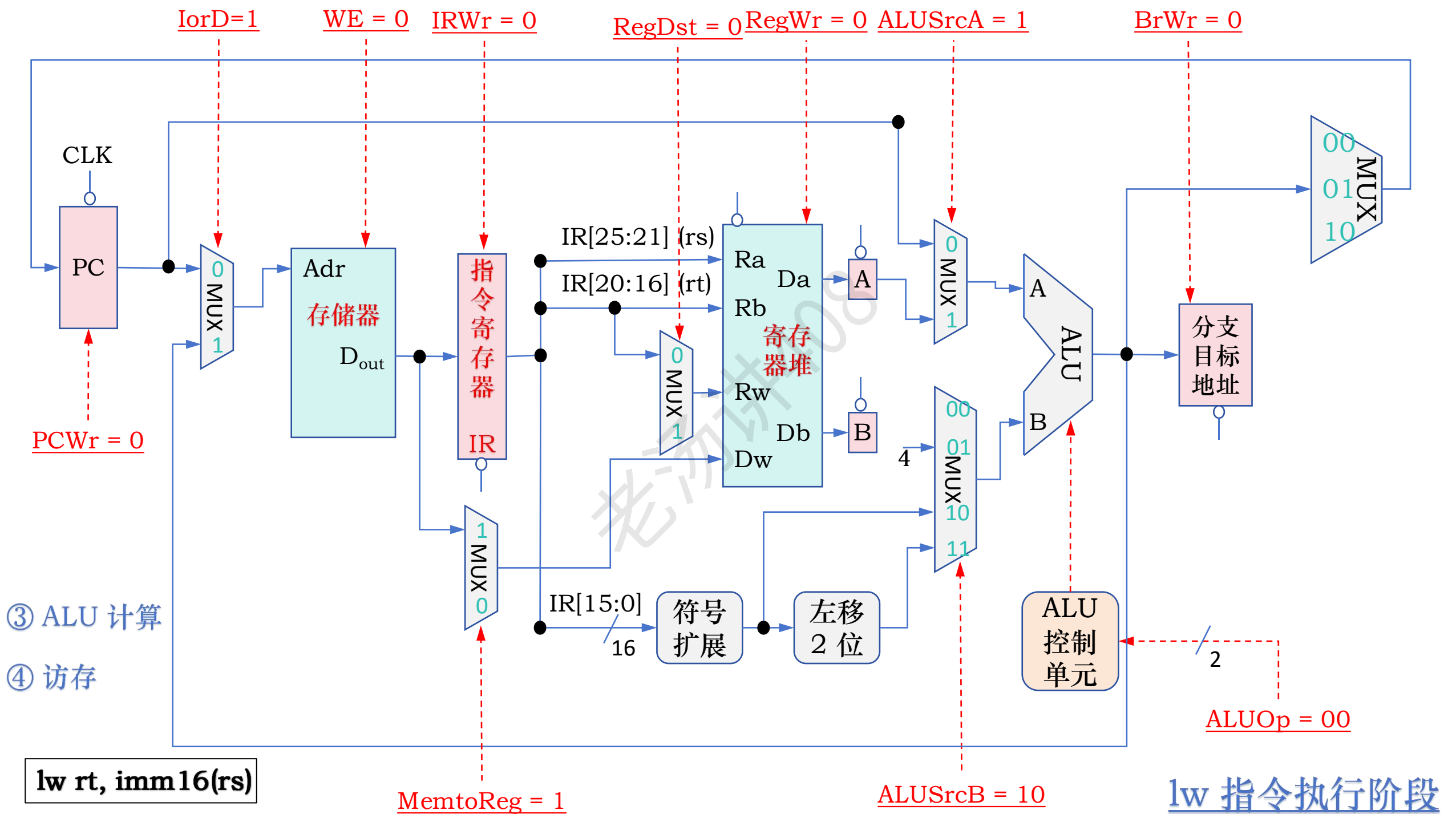
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



状态: MemAdr

ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

状态: MemFetch

RegDst = 0
MemtoReg = 1
ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



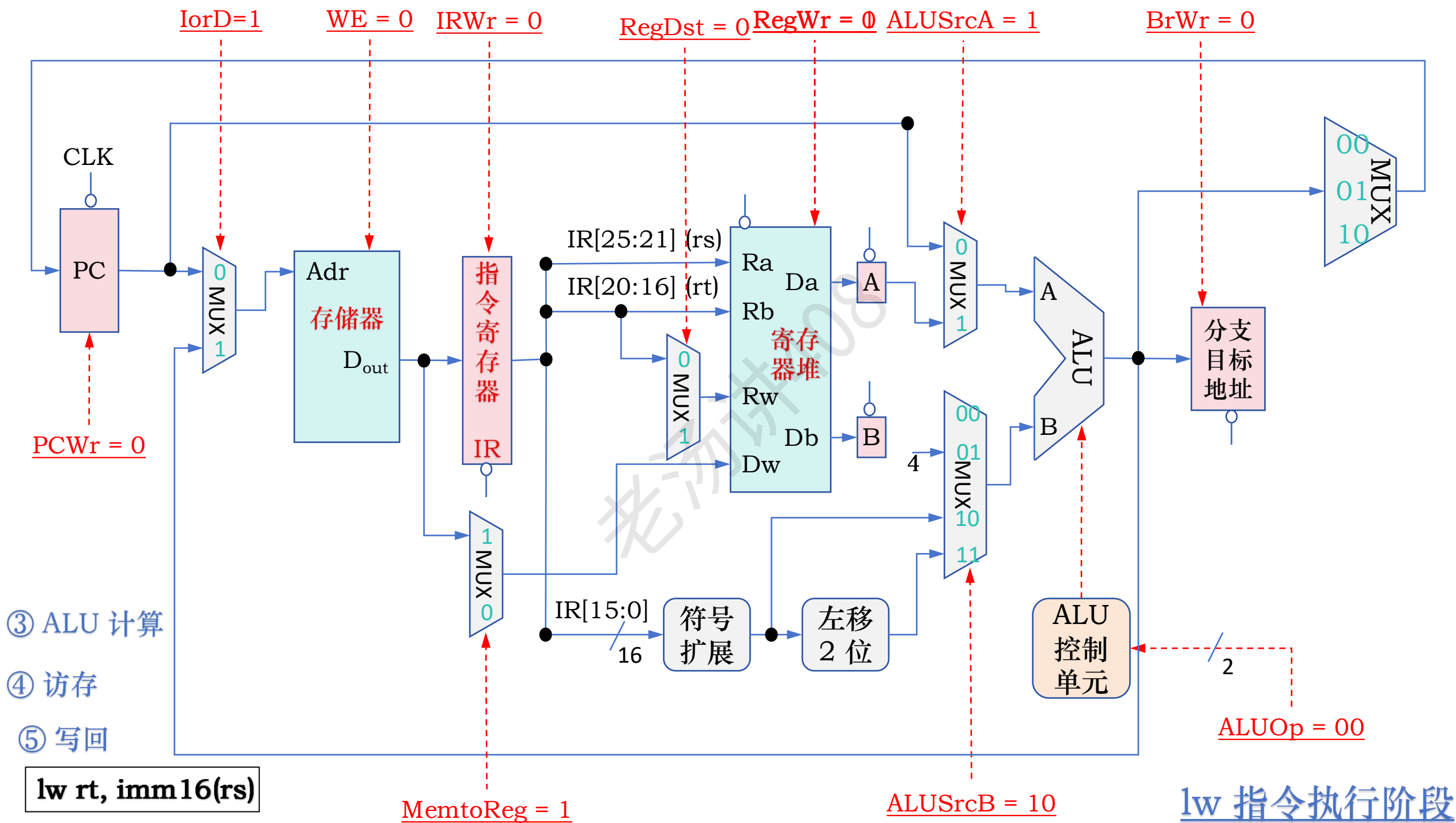
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

lw 指令执行阶段



状态: MemAdr

ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

状态: MemFetch

RegDst = 0
MemtoReg = 1
ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

状态: lwFinish

RegDst = 0
MemtoReg = 1
ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 1
BrWr = 0
.....

lw 指令执行阶段

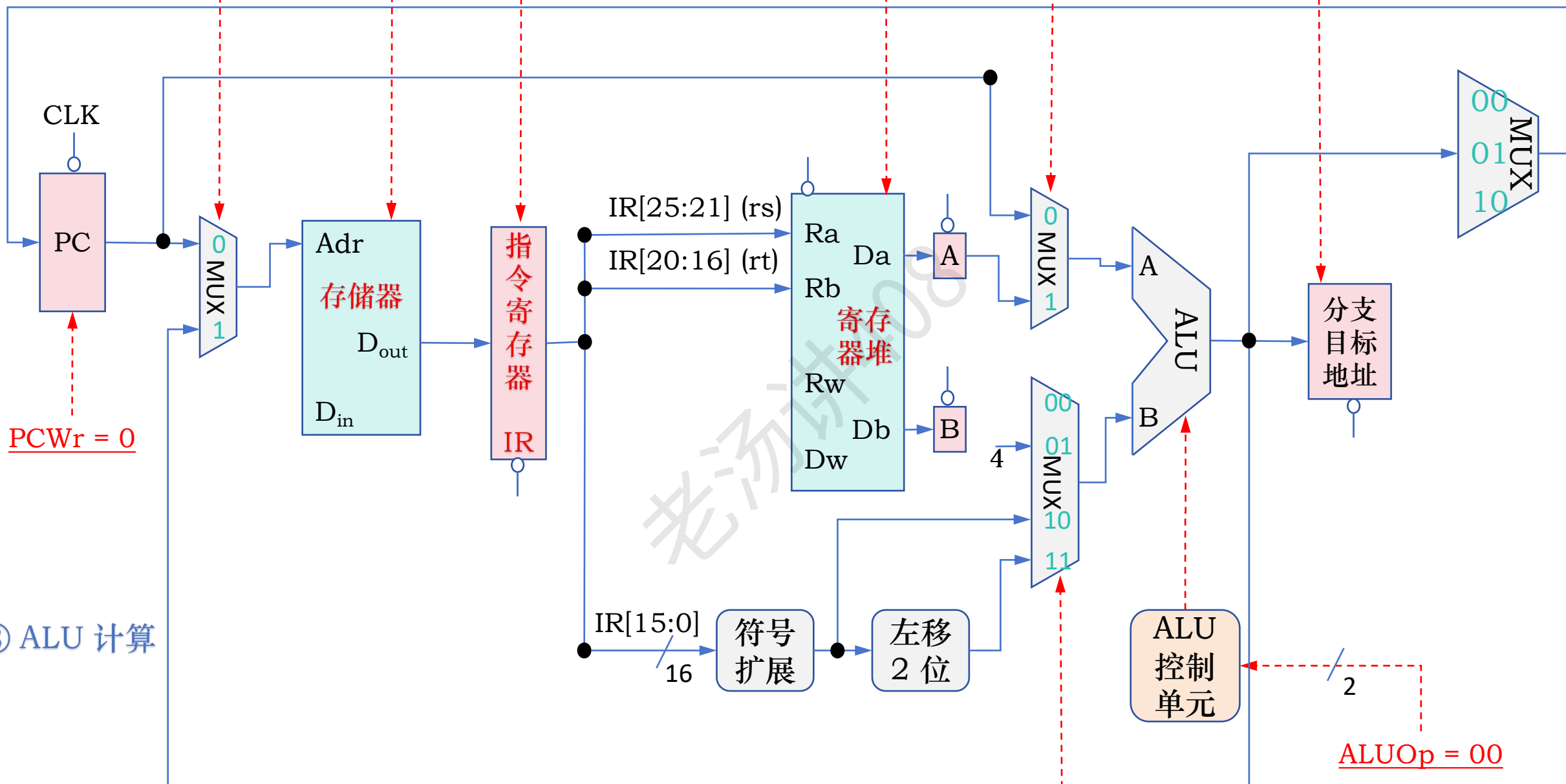
IorD=1

WE = 0

IRWr = 0

RegWr = 0 ALUSrcA = 1

BrWr = 0



`sw rt, imm16(rs)`

ALUSrcB = 10

sw 指令执行阶段

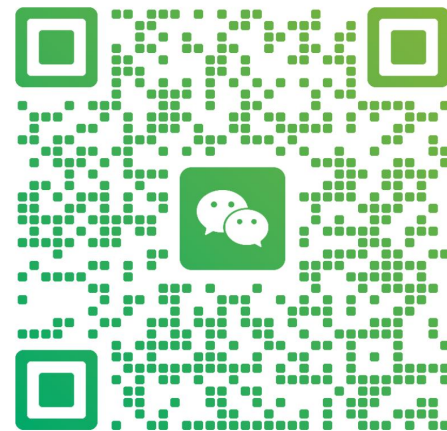
状态: MemAdr

ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

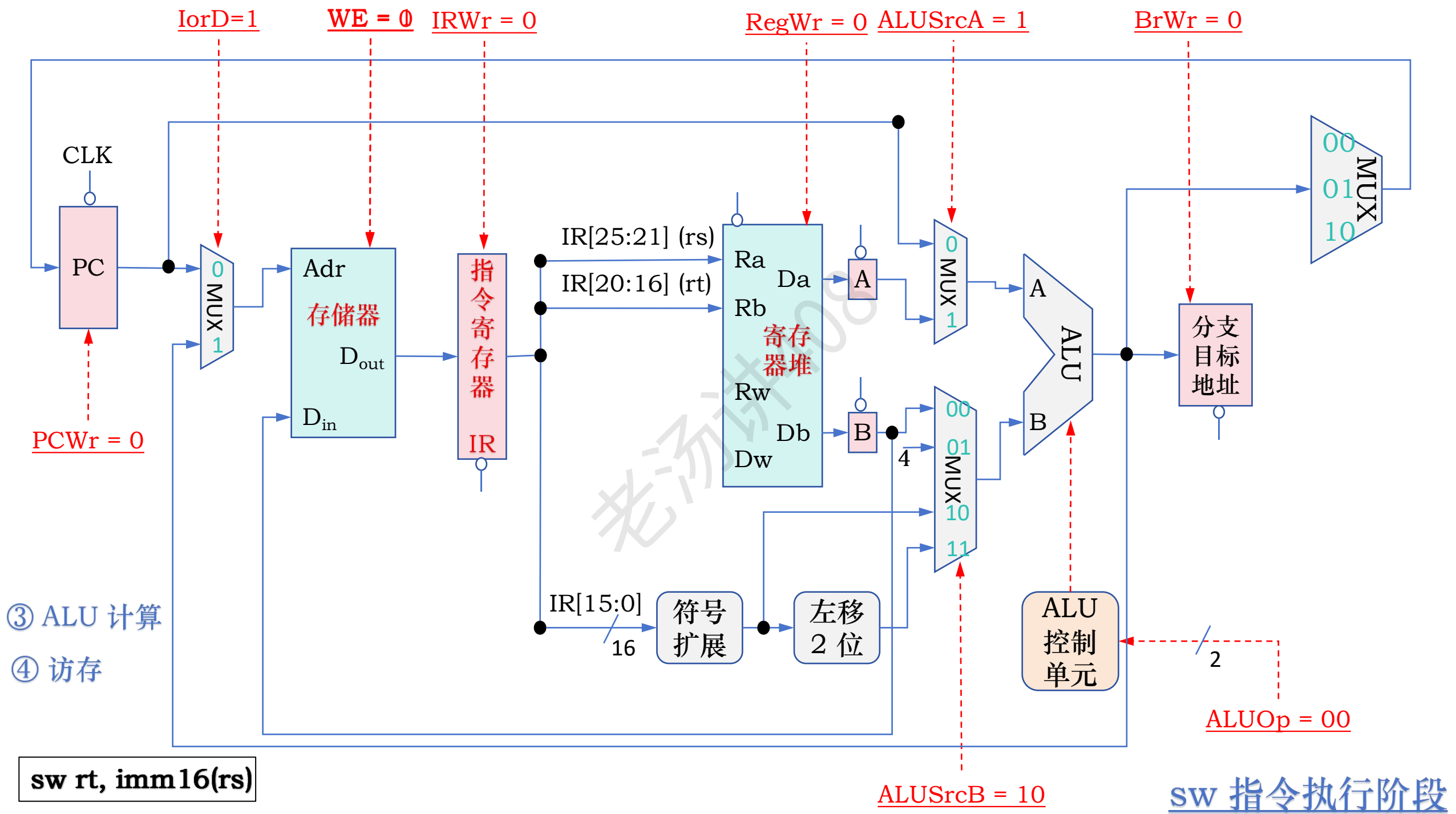
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。



状态: MemAdr

ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

状态: swFinish

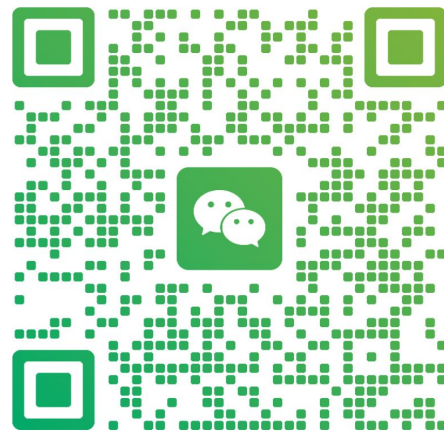
ALUSrcA = 1
ALUSrcB = 10
ALUOp = 00
IorD=1
PCWr = 0
IRWr = 0
WE = 1
RegWr = 0
BrWr = 0
.....

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



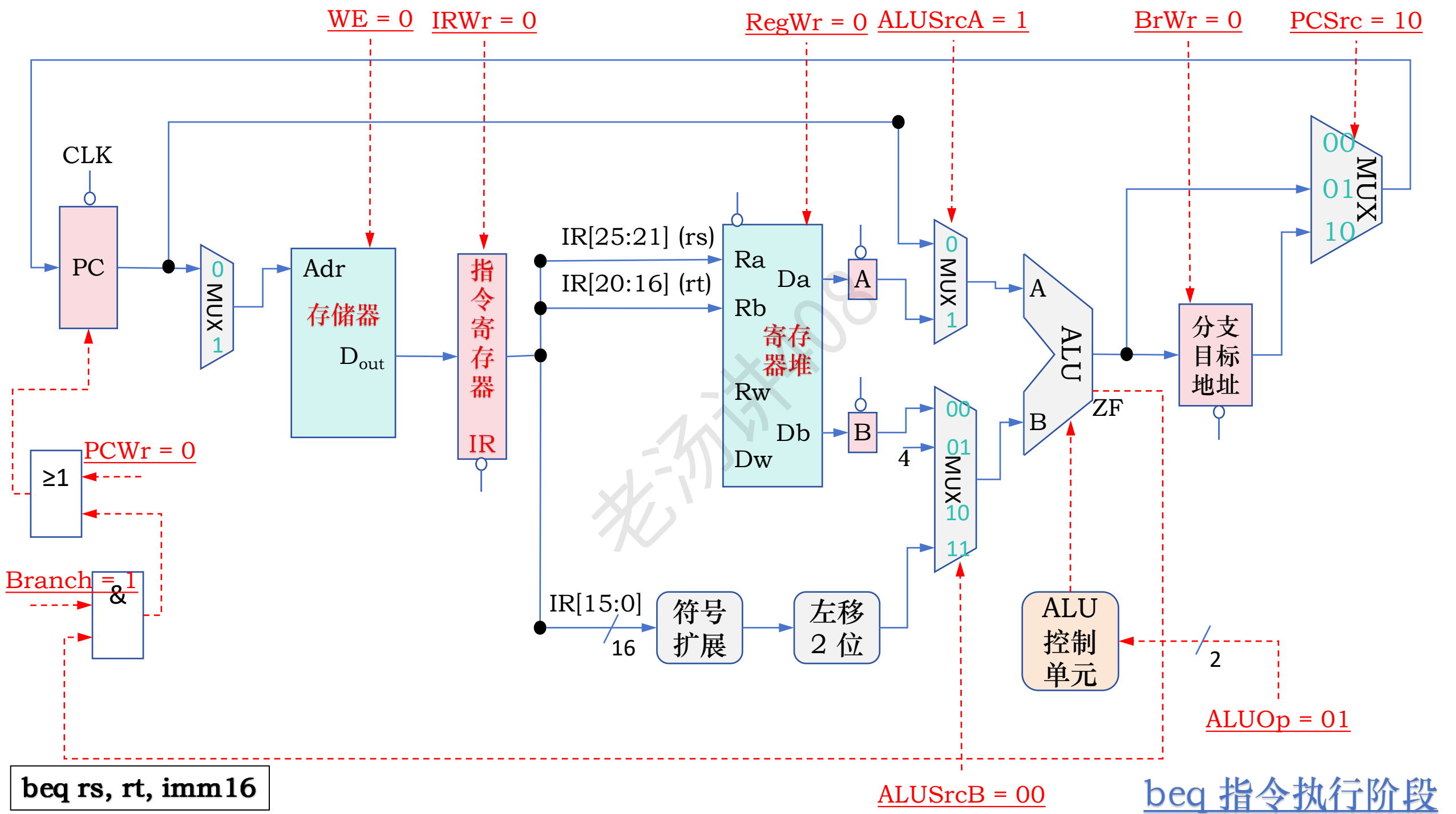
老汤

浙江 杭州



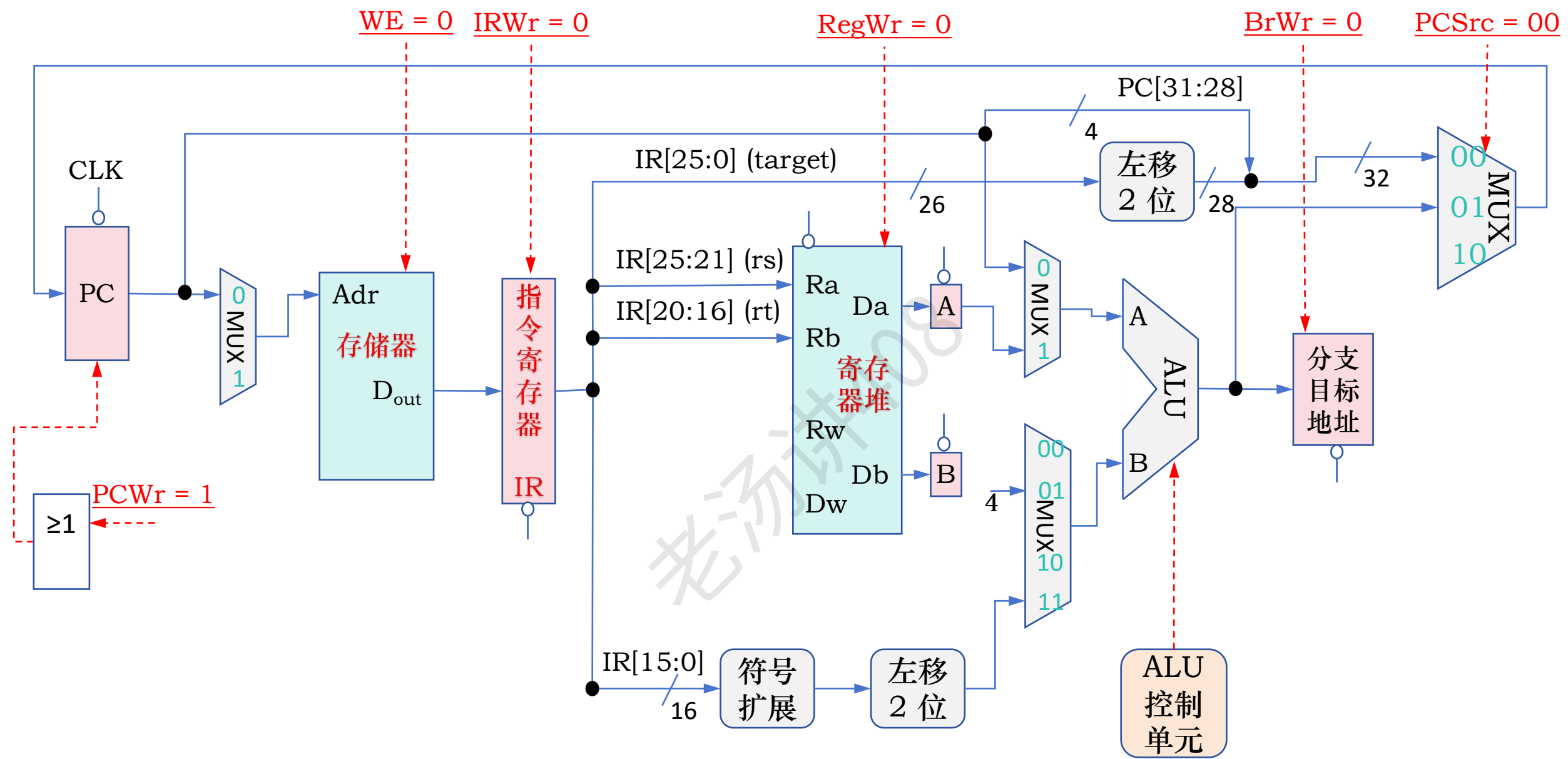
扫一扫上面的二维码图案，加我为朋友。

sw 指令执行阶段



状态: BrFinish

ALUSrcA = 1
ALUSrcB = 00
ALUOp = 10
PCSrc = 10
Branch = 1
PCWr = 0
IRWr = 0
WE = 0
RegWr = 0
BrWr = 0
.....

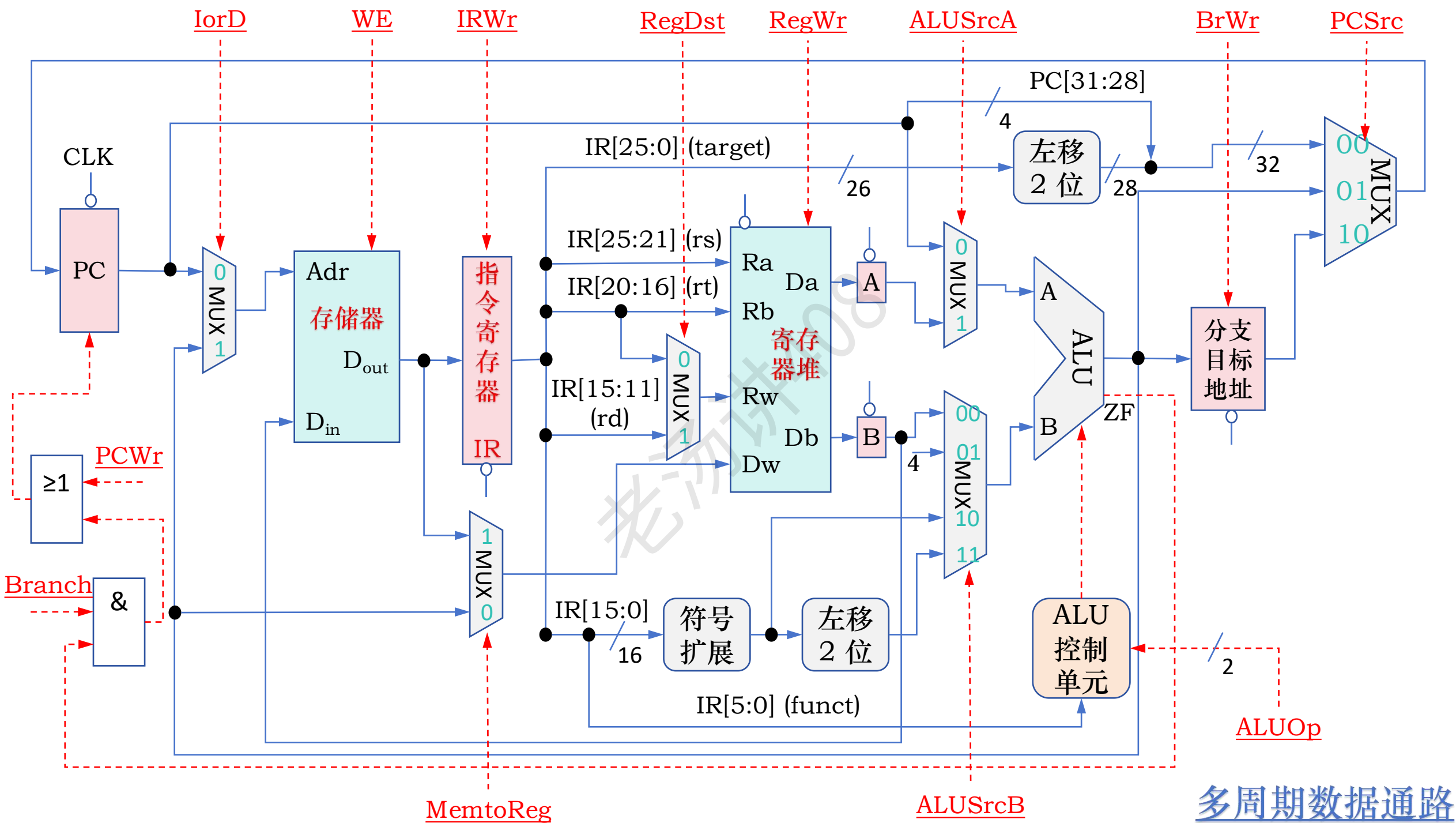


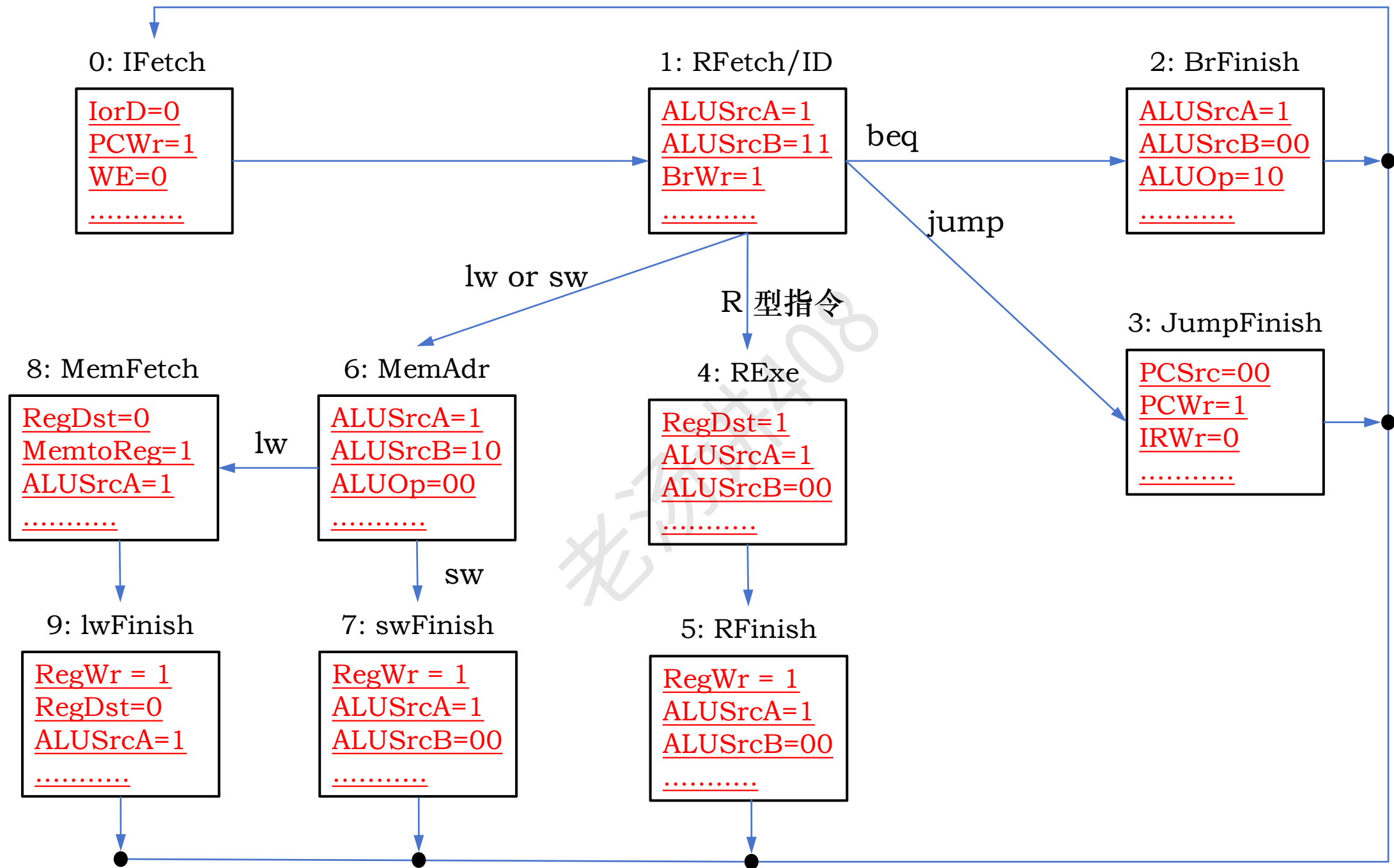
jump target

jump 指令执行阶段

状态: JumpFinish

<u>PCSrc = 00</u>
<u>PCWr = 1</u>
<u>IRWr = 0</u>
<u>WE = 0</u>
<u>RegWr = 0</u>
<u>BrWr = 0</u>
<u>.....</u>





多周期处理器设计：主控制单元

① 硬布线控制器设计

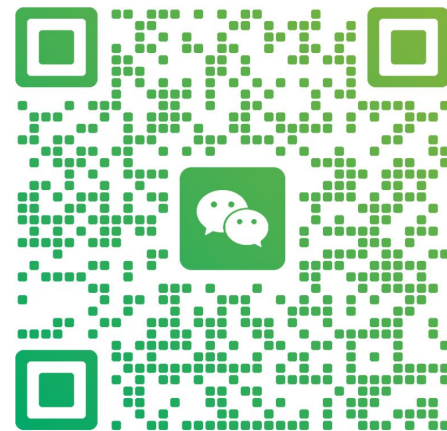
② 微程序控制器设计

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。